YOLOZU Manual (Detailed)

YOLOZU contributors

June 20, 2026

Abstract

YOLOZU is a framework-agnostic evaluation toolkit for vision model outputs. Its stable production lane is predictions validation and evaluation through one predictions interface contract, so teams can compare outputs across training stacks, export paths, and runtime backends without changing the scoring surface. The toolkit supports detection, segmentation, keypoints, monocular depth, and 6DoF pose evaluation, while keeping model training implementations optional and decoupled.

The manual is organized around artifact-first operation: save predictions, validate them early, score them under pinned settings, and record enough metadata to reproduce decisions. Backend parity, benchmark orchestration, SynthGen handoff, continual learning, test-time training, and refinement workflows are documented as secondary, experimental, or research lanes rather than as the default production path.

Support / legal:

- Contact: `develop@toppymicros.com`
- © 2026 ToppyMicroServices OÜ
- Legal address: Karamelli tn 2, 11317 Tallinn, Harju County, Estonia
- Registry code: 16551297

Contents

Part I

User Quickstart

Chapter 1

Overview

This chapter introduces YOLOZU's predictions-first evaluation paradigm, the primary repository entry points, and the manual's readiness labels.
It is designed to make workflow selection quick and reduce ambiguity around command execution, artifact handling, and support boundaries. No prerequisites are required beyond reading. Rely on the `-help` flag for the specific CLI in use, and treat JSON artifacts as the definitive, stable interface.

Who should read this chapter: Start here if you are new to YOLOZU and need the shortest route to the right workflow and CLI surface.

1.1 What YOLOZU Is

YOLOZU is a framework-agnostic evaluation toolkit for vision models. Its primary production lane is predictions validation/evaluation through one stable predictions interface contract. Continual learning and test-time adaptation remain important, but they are not the first production lane for most adopters.

The shortest human summary. If you only remember one sentence from this chapter, let it be this:

> YOLOZU is the part of the stack that keeps model outputs comparable and reproducible even when the underlying training or inference stack changes.

These habits are the manual's operating rules:

- **Operating Rule 1: Save artifacts.** Keep predictions, reports, settings, and provenance with the run.
- **Operating Rule 2: Validate before scoring.** Run schema/interface checks before interpreting metrics.
- **Operating Rule 3: Pin the protocol.** Compare runs only when dataset split, preprocessing, thresholds, and metric settings are explicit.
- **Operating Rule 4: Separate production from research.** Do not let backend, TTT, or refinement differences silently become evaluation drift.

If you are learning this space for the first time. It helps to separate three questions that are often mixed together:

- **How is a model trained?** This belongs to the training stack.
- **How is a model executed?** This belongs to the inference/runtime stack.
- **How are results compared fairly?** This is where YOLOZU is strongest.

Many repositories try to answer all three questions at once. YOLOZU is more useful when you read it the other way around: first understand the evaluation surface, then understand how

exports and runtime backends plug into that surface, and only after that move into continual learning, TTT, or refinement.

What problem this manual is trying to solve. The practical problem is not merely "run inference" but "make results comparable enough that you can trust a decision." In vision work, small changes in preprocessing, runtime, export format, or evaluation protocol can create large differences in reported performance. The manual therefore tries to teach two habits:

- separate *producing predictions* from *judging predictions*,
- keep the judging side pinned and reproducible even when the model side changes.

YOLOZU at a glance.

- Framework-agnostic evaluation toolkit for vision models.
- Primary lane: predictions validation/evaluation across frameworks and runtimes.
- Secondary/research lanes: continual learning, TTT, backend parity, Hessian refinement.
- Optional integration appendices: MCP automation and SynthGen handoff.
- Predictions interface contract (JSON artifacts).
- Multi-task: detection, segmentation, keypoints, monocular depth, and 6DoF pose.
- Deployment + interoperability: ONNX export and runtime backends.
- Automation-ready development: stable CLIs, machine-readable manifests, and regression-testable artifacts.

Continual, online, and TTT in one line. These terms all involve distribution shift, but they should lead to different operational choices:

- **Continual learning**: staged retraining with explicit evaluation gates before checkpoint promotion.
- **Online learning**: live or near-live model updates; highest operational risk and not the default YOLOZU path.
- **TTT**: temporary inference-time adaptation for the current batch or stream; compare before/after artifacts and reset policy.

Read Chapter 14 for continual-learning governance and Chapter 15 for TTT method details.

Where MCP and SynthGen fit. MCP automation and SynthGen handoff are useful integration surfaces, but they are not part of the primary production evaluation lane. Read the core path first: installation, predictions interface contract, validation, evaluation, and benchmark qualification. Use the integration appendices only when you need client automation or synthetic-data intake after that core workflow is clear.

1.2 Readiness Levels

The manual uses three readiness labels:

Label	Meaning	Primary chapters/workflows
Stable	CI-backed default path for normal users.	Installation, predictions interface contract, validation/evaluation, dataset wrappers, CLI entrypoints.
Experimental	Useful but runtime-, environment-, or artifact-dependent.	Backend parity, benchmark orchestration, runtime export paths, SynthGen handoff, macOS/MPS qualification.
Research	Reproduction, ablation, or method exploration; not the production default.	Continual learning, TTT, Hessian/refinement workflows, geometry/depth/6DoF experiments.

Use `docs/production_readiness.md` as the prose source of truth for this classification.

Enterprise-friendly repository posture. YOLOZU keeps the shipped repository code under an Apache-2.0 policy, does not ship built-in telemetry or phone-home style usage collection in its tooling, and relies on explicit quality-control mechanisms such as tests, manifests, CI gates, and provenance/reporting helpers. This improves enterprise deployability, but it does not remove the separate review boundary for datasets, model weights, vendor runtimes, or hosted deployment environments.

1.3 Automation-Oriented Design Intent

This repository is designed for reliable automation and human review. In practical terms, this means:

- Automation clients can use the codebase safely through stable command surfaces and well-defined interface contracts.
- Scripted workflows can add experiments, scripts, or reports without breaking existing interfaces.
- JSON schemas and machine-readable registries are first-class interfaces, not optional metadata.

Operational guidance for human operators and automated workflows follows the operating rules above: artifacts are first-class, interface contracts are validated before scoring, and evaluation protocols remain pinned.

High-level objectives include:

- Generating stable, versioned JSON artifacts for predictions, evaluation reports, and run metadata.
- Ensuring CPU-minimum compatibility for development and testing; GPU acceleration remains optional for training and inference.
- Supporting backend-agnostic workflows across PyTorch, ONNX Runtime, TensorRT, and custom C++/Rust implementations.
- Making migration easier by accepting common dataset ecosystems, such as YOLO-style data.yaml formats and COCO JSON, through lightweight wrapper artifacts that preserve original datasets.
- Enforcing a production-oriented run interface contract for training operations, with standard artifact paths, resume behavior, and model exports.
- Maintaining a strict Apache-2.0 operational posture, ensuring no non-Apache dependencies are present in shipped code paths.
- Avoiding built-in telemetry in shipped tooling; quality control is performed through explicit, reviewable checks instead.

1.4 Navigating the Repository

The primary entry points for navigating the project are:

- `README.md`: Provides a concise overview and essential canonical commands.
- `docs/README.md`: Serves as the documentation index, organized by *task objective*.
- `docs/yolozu_spec.md`: Contains the feature summary and technical specification.
- `tools/`: Houses power-user scripts and the repository's wrapper CLI.

Module path policy. Canonical Python modules are organized in categorized packages such as `yolozu/core`, `yolozu/datasets`, `yolozu/eval`, `yolozu/inference`, `yolozu/predictions`, `yolozu/training`, and `yolozu/geometry`. New code should import these canonical paths directly. Legacy top-level imports are preserved through package-level aliasing for compatibility.

1.5 How to use this manual

Different readers usually arrive with different first questions. A quick routing guide helps:

- **"I want the main production lane."** Start with installation, predictions interface contract, and evaluation/export workflows.
- **"I already have predictions and only need evaluation."** Start with predictions interface contract and evaluation/export workflows.
- **"I need parity or benchmark qualification."** Start with parity and benchmark protocols after installation and evaluation/export workflows.
- **"I care about forgetting, TTT, or research ablations."** Start with the research workflow, continual-learning, and TTT chapters.
- **"I care about geometry, depth, or pose refinement."** Start with the depth/6DoF and TTT/refinement chapters.
- **"I am integrating MCP clients or SynthGen handoff."** Start with the integration appendices after the production evaluation path.
- **"I am modifying tools or documentation."** Start with the tool registry and manifest-driven documentation chapters.

A good first reading order for most users.

1. Chapter 1: understand what kind of repository this is.
2. Chapter 2: get to a working environment.
3. Chapter 5: choose the right evaluate/export workflow.
4. Only then branch into the more specialized chapters.

1.6 What to learn before the advanced chapters

If Chapters 9–16 look heavy at first glance, that is normal. They assume that you already understand the distinction between:

- a **model artifact** (weights, runtime bundle, engine),
- a **predictions artifact** (what the model emitted),
- an **evaluation artifact** (what the scoring pass concluded), and
- a **parity artifact** (what changed between two backends or runs).

Once those four artifact types feel intuitive, the rest of the manual becomes much easier to read:

- backend parity becomes a question of *comparing outputs fairly*,
- continual learning becomes a question of *comparing stages fairly*,
- TTT becomes a question of *comparing before/after behavior fairly*,
- depth and 6DoF chapters become a question of *making geometric assumptions explicit*.

1.7 Foundations and further reading

The end-of-manual References section is not only a formality; it is also a reading map for the ideas behind this repository. A few anchors are especially useful:

- **Evaluation datasets and reporting:** COCO remains a practical reference point for detection and keypoint evaluation vocabulary [14].
- **Runtime backends:** the everyday backend trio in this manual is PyTorch, ONNX Runtime, and TensorRT [21, 18, 19].
- **Continual learning / distillation:** the manual's self-distillation framing is easier to place if you have seen classic distillation work and newer continual-learning variants [8, 3, 23].
- **Test-time training / adaptation:** the TTT chapters build on the general test-time training idea and its entropy-minimization branch [24, 26, 5].

- **Masked-image modeling for adaptation:** the MIM sections borrow intuition from denoising autoencoders, inpainting-style self-supervision, and later masked-image modeling systems [25, 20, 7, 27, 4, 15].
- **Depth and 6DoF pose:** for geometric background, the repository's depth and pose chapters connect to monocular depth estimation and common pose-evaluation practice [2, 22, 28, 9, 10, 12].

If you are reading this manual as an educational document rather than as an operator handbook, this is a good strategy:

1. read the chapter you need for the workflow,
2. note the unfamiliar terms,
3. jump to the References section at the end,
4. then come back to the implementation details.

1.8 Canonical CLI and compatibility wrapper

YOLOZU now defines one supported top-level CLI surface:

- **Canonical CLI**: Invoked via `yolozu ...` or `python3 -m yolozu ...`, this is the supported command interface for both installed and repository-root workflows.
- **Compatibility wrapper**: `python3 tools/yolozu.py ...` remains available in a repo checkout for older scripts, but it should be treated as a legacy shim rather than as a second supported surface.

Direct helper scripts under `tools/` still matter for repo-only workflows, but the supported top-level entrypoint is the canonical `yolozu` CLI.

Rule of thumb. If you are unsure which CLI to start with:

- start with `yolozu ...` (or `python3 -m yolozu ...`),
- reach for direct `tools/*.py` helpers only when a workflow is explicitly documented as repo-only,
- treat `python3 tools/yolozu.py ...` as a backwards-compatibility bridge.

Chapter 2

Installation and Setup

This chapter guides you through establishing a functional YOLOZU environment and validating it via preliminary sanity checks. It ensures the creation of a reproducible toolchain while explicitly detailing backend availability and dependencies. Python 3.10 or newer is mandatory. Optional extras vary based on the selected backend; notably, TensorRT workflows typically necessitate a Linux environment equipped with an NVIDIA GPU.

Who should read this chapter: Read this chapter when you are setting up a fresh environment or checking which extras and runtimes you actually need.

2.1 Python Requirements

YOLOZU targets Python 3.10 and newer.

2.2 Pip Installation (User Path)

For standard usage, the recommended installation procedure is:

```
python3 -m pip install yolozu
yolozu --help
yolozu doctor --output -
```

Stable versus qualified paths. The default stable path is CPU-first install plus predictions evaluation. macOS/MPS should be treated as a **qualification path**: it is supported when `torch.backends.mps.is_available()` returns true, but it is not implied merely because a tool can run on macOS.

2.2.1 First successful run (2 commands: install → smoke)

For the quickest reproducible success path with evaluation output and PNG evidence:

```
python3 -m pip install yolozu
bash scripts/smoke.sh
```

The smoke script uses committed assets under `data/smoke` and writes `reports/smoke_coco_eval_dry_run.json` plus SynthGen smoke artifacts under `reports/` (`smoke_synthgen_summary.json`, `smoke_synthgen_eval.json`, `smoke_synthgen_overlay.png`), and instance-seg overlay PNGs under `reports/smoke_demo_instance_seg/overlays/*.png`.

If your predictions come from another backend, validate first:

```
yolozu validate predictions data/smoke/predictions/predictions_dummy.json --strict
```

2.2.2 First-failure checklist

If the first run fails, do not jump straight into backend debugging. Check the artifact path first, then the optional runtime that produced it. The checklist below gives a symptom, likely cause, and verification command for the common first-run failures; the Troubleshooting chapter expands the same cases into a fuller troubleshooting playbook.

Symptom: `pip install yolozu` fails. Likely cause: Python or the packaging toolchain is too old. Verification command:

```
python3 --version
python3 -m pip --version
```

Symptom: missing import after install. Likely cause: the workflow needs an optional extra. Verification command:

```
python3 -m pip install 'yolozu[demo]'
python3 -m pip install 'yolozu[onnxrt]'
```

Symptom: COCO metrics look shifted. Likely cause: `category_id` values do not match the evaluator class mapping. Verification command:

```
yolozu validate predictions <predictions.json> --strict
```

Symptom: predictions schema error. Likely cause: required fields, paths, or task-specific payloads are missing. Verification command:

```
yolozu validate predictions <predictions.json> --strict
```

Symptom: ONNX Runtime model-load error. Likely cause: ORT cannot load the exported ONNX IR/opset or runtime build. Verification command:

```
python3 -c "import onnxruntime as ort; print(ort.__version__)"
```

Symptom: TensorRT command fails locally. Likely cause: TensorRT needs a compatible Linux/NVIDIA/CUDA stack. Verification command:

```
python3 tools/gpu_validation_preflight.py --help
```

Symptom: metrics changed after protocol edit. Likely cause: evaluation thresholds, class order, score filtering, NMS, or protocol pins changed. Verification command:

```
yolozu eval-suite --help
```

Then inspect the saved suite/report protocol fields.

Optional extras let you install only the dependencies you need for a given workflow:

```
python3 -m pip install 'yolozu[demo]'    # PyTorch demonstration utilities (includes
    timm + opencv-contrib + transformers for depth demo)
python3 -m pip install 'yolozu[onnxrt]'  # ONNX Runtime tooling
python3 -m pip install 'yolozu[coco]'    # COCOeval support
python3 -m pip install 'yolozu[full]'    # Comprehensive installation
```

If you are operating from a source checkout (editable install), extras use the dot-prefix form:

```
python3 -m pip install -e '.[demo]'
```

2.2.3 Optional demos (CPU)

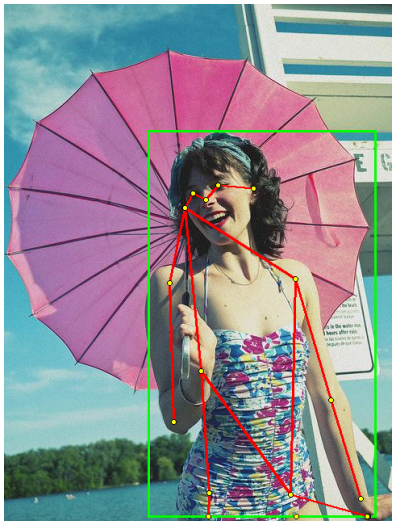
The demo commands are intentionally designed to keep the invocation short:

```
python3 -m pip install -U 'yolozu[demo]'
yolozu demo
yolozu demo instance-seg
yolozu demo instance-seg-tta
yolozu demo instance-seg --background yolo-bbox --yolo-root data/smoke --yolo-split
    val --inference none
yolozu demo keypoints
yolozu demo pose
yolozu demo pose --backend aruco  # cached sample in demo_output/pose/_samples
yolozu demo pose --backend densefusion
yolozu demo depth
yolozu demo depth --compare
yolozu demo train  # downloads ResNet18 on first run
yolozu demo continual --compare --markdown
```

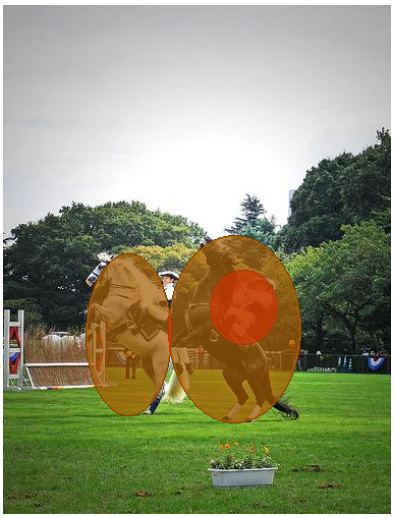
Note: `yolozu demo depth` defaults to Depth Anything (via Transformers) and can also run MiDaS/DPT. Both paths download weights on the first run.

2.2.4 Real-image showcase

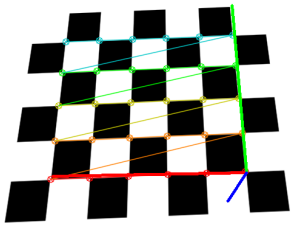
For compact visual proof that the demo suite produces useful artifacts on real images, the manual shows one artifact for each advertised task:



(a) `yolozu demo keypoints`



(b) `yolozu demo instance-seg`



(c) `yolozu demo pose -backend aruco`

Figure 2.1: Real-image outputs from the keypoints, instance segmentation, and 6DoF pose demos. The tasks differ, but all stay aligned with the predictions interface contract mindset.

The corresponding commands are:

```
python3 -m pip install -U 'yolozu[demo]'
yolozu demo keypoints
python3 scripts/download_coco_instances_tiny.py --out-root data/coco --split val2017 --
    num-images 8 --seed 0
yolozu demo instance-seg --background coco-instances --inference auto --num-images 1 --
    max-instances 8 --score-threshold 0.25
yolozu demo pose --backend aruco
```

To enable the COCO instances (polygon) background for the instance-seg demo without long flags:

```
python3 scripts/download_coco_instances_tiny.py
yolozu demo instance-seg
```

To run a visible raw-versus-TTA comparison on a corrupted real COCO image:

```
yolozu demo instance-seg-tta --run-dir reports/demo_instance_seg_tta
```

This writes `reports/demo_instance_seg_tta/selected/overlay_raw.png`, `reports/demo_instance_seg_tta/selected/overlay_tta.png`, and `reports/demo_instance_seg_tta/instance_seg_tta_demo_report.json`.

2.3 Source Checkout (Repository Path)

If you are operating directly within the repository (e.g., utilizing the training stack or power-user scripts), follow this procedure:

```
python3 -m pip install -r requirements-test.txt
python3 -m pip install -e .
python3 -m pytest -q
```

2.4 Recommended Deployment Flow

The canonical deployment pipeline follows this progression:

PyTorch → ONNX → TensorRT

On macOS, you can typically run CPU-bound tooling (validation, evaluation, and ONNX Runtime CPU export). Treat MPS as available only when the local Torch runtime reports success for `torch.backends.mps.is_available()`. If that check is false, treat the machine as CPU-only for YOLOZU purposes. Python itself can still create the environment with `venv` and `pip`; when MPS does not come up, the usual blocker is the Torch wheel/runtime combination rather than Python environment creation. Miniforge/conda is therefore a fallback for some Apple Silicon hosts, not a hard requirement. TensorRT execution is generally Linux/NVIDIA-centric. TensorRT workflows are covered in the Real-Time vs. Batch Inference chapter. The canonical pipeline strictly follows PyTorch → ONNX → TensorRT and includes parity checks at each stage.

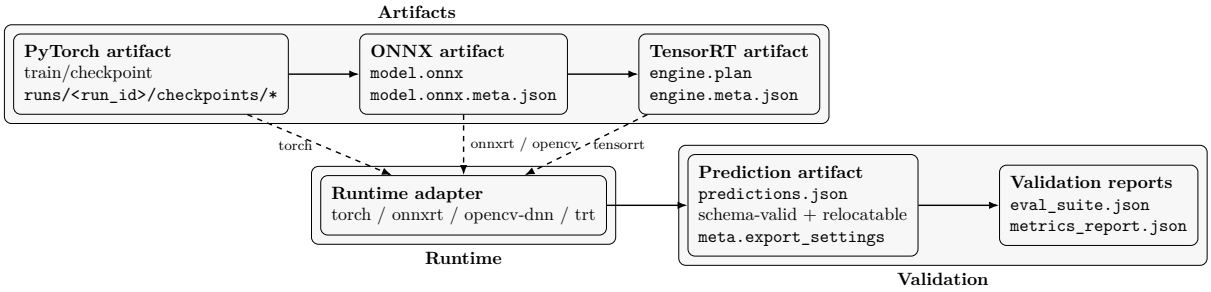


Figure 2.2: Recommended deployment flow, grouped by artifact creation, runtime adaptation, and validation evidence.

Chapter 3

Core Concepts and Interface Contracts

This chapter elucidates the core interface contracts (schemas) and the dataset and prediction migration methodologies employed throughout the repository.
It ensures evaluation stability across diverse backends and over time by mandating explicit, versionable inputs and outputs.
This documentation assumes adherence to the specified schemas and treats generated wrapper artifacts (e.g., `dataset.json`) as integral components of the workflow.

Who should read this chapter: Read this chapter if you need to understand the predictions interface contract, dataset wrappers, and why YOLOZU treats artifacts as first-class.

3.1 Contract-First Evaluation

The interface contracts (schemas) are the foundational product of this repository. They empower you to:

- Conduct equitable comparisons across disparate inference backends.
- Guarantee the reproducibility of evaluations over time.
- Maintain tooling stability, even as underlying models undergo architectural evolution.

Core interface contracts include:

- **Predictions Schema**: `docs/predictions_schema.md`
- **Adapter interface contract**: `docs/adapter_contract.md`
- **Training run interface contract (Run Contract)**: `docs/run_contract.md`
- **SynthGen intake interface contract**: `docs/synthgen_contract.md`

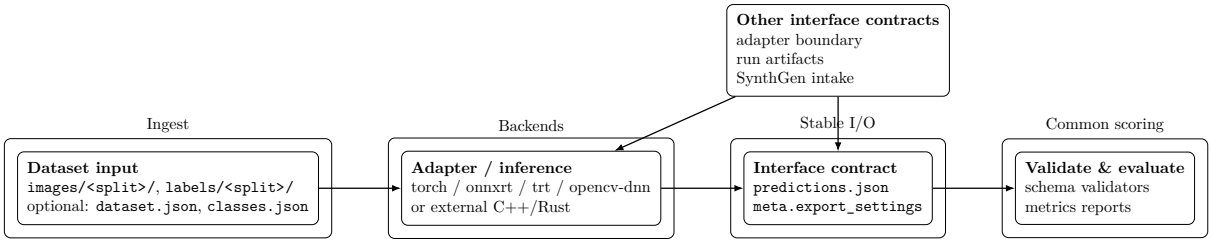


Figure 3.1: YOLOZU uses stable interface contracts to decouple inference backends from evaluation.

When links and prose disagree, treat these interface contract documents as canonical and re-verify against `tools/manifest.json` and tool `-help` output.

3.2 Datasets

The majority of tools anticipate a YOLO-style dataset hierarchy (comprising `images/`, `labels/`, and splits), supplemented by optional metadata. Supported image formats: JPEG (`.jpg`, `.jpeg`), PNG, BMP, TIFF (`.tif`, `.tiff`), WebP, and GIF.

3.2.1 Frictionless Migration (Utilizing Datasets "As-Is")

YOLOZU is engineered to evaluate prevalent training ecosystems with minimal friction. The central paradigm is to circumvent manual dataset restructuring by generating lightweight **descriptor artifacts**.

In practice, migration commands yield:

- A concise `dataset.json` wrapper (which references existing images and labels), and/or
- Converted label files (e.g., translating COCO JSON to YOLO text labels) accompanied by a wrapper.

The overarching objective is to preserve the integrity of source datasets while rendering evaluation inputs explicit and versionable.

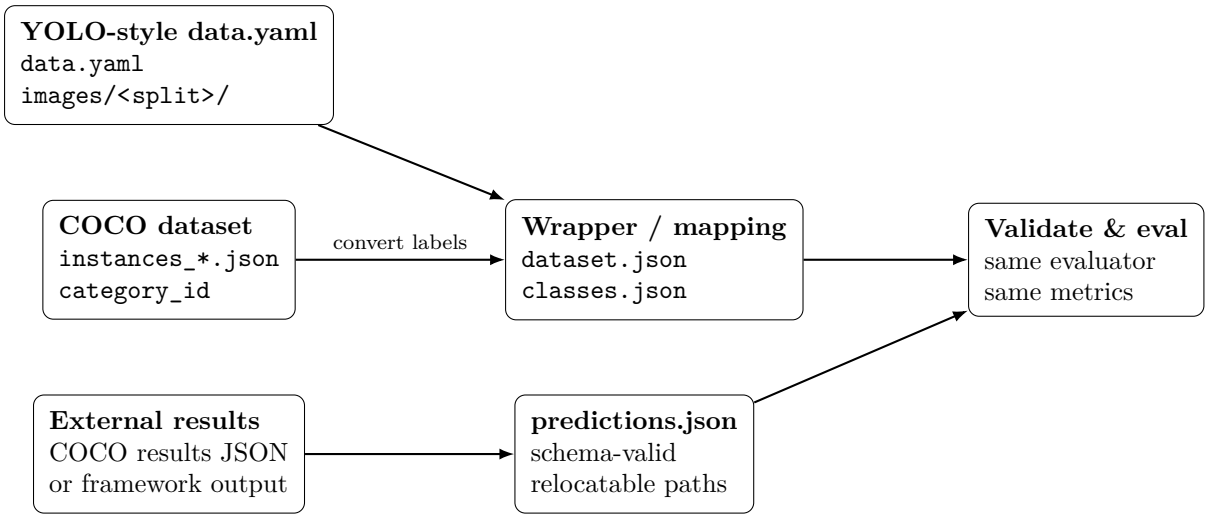


Figure 3.2: Migration map: keep native training ecosystems, convert only the interface boundary artifacts.

3.2.2 YOLO-style data.yaml Datasets (YOLOv8/YOLO11)

If your dataset is already defined by a YOLO-style `data.yaml` (referencing `images/<split>/` and `labels/<split>/`), YOLOZU can generate a lightweight `dataset.json` wrapper, leaving the original dataset entirely unmodified.

For precise command syntax and validation procedures, refer to Chapter 5 (Workflow D: Migrate Datasets and Predictions).

3.2.3 COCO JSON Datasets (Migration)

Numerous ecosystems rely on COCO-style `instances_*.json` files for object detection. Because YOLOZU consumes YOLO-style text labels, the supported migration trajectory is: COCO JSON → YOLO labels + `dataset.json` wrapper.

For the specific conversion command, consult Chapter 5 (Workflow D: Migrate Datasets and Predictions).

3.2.4 Importing Predictions from External Ecosystems

Frameworks such as Detectron2, MMDetection, and YOLOX frequently export detection results in COCO format. Typical fields include:

- `image_id`
- `category_id`
- `box`
- `score`

YOLOZU seamlessly converts these outputs into a compliant `predictions.json`:

```
yolozu migrate predictions \
  --from coco-results \
  --results /path/to/coco_results.json \
  --instances /path/to/instances_val2017.json \
  --output reports/predictions.json \
  --force
```

Following conversion, proceed with standard validation and evaluation.

3.2.5 External SynthGen intake contract

For synthetic data pipelines, YOLOZU intentionally limits scope to **intake and evaluation**; generation remains in an external repository. The intake boundary is fixed by `docs/synthgen_contract.md` and `schemas/synthgen_sample.schema.json`, with required fields such as `image`, `depth_ndc`, `inst_id`, `sem_id`, and `kpts2d`.

Validate shard metadata before training/evaluation:

```
python3 tools/validate_synthgen_contract.py \
  --input /path/to/synthgen_dataset/shards/train_000.jsonl \
  --max-samples 200
```

3.3 Predictions JSON

A predictions artifact typically manifests as a JSON array containing per-image objects. While the precise structure varies by task (e.g., bounding box, segmentation, pose estimation), the guiding principles remain constant:

- **Stability**: Keys and data types are strictly schema-validated.
- **Portability**: File paths are relative and relocatable wherever feasible.
- **Traceability**: Metadata (such as run ID, Git SHA, and environment details) is embedded alongside the results.

3.3.1 Minimal task examples

Use these examples as shape checks, not as good predictions. They show the smallest artifact that a downstream evaluator can understand, plus the fields that actually affect scoring.

Detection

```
{
  "predictions": [
    {
      "schema_version": 2,
      "image": "images/val/img_0001.jpg",
      "image_w": 640,
      "image_h": 480,
      "detections": [
        {
```

```
          "class_id": 0,
          "score": 0.91,
          "bbox": {"cx": 0.50, "cy": 0.48, "w": 0.20, "h": 0.16}
        }
      ]
    }
  ],
  "meta": {
    "task": "detect",
    "export_settings": {"bbox_format": "cxcywh_norm"}
  }
}
```

- Required: `predictions`, per-entry `image`, and each detection's `score` plus `bbox`.
- Usually required for evaluation: `class_id`, `image_w`, and `image_h`.
- Protocol metadata should pin `meta.export_settings`.
- Scoring uses: class ID, score order, box coordinates, dataset labels, and the chosen evaluation protocol.

Instance segmentation

```
{
  "schema_version": 1,
  "predictions": [
    {
      "image": "images/val/img_0001.jpg",
      "instances": [
        {
          "class_id": 0,
          "score": 0.88,
          "mask": "pred_masks/img_0001_inst0.png"
        }
      ]
    }
  ],
  "meta": {"task": "instance_segmentation"}
}
```

- Required: per-entry `image`, `instances`, and each instance's `class_id` plus `mask`.
- Optional but normally useful: `score`; without it, ranking-sensitive AP summaries are less informative.
- Scoring uses: mask pixels, class ID, score order, GT masks, and the mask loading policy.

Keypoints

```
{
  "predictions": [
    {
      "schema_version": 2,
      "image": "images/val/person_0001.jpg",
      "image_w": 640,
      "image_h": 480,
      "detections": [
        {
          "class_id": 0,
          "score": 0.93,
          "bbox": {"cx": 0.52, "cy": 0.50, "w": 0.28, "h": 0.62},
          "keypoints": [
            {"x": 0.50, "y": 0.22, "v": 2},
            {"x": 0.54, "y": 0.24, "v": 2}
          ]
        }
      ]
```

```
    }
  ],
  "meta": {"task": "keypoints", "keypoints_format": "xy_norm"}
}
```

- Required by the base predictions interface contract: the same detection fields as detection.
- Required for keypoint scoring: `keypoints`; visibility `v` is optional but recommended when the dataset uses visible/labeled states.
- Scoring uses: detection matching by IoU, keypoint coordinates, visibility, bbox-normalized PCK, and optional OKS settings.

Depth artifact Depth evaluation compares a predicted depth map with a GT depth map. In the benchmark lane, depth is therefore represented as an artifact-backed depth record rather than as a list of boxes:

```
{
  "schema_version": 1,
  "kind": "benchmark_depth_predictions_artifact",
  "format": "onnx",
  "status": "reference_artifact",
  "source_path": "reports/depth/pred_depth.npy",
  "shape": [480, 640],
  "dtype": "float32",
  "sha256": "..."
}
```

- Required: `source_path`; benchmark reports also record `shape`, `dtype`, and `sha256` for reproducibility.
- Optional but useful: `min`, `max`, and `run_meta`.
- Depth reports should also record the alignment mode used by `eval_depth.py`.
- Scoring uses: predicted depth pixels, GT depth pixels, optional valid-pixel mask, scale factors, and alignment policy.

6DoF pose

```
{
  "predictions": [
    {
      "schema_version": 2,
      "image": "images/val/pose_0001.jpg",
      "image_w": 640,
      "image_h": 480,
      "detections": [
        {
          "class_id": 2,
          "score": 0.86,
          "bbox": {"cx": 0.46, "cy": 0.55, "w": 0.18, "h": 0.22},
          "rot6d": [1.0, 0.0, 0.0, 0.0, 1.0, 0.0],
          "log_z": 0.18,
          "offsets": [0.01, -0.02]
        }
      ]
    }
  ],
  "meta": {"task": "pose6d", "depth_unit": "metric"}
}
```

- Required by the base predictions interface contract: the same detection fields as detection.
- Required for pose scoring: `rot6d` plus either recoverable depth fields such as `log_z` and `offsets`, or an explicit translation field when the producing backend writes one.

- Scoring uses: detection matching by IoU, rotation error, translation/depth error, optional CAD-point ADD/ADDS, and success thresholds.

To rigorously validate a predictions artifact:

```
python3 tools/validate_predictions.py /path/to/predictions.json --strict
```

3.3.2 Metric glossary

mAP Mean Average Precision. It summarizes precision-recall quality after sorting predictions by score.

mAP50 AP at IoU 0.50. Useful for quick detection sanity checks, but less strict than multi-threshold AP.

mAP50-95 COCO-style AP averaged across IoU thresholds from 0.50 to 0.95. This is stricter about box or mask localization.

IoU Intersection over Union. The overlap between predicted and GT boxes or masks divided by their union.

NMS Non-Maximum Suppression. A postprocess that removes overlapping detections. Protocols must say whether NMS already happened before evaluation.

OKS Object Keypoint Similarity. COCO's keypoint match score; it scales keypoint error by object size and keypoint-specific sigmas.

PCK Percentage of Correct Keypoints. YOLOZU's lightweight keypoint score uses a distance threshold normalized by object size.

ADD Average Distance of Model Points. A 6DoF pose metric using fixed point correspondences.

ADDS Symmetry-aware ADD variant that matches each transformed point to the nearest GT point.

abs_rel Mean absolute relative depth error: `|pred - gt| / gt`.

RMSE Root mean squared depth error. It penalizes large depth mistakes more strongly than mean absolute error.

delta1 Depth threshold accuracy: the fraction of pixels where predicted and GT depth are within a fixed ratio threshold.

calibration Whether confidence scores behave like probabilities. A calibrated score of 0.8 should be correct about 80 percent of the time on similar data.

parity A backend agreement check. It asks whether two runtimes or formats produce equivalent artifacts under pinned tolerances.

protocol pinning Recording dataset split, preprocessing, thresholds, NMS state, metric family, and tool version so future comparisons remain fair.

3.4 Adapters (Bring-Your-Own Inference)

An adapter functions as a thin interface layer bridging a specific inference implementation and YOLOZU's standardized interface contracts. If your system can generate a compliant `predictions.json`, it can be evaluated.

In practical terms, this entails:

- **Input**: Dataset records supply an `image` path (and optional label/sidecar metadata).
- **Output**: Per-image `detections` must include `class_id`, `score`, and `box` (typically formatted as `cxcywh_norm`), alongside any task-specific optional fields.
- **Validation**: It is imperative to execute `tools/validate_predictions.py -strict` prior to evaluation.

3.5 Reports and Reproducibility

The majority of workflows are architected to emit:

- Machine-readable JSON reports.
- Optional HTML overlays for visual debugging.
- Stable artifact nomenclature within a designated run directory.

This design is highly intentional: it streamlines regression testing and facilitates effortless cross-run comparisons.

Chapter 4

CLI Reference (Practical)

This chapter provides a concise summary of the most frequently utilized commands and demonstrates how to explore the complete Command Line Interface (CLI) surface via the `-help` flag. It is designed to minimize the time to first execution and to clearly delineate which commands are install-safe versus those restricted to repository-only workflows. Utilize `yolozu -help` or `python3 -m yolozu -help` for the supported CLI surface. The legacy `python3 tools/yolozu.py -help` wrapper remains available in a repo checkout for backwards compatibility only. Because implementation details evolve, the `-help` output remains the definitive source of truth.

Who should read this chapter: Read this chapter when you want copy-paste entrypoints first and detailed tool discovery second.

4.1 Design Philosophy

The CLI surface now has one supported top-level entrypoint plus a compatibility shim:

- `yolozu` / `python3 -m yolozu`: the supported top-level CLI.
- `python3 tools/yolozu.py`: a legacy compatibility wrapper for older repo-local scripts.

This chapter documents the *high-traffic commands* on the supported surface and points directly to repo-only helper scripts when a workflow does not belong on the top-level CLI.

4.2 Discoverability

As a best practice, always initiate discovery with:
`yolozu -help` and `yolozu <command> -help`.
In a source checkout, `python3 -m yolozu -help` is equivalent. The legacy `python3 tools/yolozu.py -help` wrapper is compatibility-only.

Manual drift is guarded by `python3 tools/audit_manual_cli_drift.py`. When the top-level CLI changes, update this chapter and only extend `docs/manual_cli_drift_allowlist.json` for intentional prose placeholders or repo-only helpers, not for real `yolozu` commands.

4.3 Common Commands

Command	When to use	Minimal command	Primary output	Repo?
`yolozu doctor`	Check environment and proof the CPU eval path.	`yolozu doctor -proof`	diagnostics plus proof report	No
`yolozu guide`	Pick the next beginner-safe route.	`yolozu guide -goal first-run`	route map with PNG demo output	No

`bash scripts/smoke.sh`	First offline acceptance run from repo assets.	`bash scripts/smoke.sh`	`reports/smoke_coco_eval_dry_run.json` and PNG evidence	Yes
`yolozu demo`	Run the bundled CPU-friendly demo suite.	`yolozu demo`	`demo_output/`	No
`yolozu demo instance-seg`	Produce a quick instance-seg overlay.	`yolozu demo instance-seg -run-dir reports/quickstart_instance_seg -progress`	report plus PNG overlays	No
`yolozu demo keypoints`	Produce a keypoint overlay.	`yolozu demo keypoints`	`demo_output/keypoints/`	No
`yolozu demo pose`	Produce a 6DoF pose overlay.	`yolozu demo pose -backend aruco`	`demo_output/pose/`	No
`yolozu demo depth`	Produce a monocular depth visualization.	`yolozu demo depth`	`demo_output/depth/`	No
`yolozu validate predictions`	Validate a predictions artifact before scoring.	`yolozu validate predictions -help`	validation result	No
`yolozu eval-coco`	Score detection predictions with COCO-style metrics.	`yolozu eval-coco -help`	metrics JSON	No
`yolozu benchmark`	Compare benchmark/parity lanes under pinned settings.	`yolozu benchmark -help`	benchmark report JSON	No
`yolozu parity`	Compare two predictions artifacts for drift.	`yolozu parity -help`	parity JSON	No
`yolozu train`	Run config-driven training or external training wrappers.	`yolozu train <train_contract.yaml> -help`	run bundle under `runs/`	No
`yolozu train-orchestrate`	Plan or execute a small multi-backend training batch.	`yolozu train-orchestrate -help`	`training_orchestration_report.json`	No
`bash scripts/ttt_compare.sh`	Compare baseline vs adapted TTT outputs.	`bash scripts/ttt_compare.sh -help`	compare JSON/Markdown	Yes
`python3 tools/eval_pose.py`	Repo-only pose metric helper.	`python3 tools/eval_pose.py -help`	pose metrics JSON	Yes
`python3 tools/eval_depth.py`	Repo-only depth metric helper.	`python3 tools/eval_depth.py -help`	depth metrics JSON	Yes
`python3 tools/validate_tool_manifest.py`	Validate the machine-readable tool registry.	`python3 tools/validate_tool_manifest.py -manifest tools/manifest.json -require-declarative`	validation result	Yes
`python3 tools/audit_manual_cli_drift.py`	Check this chapter against CLI help surfaces.	`python3 tools/audit_manual_cli_drift.py -help`	drift audit result	Yes
`bash release.sh`	Run release automation from a maintainer checkout.	`bash release.sh -help`	tag/release handoff	Yes

Where did the long command descriptions go? This table is intentionally an entry-point map, not the full CLI reference. Use `docs/generated/cli_reference.md` for the generated command and manifest surface. For complete option lists, use `-help`. For repository helper metadata, use `tools/manifest.json`. When this chapter changes, run `python3 tools/audit_manual_cli_drift.py` so the manual stays aligned with the implemented CLI surface.

4.4 Demo Output Examples (Images)

The `yolozu demo *` commands are designed to produce concrete artifacts (JSON reports and overlay images) that help users verify they can run end-to-end workflows.

The figures below are committed snapshots generated from `data/smoke` using the following commands:

```
yolozu demo instance-seg \
  --background yolo-bbox --yolo-root data/smoke --yolo-split val --inference none \
  --num-images 1 --image-size 320 --max-instances 3 \
  --run-dir demo_output/manual/instance_seg

yolozu demo keypoints \
  --image data/smoke/images/val/000000000036.jpg --device cpu --max-persons 1 \
  --run-dir demo_output/manual/keypoints

yolozu demo depth \
  --image data/smoke/images/val/000000000036.jpg --device cpu --model midas_small \
  --run-dir demo_output/manual/depth

yolozu demo pose \
  --backend chessboard --sample-source synthetic \
  --run-dir demo_output/manual/pose
```

4.4.1 Long-tail recipe plugin options (PyTorch)

`yolozu long-tail-recipe` supports explicit plugin choices for loss, validation metric, and learning-rate scheduler.

```
yolozu long-tail-recipe \
  --dataset data/smoke --split val \
  --loss-plugin torch_cross_entropy \
  --metric-plugin torch_top1_accuracy \
  --lr-scheduler torch_step_lr \
  --output reports/long_tail_recipe_torch.json --force
```

Primary plugin flags:

- `-loss-plugin`: `none`, `focal`, `ldam`, `torch_cross_entropy`, `torch_nll_loss`, `torch_bce_with_logits`
- `-metric-plugin`: `none`, `torch_top1_accuracy`, `torch_top5_accuracy`, `torch_cross_entropy`
- `-lr-scheduler`: `none`, `torch_step_lr`, `torch_cosine_annealing_lr`, `torch_reduce_on_plateau`, `torch_one_cycle_lr`

4.5 YAML Config-Driven Usage (Train/Test/Resume)

YOLOZU supports the ingestion of a YAML or JSON configuration file as its primary positional argument. Internally, `yolozu train` delegates the `-config <file>` argument to `rtdetr_pose.train_minimal`, whereas `yolozu test` maps YAML keys directly to `yolozu.scenarios_cli` arguments.

Use Case	YAML File	Command Pattern

Train (Default)	configs/examples/train_setting.yaml	yolozu train <train_setting.yaml>
Train (Contract)	configs/examples/train_contract.yaml	yolozu train <train_contract.yaml> -run-id <id>
Resume	Same as Train (Contract)	yolozu train <train_contract.yaml> -run-id <id> -resume
Train (External YOLOX)	configs/examples/finetune_external/yolox_s_finetune_smoke.py	yolozu train -external-backend yolox <exp.py> -dataset <root> -split <split> -dry-run
Train (External Detectron2)	configs/examples/finetune_external/detectron2_finetune_smoke.yaml	yolozu train -external-backend detectron2 <config.yaml> -dataset <root> -split <split> -task-family bbox|segmentation|keypoints -dry-run
Train (External MMDetection)	configs/examples/finetune_external/mmdetection_finetune_smoke.py	yolozu train -external-backend mmdetection <config.py> -dataset <root> -split <split> -task-family bbox|segmentation -dry-run
Train (External MMPose)	configs/examples/finetune_external/mmpose_finetune_smoke.py	yolozu train -external-backend mmpose <config.py> -dataset <root> -split <split> -dry-run
Train (External MMSeg)	configs/examples/finetune_external/mmseg_finetune_smoke.py	yolozu train -external-backend mmseg <config.py> -dataset <root> -split <split> -dry-run
Train (External TAO)	configs/examples/finetune_external/tao_finetune_smoke.yaml	yolozu train -external-backend tao <config.yaml> -dataset <root> -split <split> -task-family bbox|segmentation|keypoints -dry-run
Train (Optional Ultralytics)	yolo11n.pt	yolozu train -external-backend ultralytics <model.pt> -dataset <root> -split <split> -dry-run
Train (Optional HF DETR)	facebook/detr-resnet-50	yolozu train -external-backend hf-detr <model-id> -dataset <root> -split <split> -dry-run
Train orchestration	reports/train_orchestration_spec.json	yolozu train-orchestrate -spec <spec.json> -output reports/training_orchestration_report.json [-registry-out reports/training_registry.jsonl]
Test (Scenario)	configs/examples/test_setting.yaml	yolozu test <test_setting.yaml> [test_args...]

Concrete execution examples:

```
yolozu train configs/examples/train_setting.yaml
yolozu train configs/examples/train_contract.yaml --run-id exp01
yolozu train configs/examples/train_contract.yaml --run-id exp01 --resume
yolozu train --external-backend yolox configs/examples/finetune_external/
    yolox_s_finetune_smoke.py --dataset data/smoke --split val --dry-run
yolozu train --external-backend detectron2 configs/examples/finetune_external/
    detectron2_finetune_smoke.yaml --dataset data/smoke --split val --task-family bbox
     --dry-run
```

Figure 4.1: Instance-seg demo overlay (`-background yolo-bbox -inference none`). This offline mode uses GT-derived smoke predictions for visualization and evaluation smoke checks.

Figure 4.2: Keypoints demo overlay (torchvision Keypoint R-CNN).

Figure 4.3: Depth demo overlay (MiDaS small; relative depth visualization).

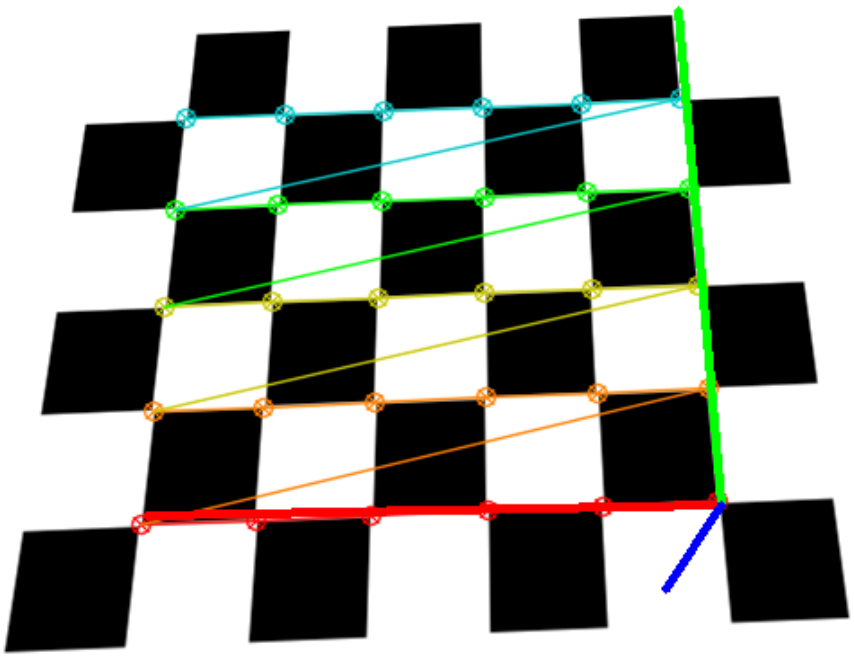


Figure 4.4: Pose6D demo overlay (chessboard + OpenCV solvePnP).

```
yolozu train --external-backend mmdetection configs/examples/finetune_external/
    mmdetection_finetune_smoke.py --dataset data/smoke --split val --task-family bbox
    --dry-run
yolozu train --external-backend mmpose configs/examples/finetune_external/
    mmpose_finetune_smoke.py --dataset data/smoke --split val --dry-run
yolozu train --external-backend mmseg configs/examples/finetune_external/
    mmseg_finetune_smoke.py --dataset data/smoke --split val --dry-run
yolozu train --external-backend tao configs/examples/finetune_external/
    tao_finetune_smoke.yaml --dataset data/smoke --split val --task-family bbox --dry-
    run
yolozu train --external-backend ultralytics yolo11n.pt --dataset data/smoke --split
    val --dry-run
yolozu train --external-backend hf-detr facebook/detr-resnet-50 --dataset data/smoke --
    split val --dry-run
yolozu train-orchestrate --spec reports/train_orchestration_spec.json --output reports
    /training_orchestration_report.json
yolozu train-orchestrate --spec reports/train_orchestration_spec.json --output reports
    /training_orchestration_report.json --registry-out reports/training_registry.jsonl
     --execute
yolozu test configs/examples/test_setting.yaml --adapter precomputed --predictions
    reports/predictions.json
```

Backbone substitutions are governed exclusively by the model configuration JSON, rather than by modifying adapter commands directly:

```
yolozu train configs/examples/train_setting.yaml \
  --model-config rtdetr_pose/configs/base.json
```

For comprehensive details regarding the backbone contract and supported architectures, refer to `docs/backbones.md`.

4.5.1 Test YAML and the Override Rule

The default test YAML configuration is intentionally minimalist (typically specifying only the adapter, dataset, split, and max_images). Runtime-specific parameters are conventionally supplied as CLI overrides:

```
yolozu test configs/examples/test_setting.yaml \
  --adapter rtdetr_pose \
  --config rtdetr_pose/configs/base.json \
  --checkpoint /path/to/checkpoint.pt \
  --dataset data/smoke \
  --max-images 50
```

This paradigm directly mirrors the documented real-model workflow detailed in `docs/real_model_interface.md`.

4.6 Repository Power Tools

A multitude of task-specific scripts are housed within the `tools/` directory. Notable examples include:

- `tools/validate_predictions.py`
- `tools/eval_instance_segmentation.py`
- `tools/eval_segmentation.py`
- `tools/export_predictions.py` (Facilitates adapter-based export)

For an exhaustive inventory, consult `tools/manifest.json` (for machine-readable metadata) and the documentation index (for human-readable guidance).

Chapter 5

Troubleshooting

This chapter provides an accelerated debugging playbook: it emphasizes validating artifacts first, followed by investigating common integration failure modes. Adhering to this playbook drastically shortens iteration times by identifying schema violations, pathing errors, and class-mapping discrepancies before you attempt to interpret potentially flawed metrics. It is most effective when validators are executed on the exact artifacts slated for evaluation. Pay meticulous attention to the `-pred-root` parameter and ensure consistent class ordering between the exporter and the evaluator.

Who should read this chapter: Read this chapter when results look wrong and you want the shortest route to the first likely failure point.

5.1 Validate early and often

If evaluation results appear anomalous, your immediate first step must be artifact validation.

To validate the standard predictions schema:

```
python3 tools/validate_predictions.py /path/to/predictions.json --strict
```

To validate instance segmentation predictions:

```
python3 tools/validate_instance_segmentation_predictions.py /path/to/preds.json
```

5.2 Common failure modes

- **Pip install failures**: Confirm Python 3.10 or newer with `python3 -version`, then confirm the installer with `python3 -m pip -version`. If the interpreter is too old, create a fresh environment before installing YOLOZU.
- **Missing optional extras**: Import errors for demo, ONNX Runtime, COCO, or backend utilities usually mean the base package was installed without the needed extra. Reinstall the specific lane with `python3 -m pip install 'yolozu[demo]'` or `python3 -m pip install 'yolozu[onnxrt]'`, or the extra documented by the workflow.
- **COCO `category_id` mismatches**: If scores look shifted by class, validate the predictions artifact and compare `category_id` values against the dataset wrapper and `classes.txt`. Start with `yolozu validate predictions`.
- **Invalid predictions schema**: Missing fields, non-relocatable paths, or task-specific payload errors should be fixed before evaluation. In a source checkout, run:

  ```
  yolozu validate predictions <predictions.json> --strict
  ```

  For packaged installs, run `yolozu validate predictions`.

- **ONNX Runtime model-load failures**: If ORT reports unsupported ONNX IR versions, regenerate artifacts with a compatible IR version (CI smoke pins IR v11 for compatibility). Confirm the runtime version before changing the exporter.
- **TensorRT environment**: TensorRT environment constraints are Linux/NVIDIA-centric. Use `python3 tools/gpu_validation_preflight.py -help` and the documented container workflow to keep CUDA/TRT/ORT compatibility pinned.
- **Protocol-driven metric changes**: A metric change after a config or workflow edit may be legitimate if IoU thresholds, class order, score filtering, NMS, or protocol pins changed. Inspect the saved report and suite metadata before comparing old and new scores.
- **Pathing issues**: Relative paths embedded within the JSON artifact fail to resolve correctly against the specified `-pred-root`.
- **Class mapping mismatches**: The ordering defined in `classes.txt` diverges from the internal mapping utilized by the model or exporter.
- **Backend discrepancies**: Pre-processing or post-processing logic varies subtly across different inference backends.
- **CI scope optimization**: Required CI uses a change-scope fast path; docs/metadata-only commits skip heavy test jobs by design.
- **Container workflow expectations**: `.github/workflows/container.yml` now builds container-related pull requests for early validation, while image publication remains gated to tag/manual runs. Manual republishes should pass `release_tag=vX.Y.Z`; when `NGC_API_KEY` is configured, the same workflow mirrors images to `nvcr.io/yolozu/...`.
- **Container trigger scope**: Container checks on pull requests and `main` pushes stay restricted to container-related paths (workflow/deploy/packaging inputs) to avoid unnecessary runs.
- **Security scan backlog vs real regressions**: `.github/workflows/codeql.yml` and `.github/workflows/scorecard.yml` report repository posture, not just runtime test failures. Pull requests run the default `ci` workflow and workflow-only release/security regression checks; Scorecard and CodeQL publish the default-branch posture after merge or on schedule.
- **Fuzzing coverage expectations**: `.github/workflows/cflite_pr.yml` and `.github/workflows/cflite_batch.yml` exercise predictions normalization via ClusterFuzzLite. When predictions parsing/canonicalization changes, update `.clusterfuzzlite/` and `fuzz/` in the same pull request.
- **Self-hosted GPU workflow behavior**: `.github/workflows/ngc_test.yml` probes for an idle `self-hosted + gpu` runner first; if none is online/idle, the workflow exits via a deterministic skip job. If probe API access is denied (`403`), configure repository secret `RUNNER_DISCOVERY_TOKEN` to enable runner discovery.

5.3 Further reading and resources

- Tools index: `docs/tools_index.md`
- External inference documentation: `docs/external_inference.md`
- Predictions schema specification: `docs/predictions_schema.md`
- Training and export overview: `docs/training_inference_export.md`
- CI incident logbook: `docs/ci_incidents.md`
- Release reliability checklist: `docs/release_reliability_checklist.md`
- Security and cryptography scope note: `docs/security_crypto_scope.md`

Part II

Production Evaluation Manual

Chapter 6

Workflows: Evaluate and Export

This chapter outlines the canonical workflows for evaluating existing artifacts, exporting predictions from a backend, and executing convenient folder-level inference. It standardizes the generation of `predictions.json` and evaluation reports, ensuring that results remain strictly comparable and highly automatable. You will require a dataset root and one or more prediction artifacts; backend-specific operations may necessitate additional dependencies.

Who should read this chapter: Read this chapter when you already have data or model outputs and need to decide the fastest safe path to evaluation, export, or migration.

Operator opening.

Goal Pick the shortest route from existing outputs, backend inference, folder inference, or migration to a validated evaluation report.

When to use Use this when you need to evaluate `predictions.json`, export one from a backend, inspect folder-level overlays, or normalize data from another ecosystem.

Inputs A dataset root or image folder; optionally an existing `predictions.json`, model artifact, backend runtime, or native dataset definition.

Command `yolozu validate predictions ... -strict` first, then the matching eval/export/migrate command for Workflow A–D.

Outputs `predictions.json`, validation output, evaluation JSON/HTML, overlays, or normalized dataset wrappers depending on the workflow.

How to verify Open the report JSON, confirm schema validation passed, and check that dataset split, class ids, and prediction counts match the intended run.

Common failure modes Missing files, stale relative paths, class-id drift, backend runtime imports, unsupported model shapes, or conceptual examples copied without real paths.

Readiness classification. Workflow A in this chapter is the **stable main production lane** for YOLOZU: validate predictions, then evaluate them under one pinned predictions interface contract. Export paths and backend-specific execution remain useful, but they should be understood as secondary to that main lane. See `docs/production_readiness.md`.

The shortest human summary. Most people arrive here with one of four questions:

- "I already have predictions. How do I evaluate them?"
- "I have a model and a dataset. How do I export predictions safely?"
- "I just want quick image-folder inference and overlays."
- "My data lives in another ecosystem. How do I migrate without rewriting everything?"

This chapter is organized exactly around those four questions.

Fast chooser. If you are unsure where to start:

- choose **Workflow A** when inference already happened elsewhere,

- choose **Workflow B** when you need YOLOZU to run inference and emit the standardized artifact,
- choose **Workflow C** when you want the quickest folder-level visual result,
- choose **Workflow D** when the problem is data or prediction migration rather than model execution.

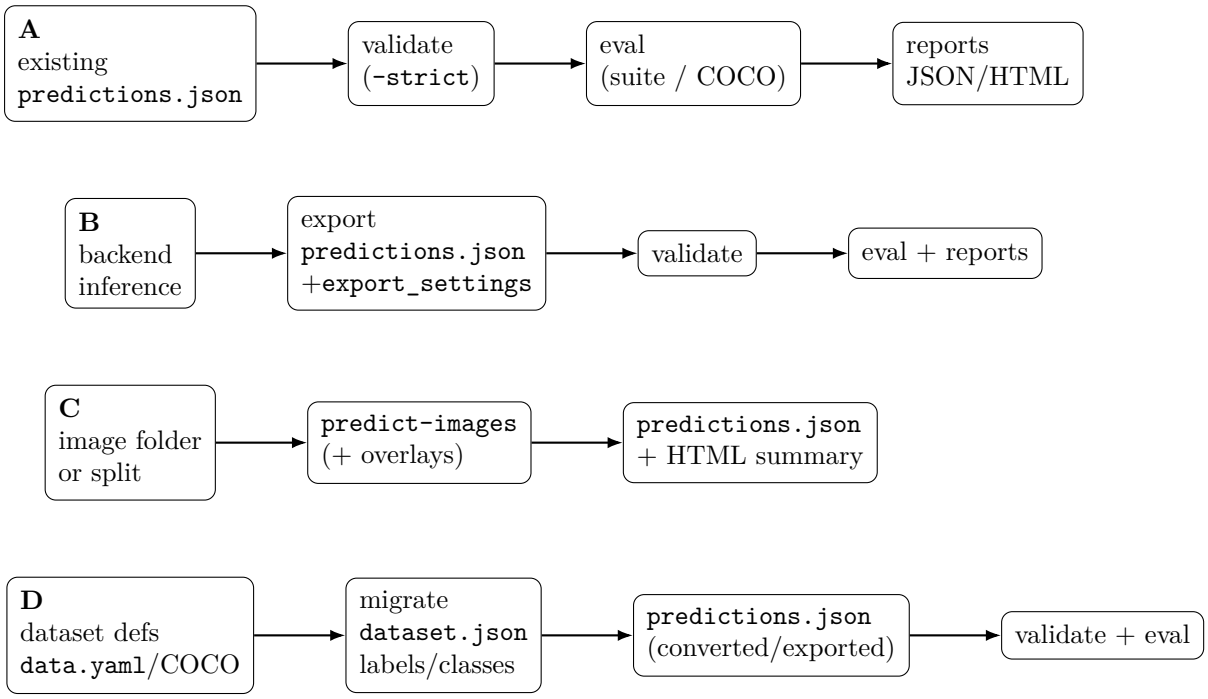


Figure 6.1: Workflow overview (A–D): keep inference flexible, fix the interface boundary at `predictions.json`.

6.1 Workflow A: Evaluate existing predictions.json

This is the safest and simplest path when a different system already produced model outputs. You do not need to rerun inference; you only need a valid predictions artifact and a dataset root.

If you have already generated a `predictions.json` artifact from any inference backend, you can evaluate it directly:

```
yolozu validate predictions /path/to/predictions.json --strict
python3 tools/eval_suite.py --dataset /path/to/yolo-dataset --predictions-glob '/path/
    to/predictions.json'
```

This represents the most common integration pathway for external inference systems.

6.2 Workflow B: Run inference and export predictions

Choose this path when you want YOLOZU to be the boundary that turns backend-specific execution into a stable predictions artifact.

Depending on your deployment environment, several backends are available:

- **Torch backend (research):** Exports predictions directly from a training checkpoint.
- **ONNX Runtime:** Facilitates CPU-friendly exports and rigorous parity checks.
- **OpenCV DNN (YOLO head):** Alternative ONNX execution via OpenCV DNN.
- **OpenCV DNN (RT-DETR, no NMS):** Logits+boxes decode path for RT-DETR ONNX (CUDA/CPU).

- **TensorRT:** Provides a pinned Linux/NVIDIA pipeline optimized for maximum throughput.

For production-oriented external inference, see `docs/external_inference.md` and the submodule-ready templates: `examples/infer_cpp/` (CMake) and `examples/infer_rust/` (Rust).

6.2.1 Rust template: optional ONNXRuntime mode

The Rust template defaults to a minimal no-dependency interface-contract example, and can optionally enable ONNXRuntime via Cargo feature flags:

```
# Minimal contract example (always available)
cargo build --release --manifest-path examples/infer_rust/Cargo.toml
examples/infer_rust/target/release/yolozu_infer_rust --out reports/pred_rust_stub.json

# Optional ONNXRuntime mode
cargo build --release --features onnxruntime --manifest-path examples/infer_rust/Cargo.
    toml
examples/infer_rust/target/release/yolozu_infer_rust \
  --mode onnxrt \
  --onnx /abs/path/model.onnx \
  --input-shape 1,3,64,64 \
  --image images/val/000001.jpg \
  --out reports/pred_rust_onnxrt.json
```

Environment note: The ONNXRuntime feature is intended for pinned Linux production images where Rust toolchains and ONNXRuntime shared libraries are provisioned explicitly.

Conceptual (not copy-paste as-is): repository-wrapper export shape. Use real paths from `python3-myolozuexport--help` for execution.

```
# ONNXRuntime backend (input shape must match ONNX)
python3 -m yolozu export --backend onnxrt --dataset /path/to/yolo-dataset --split val
    --onnx /path/to/model.onnx --output reports/predictions.json

# TorchScript detection artifact with declared combined-output decode
python3 tools/export_predictions_torchscript.py --dataset /path/to/yolo-dataset --
    split val --model /path/to/model.torchscript --output reports/pred_torchscript.
    json --wrap

# ExecuTorch backend (.pte model; dry-run available for interface contract checks)
python3 -m yolozu export --backend executorch --dataset /path/to/yolo-dataset --split
    val --model /path/to/model.pte --output reports/pred_executorch.json --dry-run

# ExecuTorch non-dry decode: run ExecuTorch externally, then decode runtime rows
python3 -m yolozu export --backend executorch --dataset /path/to/yolo-dataset --split
    val --model /path/to/model.pte --runtime-output-json reports/
    executorch_runtime_outputs.json --boxes-scale norm --output reports/
    pred_executorch.json --force

# OpenCV DNN unified backend (auto decode + preset preprocess + IO dump)
python3 -m yolozu export --backend opencv-dnn --onnx /path/to/model.onnx --dataset /
    path/to/yolo-dataset --split val --max-images 8 --imgsz 640 --preprocess
    yolo_letterbox_640 --decode auto --dump-io reports/opencv_dump_io.json --output
    reports/pred_opencv_dnn_unified.json --force

# OpenCV DNN RT-DETR backend (no NMS)
python3 -m yolozu export --backend opencv-dnn-rtdetr --onnx /path/to/rtdetr.onnx --
    dataset /path/to/yolo-dataset --imgsz 640 --score-thr 0.01 --output reports/
    pred_rtdetr_opencv_backend.json --force
```

```
# OpenCV DNN YOLO backend (with NMS)
python3 -m yolozu export --backend opencv-dnn-yolo --onnx /path/to/yolo.onnx --dataset
     /path/to/yolo-dataset --imgsz 640 --score-thr 0.25 --nms-iou 0.45 --output
    reports/pred_opencv_dnn_yolo.json --force
```

For further details, consult: `docs/training_inference_export.md`, `docs/external_inference.md`, and `docs/opencv_dnn_inference.md`. For CI command-surface checks, prefer each exporter's `-dry-run` mode so schema and metadata contracts are validated without heavyweight runtime dependencies.

ExecuTorch non-dry mode intentionally does not pretend to execute a generic mobile runtime inside YOLOZU. The declared path is: execute the `.pte` artifact in the target ExecuTorch environment, write rows shaped `[x1,y1,x2,y2,score,class_id]`, and pass that JSON via `-runtime-output-json`. Unsupported row shapes fail before a predictions artifact is written.

6.2.2 Model intake before export (registry fetch)

To standardize external checkpoints before running exporters, use the model fetch flow:

```
yolozu list models
yolozu fetch yolox-s-coco --out models --accept-license
cat models/yolox-s-coco/meta.json
```

The generated `models/<id>/meta.json` always includes `source`, `version`, `license`, `sha256`, and `created_at`.

6.2.3 YOLO user migration entrypoint (v5/v8/11/26)

For users already operating within YOLO-family pipelines, YOLOZU provides a direct interface-contract-first migration path:

```
python3 tools/import_yolo_data_yaml.py --data-yaml /path/to/data.yaml --split val --
    output data/yolo_wrapper --force
python3 tools/export_predictions_yolo_runtime.py --model yolo11n.pt --dataset data/
    yolo_wrapper --split val --protocol nms_applied --wrap --output reports/
    pred_yolo_runtime.json
python3 tools/export_predictions_yolov5.py --dataset data/yolo_wrapper --split val --
    labels-dir runs/detect/exp/labels --protocol nms_applied --output reports/
    pred_yolov5.json
```

Protocol policy for fair comparisons:

- `nms_applied`: intended for YOLOv5/YOLOv8/YOLO11 post-NMS outputs.
- `e2e_nms_free`: intended for YOLO26/RT-DETR no-NMS outputs.

When outputs contain COCO `category_id`, evaluate with class-id normalization:

```
yolozu eval-coco --dataset data/yolo_wrapper --split val --predictions reports/
    pred_yolo_runtime.json --protocol nms_applied --classes data/yolo_wrapper/labels/
    val/classes.json --output reports/coco_eval_yolo_runtime.json
```

6.2.4 YOLOX interoperability path

For teams already operating YOLOX experiments (`exp.py` + checkpoints), keep native inference and export directly to the YOLOZU contract:

```
python3 -m yolozu export --backend yolox --dataset /path/to/coco-yolo --split val2017
    --exp /path/to/yolox_exp.py --weights /path/to/yolox_ckpt.pth --imgsz 640 --score-
    thr 0.01 --nms-iou 0.65 --output reports/pred_yolox.json
python3 tools/validate_predictions.py reports/pred_yolox.json --strict
```

```
python3 tools/eval_coco.py --dataset /path/to/coco-yolo --split val2017 --predictions
    reports/pred_yolox.json --protocol nms_applied --classes /path/to/coco-yolo/labels/
    val2017/classes.json --output reports/eval_yolox.json
```

The exporter records `weights_sha256`, projected exp parameters, preprocessing assumptions, and protocol metadata under `meta.extra.export_settings`.

6.2.5 YOLO-style external training lane

When the goal is training rather than export-only interop, the recommended external YOLO-style lane is YOLOX because it fits the repository's Apache-2.0 operational policy more naturally than optional copyleft-sensitive bridges.

```
python3 -m yolozu train \
  --external-backend yolox \
  configs/examples/finetune_external/yolox_s_finetune_smoke.py \
  --dataset data/smoke \
  --split val \
  --dry-run \
  --output reports/train_external_yolox.json
```

The repo helper remains available when you want the lower-level bridge surface directly:

```
python3 tools/support_external_training.py train-yolox \
  --dataset data/smoke \
  --split val \
  --exp configs/examples/finetune_external/yolox_s_finetune_smoke.py \
  --dry-run \
  --output reports/support_external_training.train_yolox.json
```

6.2.6 Current training support

- **Stable reference lane:** the in-repo RT-DETR pose reference trainer.
- **Supported external lane:** the Apache-2.0-friendly YOLOX path exposed through `yolozu train -external-backend yolox`.
- **Optional external bridges:** Ultralytics and HF DETR, kept explicit as external runtime/license boundaries.
- **Not claimed:** a universal training platform for every model family.

When post-train ONNX export is enabled, the export attempt runs on CPU even if training itself ran on CUDA or MPS. This keeps the export path more portable and avoids backend-specific exporter failures at the end of an otherwise successful run.

This writes a machine-readable report containing:

- the resolved dataset root and split,
- a projected training configuration,
- the template command for the external launcher,
- the runtime/license boundary for the lane.

For real execution, provide your local YOLOX train launcher:

```
python3 tools/support_external_training.py train-yolox \
  --dataset /path/to/coco-yolo \
  --split train \
  --exp /path/to/yolox_exp.py \
  --train-script /path/to/YOLOX/tools/train.py \
  --batch 16 \
  --weights /path/to/yolox_ckpt.pth \
  --output reports/support_external_training.train_yolox.exec.json
```

The Ultralytics bridge remains available on the same surface, but it is documented as optional and should be reviewed separately under its own license terms:

```
python3 tools/support_external_training.py train-ultralytics \
  --dataset data/smoke \
  --split val \
  --preset smoke \
  --dry-run
```

The same top-level route also exposes the optional HF DETR bridge:

```
python3 -m yolozu train \
  --external-backend hf-detr \
  facebook/detr-resnet-50 \
  --dataset data/smoke \
  --split val \
  --dry-run \
  --output reports/train_external_hf_detr.json
```

6.2.7 SynthGen intake workflow (external generator)

When synthetic shards are generated externally (for example in `YOLOZU-synthgen`), YOLOZU provides the contract/eval side:

```
python3 tools/validate_synthgen_contract.py \
  --input /path/to/synthgen_dataset/shards/train_000.jsonl \
  --max-samples 200

python3 tools/render_synthgen_overlay.py \
  --dataset-root /path/to/synthgen_dataset \
  --schema-id animal_v1 \
  --sample-index 0 \
  --output reports/synthgen_overlay_animal.png

python3 tools/eval_synthgen.py \
  --dataset-root /path/to/synthgen_dataset \
  --predictions /path/to/synthgen_dataset/shards/predictions_synthgen.json \
  --schema-id animal_v1 \
  --output reports/synthgen_eval_animal.json
```

Schema-specific task templates are provided under:

- `configs/tasks/synthgen_animal_kpt.yaml`
- `configs/tasks/synthgen_mechanical_kpt.yaml`

For offline CI-safe smoke checks, use the bundled mini-shard fixture:

```
python3 tools/smoke_synthgen.py \
  --dataset-root data/smoke/synthgen_minishard \
  --output-dir reports
```

Path-resolution note: if an external generator emits prediction rows with shard-relative asset paths (for example `../samples/...`), keep the predictions artifact under `shards/` or rewrite those paths before evaluation.

6.3 Workflow C: Folder inference (pip CLI)

Choose this when you care more about "show me the result" than about setting up a larger benchmark or protocol-pinned evaluation loop.

A highly convenient, user-facing approach for batch inference is:

```
yolozu predict-images   --backend dummy   --input-dir /path/to/images   --output
    reports/predict_images.json   --overlays-dir reports/predict_images_overlays   --
    html reports/predict_images.html   --progress
```

This command generates a standardized predictions artifact, PNG visual overlays, and an HTML summary. With `-progress`, stderr shows stage and overlay progress. The terminal output prints the predictions path, HTML path, overlays directory, and first overlay path so new users know exactly what to open next. Use `configs/quickstart/predict_images_dummy.yaml` as the checklist for expected files.

6.4 Post-result display and reporting

Following inference or evaluation, YOLOZU generates both machine-readable artifacts and human-readable summaries.

Command family	Typical output	Usage
`yolozu predict-images`	`reports/predict_images.json`	Normalized predictions artifact
`yolozu predict-images`	`reports/predict_images.html`	Visual summary page
`yolozu predict-images`	`reports/predict_images_overlays/*.png`	Per-image visual overlays
`python3 tools/eval_suite.py`	`reports/eval_suite.json`	Protocol-pinned comparison report
`python3 tools/hpo_sweep.py`	`reports/hpo_sweep.jsonl`	Trial-level raw history
`python3 tools/hpo_sweep.py`	`reports/hpo_sweep.csv`, `reports/hpo_sweep.md`	Leaderboard and table views
`yolozu eval-instance-seg`	`reports/instance_seg_eval.json`	Instance segmentation metrics
`yolozu eval-instance-seg`	`reports/instance_seg_eval.html`	HTML report with optional overlays

6.4.1 Recommended result inspection sequence

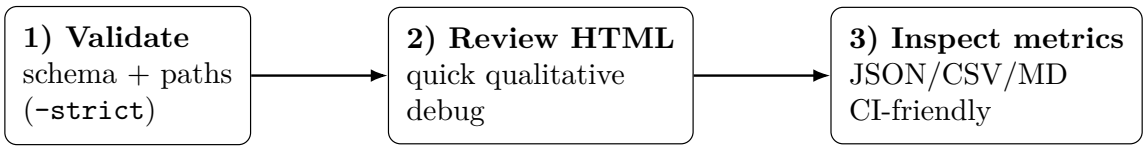


Figure 6.2: Recommended result inspection order for fast iteration and reproducible reporting.

1. **Validate the schema:**

   ```
   yolozu validate predictions reports/predict_images.json --strict
   ```

2. **Review the HTML report** to perform rapid qualitative assessments.
3. **Inspect the JSON/CSV/MD artifacts** for rigorous quantitative comparisons suitable for CI pipelines and academic publications.

An example workflow for instance segmentation, incorporating HTML and overlay outputs:

```
yolozu eval-instance-seg   --dataset examples/instance_seg_demo/dataset   --split
    val2017   --predictions examples/instance_seg_demo/predictions/
    instance_seg_predictions.json   --pred-root examples/instance_seg_demo/predictions
       --classes examples/instance_seg_demo/classes.txt   --output reports/
    instance_seg_eval.json   --html reports/instance_seg_eval.html   --overlays-dir
    reports/instance_seg_overlays   --max-overlays 10
```

6.5 Schema validation as a quality gate

It is imperative to validate prediction artifacts prior to evaluation:

```
python3 tools/validate_predictions.py /path/to/predictions.json --strict
```

This practice proactively prevents silent evaluation failures caused by malformed outputs.

6.6 Workflow D: Migrate datasets and predictions

Choose this path when the difficult part is not inference itself, but bridging from an existing dataset or prediction ecosystem into the YOLOZU interface contract.

YOLOZU provides migration utilities that generate lightweight wrapper artifacts or converters, enabling seamless integration with prevalent ecosystem datasets. The core philosophy is to preserve the original dataset immutability while generating version-controlled wrappers.

6.6.1 YOLO-style data.yaml / YOLO (YOLOv8/YOLO11)

To generate a `dataset.json` wrapper from an existing `data.yaml`:

```
yolozu migrate dataset   --from auto   --data /path/to/data.yaml   --output data/
    yolo_wrapper
```

Validate the resulting wrapper:

```
yolozu validate dataset data/yolo_wrapper
```

6.6.2 COCO JSON datasets (YOLOX / Detectron2 / MMDetection)

To convert COCO instances JSON into YOLO-formatted label text files and generate a corresponding wrapper:

```
yolozu migrate dataset   --from coco   --coco-root /path/to/coco_like_root   --split
    val2017   --output data/coco_yolo_like   --mode manifest
yolozu validate dataset data/coco_yolo_like --strict
```

To export that wrapper into external training layouts:

```
yolozu export-dataset yolo   --dataset data/coco_yolo_like   --split val2017   --out-
    dir data/coco_yolo_export   --force
yolozu export-dataset coco   --dataset data/coco_yolo_like   --split val2017   --out-
    dir data/coco_export   --force
yolozu export-dataset kitti   --dataset data/coco_yolo_like   --split val2017   --out-
    dir data/coco_kitti_export   --force
```

For semantic-segmentation descriptors, export keeps the `dataset.json` contract while materializing `images/` and `masks/` into a lightweight external layout:

```
yolozu doctor import --dataset-from auto --dataset /path/to/VOCdevkit/VOC2012 --split
    val --output -
yolozu import dataset   --from auto   --dataset /path/to/VOCdevkit/VOC2012   --split
    val   --output reports/voc_seg_wrapper   --force
yolozu export-dataset segmentation   --dataset reports/voc_seg_wrapper   --out-dir
    reports/voc_seg_export   --image-mode symlink   --force
```

For COCO keypoints roots that expose only `person_keypoints_<split>.json`, the same auto-detect preflight now recognizes the richer keypoints layout and writes a YOLOZU wrapper with `keypoint_names` / `skeleton` metadata before export:

```
yolozu doctor import --dataset-from auto --dataset /path/to/coco_keypoints_root --
    split val2017 --output -
yolozu doctor train-dataset --from coco-keypoints --dataset /path/to/
    coco_keypoints_root --split val2017 --output -
yolozu import dataset   --from coco-keypoints   --dataset /path/to/coco_keypoints_root
       --split val2017   --output reports/coco_keypoints_wrapper   --force
yolozu export-dataset coco   --dataset reports/coco_keypoints_wrapper   --split
    val2017   --out-dir reports/coco_keypoints_export   --force
```

For a repo-bundled tiny sample that exercises automatic detection plus `COCO -> YOLOZU -> YOLO/COCO/KITTI` end to end:

```
yolozu validate dataset data/conversion_tiny_coco --split val2017 --strict
yolozu doctor import --dataset-from auto --dataset data/conversion_tiny_coco --split
    val2017 --output -
yolozu migrate dataset   --from auto   --dataset data/conversion_tiny_coco   --split
    val2017   --output reports/conversion_tiny_wrapper   --force
yolozu export-dataset yolo   --dataset reports/conversion_tiny_wrapper   --split
    val2017   --out-dir reports/conversion_tiny_yolo   --force
yolozu export-dataset coco   --dataset reports/conversion_tiny_wrapper   --split
    val2017   --out-dir reports/conversion_tiny_coco   --force
yolozu export-dataset kitti   --dataset reports/conversion_tiny_wrapper   --split
    val2017   --out-dir reports/conversion_tiny_kitti   --force
```

6.6.3 Detectron2 / MMDetection predictions interop

For Detectron2/MMDetection users, keep framework-native inference and only export detections into `predictions.json`:

```
python3 tools/export_predictions_detectron2.py   --dataset data/coco_yolo_like   --
    split val2017   --config /path/to/d2_config.yaml   --weights /path/to/model_final.
    pth   --protocol nms_applied   --output reports/pred_detectron2.json

python3 tools/export_predictions_mmdet.py   --dataset data/coco_yolo_like   --split
    val2017   --config /path/to/mmdet_config.py   --checkpoint /path/to/epoch_12.pth
     --protocol nms_applied   --output reports/pred_mmdet.json
```

Evaluate with explicit class-id normalization to avoid COCO `category_id` drift:

```
python3 tools/validate_predictions.py reports/pred_detectron2.json --strict
python3 tools/eval_coco.py   --dataset data/coco_yolo_like   --split val2017   --
    predictions reports/pred_detectron2.json   --protocol nms_applied   --classes data/
    coco_yolo_like/labels/val2017/classes.json   --output reports/coco_eval_detectron2.
    json
```

The exporters preserve raw framework box convention (`xyxy_abs`) in `export_settings.raw_output_bbox_format`, while normalizing contract output to `cxcywh_norm` for strict schema validation.

6.6.4 COCO results (inference outputs) → predictions.json

To convert COCO-style detection results into the standardized YOLOZU predictions contract:

```
yolozu migrate predictions   --from coco-results   --results /path/to/coco_results.
    json   --instances /path/to/instances_val2017.json   --output reports/predictions.
    json   --force

yolozu validate predictions reports/predictions.json --strict
```

Chapter 7

Score Calibration and Long-tail Adjustments

This chapter describes post-hoc score calibration and long-tail adjustments that transform confidence scores while strictly preserving the underlying geometric predictions. These methods can improve thresholding behavior and long-tail metrics without retraining, while keeping the transformation reproducible via saved statistics artifacts. It requires a dataset and a predictions artifact. Method-specific parameters apply, and calibrated outputs should be written as separate artifacts.

Who should read this chapter: Read this chapter when your geometry is fine but your scores or class balance look systematically off.

7.1 When calibration is beneficial

Calibration is useful when your model is accurate but systematically miscalibrated (for example, consistently overconfident or underconfident), or when long-tail classes have suppressed confidence. In these cases, post-hoc calibration can improve operating points (threshold choice) and class balance *without the cost of retraining*.

7.2 The calibrate command

YOLOZU provides a unified CLI for several calibration methods. Conceptually, calibration modifies only **confidence scores**; the geometric outputs (bounding boxes, masks, keypoints, and pose fields) are left unchanged. The command writes (1) a calibrated predictions artifact and (2) a small JSON statistics report to make the transformation reproducible.

Example: fit FRACAL statistics for bounding boxes and write calibrated outputs:

```
yolozu calibrate \
  --method fracal \
  --task bbox \
  --dataset /path/to/yolo-dataset \
  --predictions reports/predictions.json \
  --out reports/predictions_calibrated.json \
  --stats-out reports/fracal_stats_bbox.json
```

Example: reuse previously computed statistics:

```
yolozu calibrate \
  --method fracal \
  --task bbox \
  --dataset /path/to/yolo-dataset \
  --predictions reports/predictions.json \
```

```
--out reports/predictions_calibrated.json \
--stats-in reports/fracal_stats_bbox.json
```

7.3 Supported methods (high-level overview)

- **FRACAL**: frequency-aware calibration based on class count statistics (the primary in-repo method; recommended for comparisons against standard baselines).
- **LA (Logit Adjustment)**: class-prior logit shift controlled by `-tau` [17].
- **NorCal**: frequency-based adjustment controlled by `-gamma`.
- **Temperature scaling**: global temperature scalar via `-temperature`, with optional fitting for supported tasks [6].

7.4 Task-specific behavior

Calibration is task-aware:

- **bbox**: adjusts detection confidence scores.
- **seg**: adjusts instance scores while leaving the binary masks unchanged.
- **pose**: adjusts detection scores while preserving keypoint coordinates and pose geometry.

7.5 Operational best practices

- Keep calibrated outputs as separate artifacts; do not overwrite your baseline predictions.
- Record the method name and key parameters (the CLI report is designed to capture this).
- Compare methods on the same baseline predictions and the same evaluation split to avoid confounding changes.

Chapter 8

Tasks: Detection, Segmentation, Keypoints

This chapter delineates the supported task families and rigorously defines their expected prediction schemas. Adhering to these specifications prevents schema mismatches and guarantees the application of the correct evaluator for each task, thereby enhancing metric accuracy and streamlining the debugging process. It assumes the presence of task-specific data fields (e.g., PNG masks for instance segmentation, and keypoint arrays for pose estimation); follow Chapter 1's operating rules by validating artifacts before scoring them.

Who should read this chapter: Read this chapter when you need to confirm task-specific fields before exporting or evaluating predictions.

8.1 Bounding-box detection

Bounding-box detection serves as the foundational task, generating per-image detections characterized by class IDs, bounding box coordinates, and confidence scores. Evaluation is predominantly conducted using COCO-style mean Average Precision (mAP) or other protocol-pinned metrics.

8.2 Instance segmentation (PNG masks)

YOLOZU evaluates instance segmentation utilizing **per-instance binary PNG masks**. This approach deliberately circumvents the complexities associated with Run-Length Encoding (RLE) or polygon-based tooling.

The minimal required prediction schema is structured as follows:

```
[
  {
    "image": "000001.png",
    "instances": [
      {"class_id": 0, "score": 0.9, "mask": "masks/000001_inst0.png"}
    ]
  }
]
```

To validate the schema:

```
python3 tools/validate_instance_segmentation_predictions.py   reports/
    instance_seg_predictions.json
```

To execute evaluation and generate visual overlays and HTML reports:

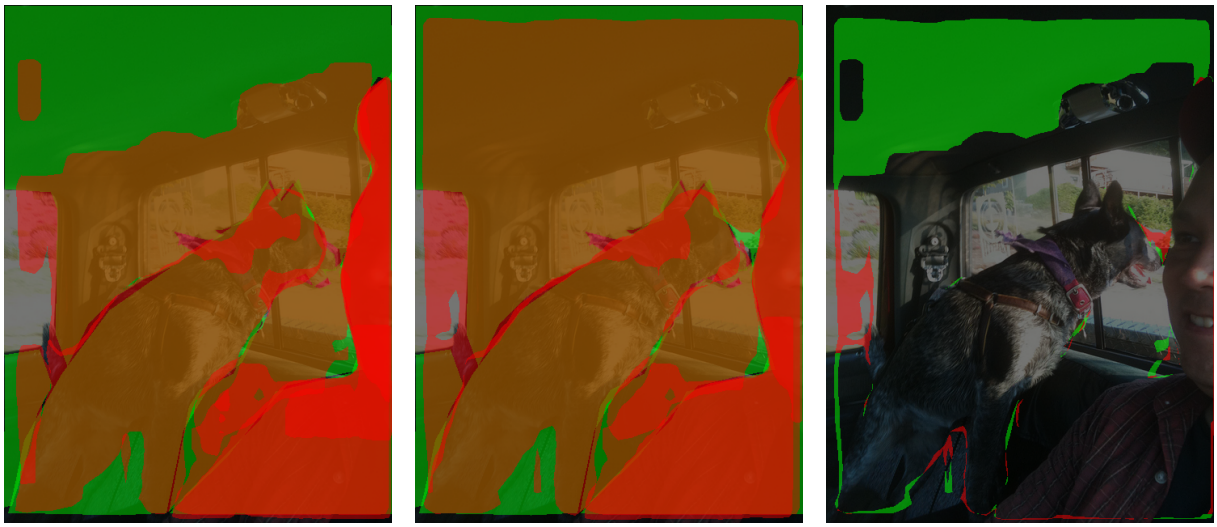
```
python3 tools/eval_instance_segmentation.py   --dataset /path/to/dataset   --split val
      --predictions /path/to/predictions.json   --pred-root /path/to/pred-root   --
    classes /path/to/classes.txt   --html reports/instance_seg_eval.html   --overlays-
    dir reports/instance_seg_overlays   --max-overlays 10
```

8.2.1 Real-image TTA compare demo

For a visible, reproducible compare on real images, YOLOZU also provides a dedicated instance-seg TTA demo using `torchvision` Mask R-CNN. It applies a deterministic corruption to a COCO image, runs raw inference, then reruns with augmentation-based TTA. For brightness corruption the demo uses brightness-lift plus horizontal flip; for the other supported corruptions it uses horizontal flip only.

```
yolozu demo instance-seg-tta --run-dir reports/demo_instance_seg_tta
```

Example result from the bundled tiny COCO subset: `image_id=407083`, `brightness` severity `5`, with `tp2->3`, `fp4->2`, and `fn1->0`. Read the figure as follows: green in the delta panel indicates area recovered by TTA or false-positive area removed by TTA; red indicates regression introduced by TTA.



(a) TTA Off (b) TTA On (c) Delta

Figure 8.1: Instance segmentation compare on the same corrupted real COCO image. In the Off/On panels, green denotes ground truth and red denotes predicted masks. In the Delta panel, green marks TTA improvements and red marks TTA regressions.

8.3 Semantic segmentation

The semantic segmentation suite provides robust dataset preparation utilities and mean Intersection over Union (mIoU) evaluation capabilities. While semantic segmentation evaluation yields mIoU (alongside per-class IoU), instance segmentation evaluation outputs COCO-style mask mAP. Ensure you utilize `tools/eval_segmentation.py` for semantic segmentation tasks and `tools/eval_instance_segmentation.py` for instance segmentation.

8.4 Keypoints / pose estimation

YOLOZU fully supports YOLO pose-style keypoints within both labels and predictions, complemented by dedicated evaluation utilities. Depending on your specific workflow requirements, you may employ Percentage of Correct Keypoints (PCK) evaluation, COCO Object Keypoint Similarity (OKS) mAP, or both.

Dataset preparation is streamlined into a single command:

```
python3 tools/prepare_keypoints_dataset.py   --source /path/to/export   --format auto
     --out /path/to/keypoints_dataset
```

Supported direct input formats include: `auto`, `yolo_pose`, `coco`, and `cvat_xml`. When processing CVAT XML, you may provide either a specific XML file or a directory containing an `annotations.xml` file. If the image root inference is ambiguous, explicitly define it using the `-cvat-images-dir` parameter.

A minimal smoke test for CVAT XML processing:

```
python3 -m pytest -q tests/test_prepare_keypoints_dataset_cvat_xml.py
```

Keypoint evaluation entry points are accessible via scripts located in the `tools/` directory (e.g., keypoint evaluation helpers). Please consult the repository documentation for the authoritative schema definitions and comprehensive usage instructions.

Chapter 9

Parity, Benchmarks, and Protocols

This chapter explains parity checks, protocol-pinned evaluation settings, and benchmark/sweep harnesses. These tools keep comparisons stable by catching silent numerical drift and tracking performance and metric trends over time. You need at least two comparable prediction artifacts for parity analysis. Protocol-driven runs also need pinned settings files, consistent preprocessing, and consistent output formats.

Who should read this chapter: Read this chapter when you are comparing backends, benchmarking runs, or trying to make metrics fair across repeated experiments.

Operator opening.

Goal Compare backend outputs and benchmark reports without mixing runtime drift into metric conclusions.

When to use Use this when at least two artifacts, formats, or runs must be compared under the same protocol and preprocessing assumptions.

Inputs Comparable `predictions.json` files, protocol settings, backend export settings, and hardware/runtime notes for benchmark runs.

Command `yolozu parity ...` for artifact comparison, or `yolozu benchmark ...` / `python3 tools/benchmark_model.py ...` for benchmark reports.

Outputs Parity JSON, benchmark JSON, export settings, evaluation reports, and skipped-format records when a backend cannot execute.

How to verify Confirm the report states real, artifact-backed, synthetic, or skipped status and records the reference backend and protocol settings.

Common failure modes Mismatched preprocessing, shape-pinned engines, hidden NMS differences, absent optional runtimes, or treating skipped formats as successful benchmarks.

Readiness classification. This chapter belongs to the **experimental** lane in YOLOZU. Parity and benchmark tooling are valuable qualification tools, but they still depend strongly on the local runtime matrix and should not be mistaken for the main production lane by themselves. See `docs/production_readiness.md`.

9.1 Backend parity

When exporting predictions across backends such as PyTorch, ONNX Runtime, and TensorRT, check for silent numerical drift. Parity tooling compares two prediction artifacts and reports the differences.

Conceptual (not copy-paste as-is): parity command shape. For a runnable shortest command, use Chapter 5 Workflow A with real artifact paths.

```
yolozu parity   --reference reports/pred_torch.json   --candidate reports/pred_onnxrt.
    json
```

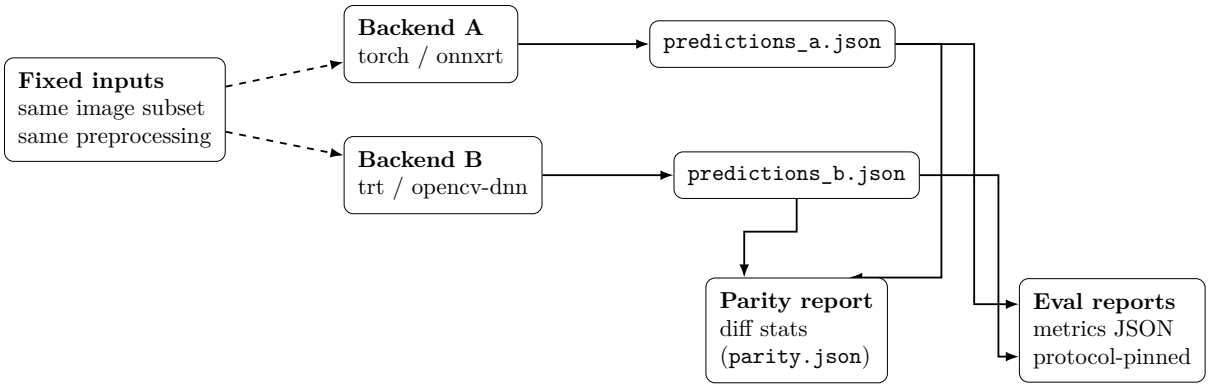


Figure 9.1: Apples-to-apples parity: keep inputs fixed, compare exported artifacts, and score with the same evaluator/protocol.

9.2 Protocols (pinning evaluation settings)

Protocols pin evaluation settings such as dataset splits, image dimensions, and metric keys. This keeps long-running comparisons stable.

For YOLO26 comparisons, the pinned protocol is defined as follows:

- Protocol file: `protocols/yolo26_eval.json`
- Task: `detect`, Dataset: `COCO`, Split: `val2017`
- Image size: `640` (enforcement is the responsibility of the export pipeline)
- Bounding box format: `cxcywh_norm`
- Primary score key: `map50_95`

A key semantic rule from the protocol documentation:

- "End-to-end (e2e) mAP" dictates that the evaluator scores detections exactly as provided; it explicitly does *not* apply Non-Maximum Suppression (NMS).

9.3 Benchmarks

Latency and FPS benchmarks are part of the regression testing surface.

The recommended user-facing entrypoint is now `yolozu benchmark`. The current implementation remains artifact-first and additionally wires real backend orchestration for `torch`, `onnx`, `engine`, and `torchscript`. TorchScript uses a declared combined-output decode path with rows `[x1,y1,x2,y2,score,class_id]`. When the real backends can execute, the command records a real dataset-pass wall-clock measurement. When they cannot, it still records skipped formats honestly rather than fabricating success.

A representative repository-wrapper invocation is:

```
python3 tools/benchmark_model.py \
  --model runs/example/model.pt \
  --onnx-model exports/example.onnx \
  --engine-model exports/example.plan \
  --data data/coco8.yaml \
  --format torch,onnx,engine \
  --protocol nms_applied \
  --latency-source auto \
  --output reports/benchmark_report.json
```

The primary artifact set is:

- `reports/benchmark_report.json`
- `reports/export_settings_<format>.json`
- `reports/predictions_<format>.json`

- reports/eval_<format>.json
- reports/parity_<format>.json

When torch plus at least one additional real backend (onnx or engine) complete successfully, parity_<format>.json is now a real backend-diff artifact rather than a synthetic placeholder record. The chosen reference backend is recorded explicitly (preferring torch when available). Synthetic/skipped parity artifacts remain only for dry-run, skipped, or synthetic-only formats.

If a detect benchmark should anchor parity on a non-torch backend, set it explicitly:

```
python3 tools/benchmark_model.py \
  --model runs/example/model.pt \
  --onnx-model exports/example.onnx \
  --data data/coco8.yaml \
  --format torch,onnx \
  --latency-source auto \
  --parity-reference-backend onnx \
  --output reports/benchmark_report.json
```

9.3.1 Benchmark task semantics

The benchmark surface now records explicit task semantics instead of treating -task as a loose label. The top-level report and each per-format result carry the canonical task, the originally requested task, accepted aliases, metric family, expected metric keys, support level, and whether the task is part of the mainstream benchmark surface or a YOLOZU-native extension.

The current task matrix is written as task cards instead of a wide table so the PDF stays readable:

detect Aliases: detect, detection. Metric family: bbox_map. Execution support: real for torch, onnx, engine, and torchscript. This is the default benchmark path.

segmentation Aliases: segmentation, seg. Metric family: mask_map. Execution support: artifact-backed real eval/parity for torch, onnx, engine, and torchscript. Benchmark mode evaluates backend mask-prediction artifacts with tools/eval_segmentation.py and compares matched masks directly.

classification Aliases: classification, classify, cls. Metric family: topk_accuracy. Execution support: artifact-backed real eval for torch, onnx, engine, and torchscript. Benchmark mode evaluates backend class-score artifacts directly and reports top1/top5/accuracy without claiming YOLOZU ran backend inference.

obb Alias: obb. Metric family: obb_map. Execution support: artifact-backed real eval for torch, onnx, engine, and torchscript. Benchmark mode evaluates backend rotated-box artifacts directly and reports OBB metrics without claiming YOLOZU ran backend inference.

keypoints Aliases: keypoints, pose. Metric family: oks_map. Execution support: artifact-backed real eval/parity for torch, onnx, engine, and torchscript. pose is normalized to canonical keypoints; benchmark mode evaluates backend predictions.json artifacts with tools/eval_keypoints.py and compares normalized keypoints directly.

depth Alias: depth. Metric family: depth_error. Execution support: artifact-backed real eval/parity for torch, onnx, engine, and torchscript. This is a YOLOZU-native extension that compares backend depth artifacts honestly instead of claiming end-to-end benchmark-surface parity.

pose6d Aliases: pose6d, 6dof, pose_6d, pose-6d. Metric family: pose6d_error. Execution support: artifact-backed real eval/parity for torch, onnx, engine, and

`torchscript`. This is a YOLOZU-native extension that compares backend prediction artifacts honestly instead of claiming end-to-end benchmark-surface parity.

For the current implementation, inference-backed real backend execution remains `detect`-first. The exceptions are `classification`, `obb`, `segmentation`, `keypoints`, `depth`, and `pose6d`: all may use `-latency-source artifact_eval`. For `classification` and `obb`, YOLOZU evaluates backend score or rotated-box artifacts and writes explicit parity placeholders. For `segmentation`, YOLOZU evaluates backend-specific mask-prediction artifacts with `tools/eval_segmentation.py` and then attaches real parity reports over matched sample ids plus per-mask mismatch summaries. For `keypoints`, YOLOZU evaluates backend-specific `predictions.json` artifacts with `tools/eval_keypoints.py` and then attaches real parity reports over matched detections plus normalized keypoints. For `depth`, YOLOZU evaluates backend-specific depth artifacts (`.npy`, `.npz`, or single-channel image files) against the ground-truth depth artifact and then attaches real parity reports. For `pose6d`, YOLOZU evaluates backend-specific `predictions.json` artifacts with `tools/eval_pose.py` and then attaches real parity reports over matched detections plus pose fields such as rotation, translation, and depth. Formats without shipped benchmark wiring are reported as unsupported/skipped rather than as synthetic benchmark successes.

9.3.2 Canonical benchmark support matrix

The source of truth for per-format, per-task artifact status is `docs/benchmark_support_matrix.md`. Use that matrix when deciding whether a benchmark lane writes real inference artifacts, consumes artifact-backed real eval/parity inputs, emits dry-run placeholders, or skips because runtime, artifact, format, or task wiring is unavailable.

The manual intentionally avoids duplicating the full table. When benchmark semantics change, update the canonical matrix, `docs/benchmark_mode.md`, this chapter, `tools/manifest.json`, and the packaged manifest copy in the same change.

9.3.3 Runtime / license boundary matrix

Benchmark support does not imply that a runtime is bundled with YOLOZU. The repository remains Apache-2.0, but several backends rely on external runtimes or vendor SDKs with separate terms.

Use these categories instead of reading backend names as license promises:

Real runtime path
: `torch`, `onnx`, `engine`, and `torchscript` have real detect benchmark paths when the matching local runtime is installed. These runtimes are still external to this repository. For `engine`, verify the Linux, NVIDIA GPU, CUDA, TensorRT, and NGC redistribution terms separately.

Adapter target only
: `executorch` and `opencv_dnn` are adapter targets. YOLOZU can describe or prepare the lane, but users must supply and validate the external runtime. For OpenCV, optional contrib/nonfree modules are not bundled.

Conditional adapter targets
: `openvino`, `coreml`, `ncnn`, `rknn`, `tflite`, and `paddle` are not bundled runtimes. Treat them as conditional adapter targets unless the support matrix explicitly says a real lane exists for the selected task and format.

9.3.4 Gap against the benchmark/export surface

Relative to the broader benchmark/export surface users now expect, YOLOZU has already matched the core benchmark entrypoint shape, but it still trails in surface breadth.

The most important remaining gaps are:

- broader format coverage beyond conditional `openvino`, such as `coreml`, `saved_model`, `tflite`, `ncnn`, `rknn`, and `paddle`
- broader parity attachment for artifact-backed `classification` and `obb` lanes
- stronger validation of format-specific knobs so unsupported combinations fail early rather than acting like inert flags

9.3.5 Highest-leverage parity-closing work

If the goal is to close the practical gap against the broader benchmark/export surface quickly, the most effective implementation order is:

1. keep conditional `openvino` honest after the current `torchscript` detect path
2. keep `docs/benchmark_support_matrix.md` as the single support matrix that distinguishes real inference/eval/parity from dry-run placeholders and unsupported/skipped paths
3. expand real parity artifacts beyond the current `torch`-anchored backend comparisons, including the artifact-backed `classification` and `obb` placeholder parity reports
4. keep format-specific flag validation strict so inert combinations fail early rather than appearing supported

This order keeps the interface contract stable, improves user-visible parity quickly, and preserves YOLOZU's Apache-2.0 repository operations policy.

This comparison is intentionally about interface behavior and user-facing surface only. YOLOZU keeps its Apache-2.0-only repository operations policy and must not pull GPL-licensed implementation code into the repository while closing these gaps.

The same boundary logic applies to training-side integrations: prefer the Apache-2.0-friendly YOLOX external lane first, and keep optional Ultralytics bridges explicitly separated rather than treating them as equivalent defaults.

For repository-side orchestration, the equivalent wrapper is `tools/benchmark_model.py`.

The benchmark harness, located at `tools/benchmark_latency.py`, generates:

- A stable JSON report (detailing a single run).
- A JSONL history file (facilitating append-only trend tracking).

This harness supports both synthetic timing evaluations and configuration-driven, per-bucket executions. In the latter mode, each bucket can ingest externally measured metrics via JSON (for instance, data generated by TensorRT latency measurement scripts).

Typically tracked metrics include FPS and latency quantiles (mean, p50, p90, p95, and p99).

9.4 Sweeps

A sweep harness is indispensable for systematically comparing numerous runs or configurations.

The sweep runner, `tools/hpo_sweep.py`, is entirely framework-agnostic and operates by executing templated commands. The core configuration model comprises:

- `base_cmd`: The command template containing `{param}` placeholders.
- `param_grid` or `param_list`: The defined hyperparameter search space.
- Optional keys for metrics extraction and designated output destinations.

The default artifacts are designed for effortless diffing and publication: `reports/hpo_sweep.jsonl`, `reports/hpo_sweep.csv`, and `reports/hpo_sweep.md`. Utilize the `-resume` flag to intelligently bypass previously completed parameter combinations.

Chapter 10

Depth + 6DoF Pose + Symmetry Handling

This chapter documents geometry conventions (units, intrinsics, and coordinate frames), pose and translation recovery, and symmetry and commonsense constraint utilities. It improves correctness and robustness of depth and pose pipelines by making assumptions explicit and by providing reusable postprocess gates. It requires consistent intrinsics after resize and letterbox, and consistent units; some metrics and utilities require sidecar metadata (for example CAD points) and schema-valid pose fields.

Who should read this chapter: Read this chapter when your workflow depends on geometry, depth, pose, intrinsics, or symmetry-aware evaluation.

10.1 Units, intrinsics, and coordinate conventions

Depth/pose workflows are extremely sensitive to units and to intrinsics consistency.

Key conventions used across docs in this repo:

- Intrinsics `K=(fx,fy,cx,cy)` are in **pixel units**.
- Intrinsics must match the **image coordinate system used by model outputs** (after resize/letterbox).
- Depth `z` uses the dataset's length unit (YOLOZU does not convert mm $\leftrightarrow$ m).

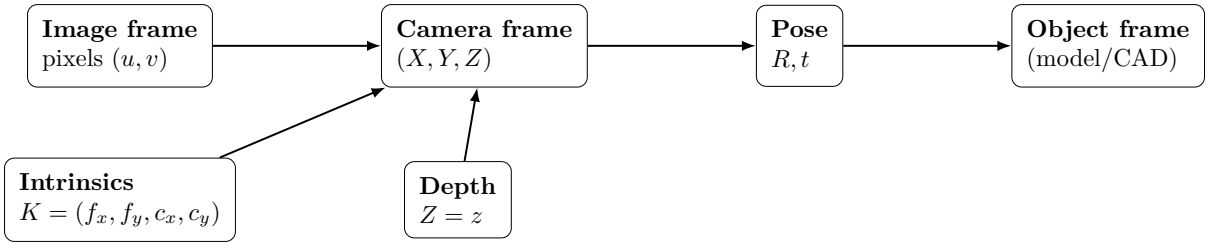


Figure 10.1: Coordinate frames: intrinsics and depth connect image pixels to 3D camera coordinates, which are then related to object frame via pose.

Depth integration modes in the reference trainer:

- `none`: depth disabled (compatibility default).
- `sidecar`: depth is read from sidecar metadata (`depth_path`/`depth`) and tracked via `depth_valid`.
- `fuse_mid`: depth is fused after projector (outside backbone $[P3, P4, P5]$ swap boundary).

Metric safety rule:

- Use `-depth-unit metric` when absolute depth costs are intended.
- For `unspecified`/`relative`, absolute depth matcher terms are disabled for safety.

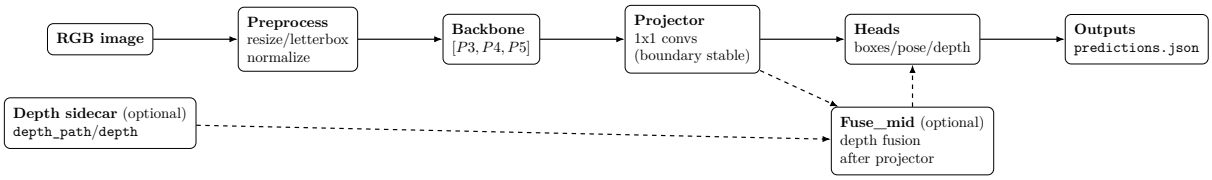


Figure 10.2: Depth modes: `sidecar` carries depth as metadata; `fuse_mid` fuses depth after the projector (outside the backbone swap boundary).

10.1.1 Generic depth evaluation CLI

For single-pair depth checks outside the SynthGen evaluator, use:

```
python3 tools/eval_depth.py \
  --pred-depth reports/pred_depth.npy \
  --gt-depth data/reference/gt_depth.npy \
  --align median_scale \
  --output reports/depth_eval.json
```

This CLI is intentionally generic and CPU-friendly. It compares one predicted depth map against one ground-truth map and writes a JSON report with:

- `abs_rel`, `sq_rel`, `rmse`, and `rmse_log`
- threshold metrics `delta1`, `delta2`, and `delta3`
- valid-pixel counts and the applied alignment scale factor

Use `-mask` when only part of the depth map should be scored. The default `-align median_scale` mode is appropriate for relative monocular depth predictions where the global scale is ambiguous.

10.1.2 Artifact-backed depth benchmark/parity

The benchmark lane now reuses the same depth evaluator to compare backend artifacts directly:

```
python3 tools/benchmark_model.py \
  --task depth \
  --model reports/depth_torch.npy \
  --onnx-model reports/depth_onnx.npy \
  --data data/reference/gt_depth.npy \
  --format torch,onnx \
  --latency-source artifact_eval \
  --depth-align median_scale \
  --output reports/benchmark_depth_report.json
```

This path is intentionally artifact-backed. YOLOZU does not pretend it ran the underlying backend inference; instead it:

- records the backend-specific depth artifact under `predictions_<format>.json`
- evaluates each artifact with `tools/eval_depth.py`
- compares candidate depth arrays against the reference backend and writes a real parity report

This closes the depth benchmark lane for backend comparison while keeping the interface contract honest about where inference actually happened.

10.1.3 Artifact-backed keypoints benchmark/parity

The same benchmark entrypoint now supports artifact-backed keypoints comparison:

```
python3 tools/benchmark_model.py \
  --task keypoints \
```

```
  --model reports/keypoints_torch.json \
  --onnx-model reports/keypoints_onnx.json \
  --data data/keypoints_dataset \
  --format torch,onnx \
  --latency-source artifact_eval \
  --keypoints-parity-kp-atol 1e-4 \
  --output reports/benchmark_keypoints_report.json
```

This lane is intentionally artifact-backed. YOLOZU does not pretend it ran the underlying backend inference; instead it:

- copies each backend `predictions.json` artifact into the benchmark artifact layout
- evaluates each artifact with `tools/eval_keypoints.py`
- compares candidate predictions against the reference backend and records score/bbox/keypoint parity

This closes the keypoints benchmark lane for backend comparison while keeping the interface contract honest about where inference actually happened.

10.1.4 Artifact-backed 6DoF benchmark/parity

The same benchmark entrypoint now supports artifact-backed 6DoF comparison:

```
python3 tools/benchmark_model.py \
  --task pose6d \
  --model reports/pose_torch.json \
  --onnx-model reports/pose_onnx.json \
  --data data/pose_dataset \
  --format torch,onnx \
  --latency-source artifact_eval \
  --pose-parity-trans-atol 1e-4 \
  --output reports/benchmark_pose6d_report.json
```

This lane is also intentionally artifact-backed. YOLOZU does not pretend it ran the underlying backend inference; instead it:

- copies each backend `predictions.json` artifact into the benchmark artifact layout
- evaluates each artifact with `tools/eval_pose.py`
- compares candidate predictions against the reference backend and records rotation/translation/depth parity

This closes the 6DoF benchmark lane for backend comparison while keeping the interface contract honest about where inference actually happened.

10.1.5 Generic 6DoF evaluation CLI

For single-artifact 6DoF checks outside the benchmark wrapper, use:

```
python3 tools/eval_pose.py \
  --dataset data/pose_dataset \
  --predictions reports/predictions_pose.json \
  --output reports/pose_eval.json
```

This CLI matches detections to GT by IoU and reports:

- pose core metrics such as `pose_success`, `rot_deg_mean`, and `trans_l2_mean`
- optional depth error when the predictions expose depth/translation
- optional CAD-point metrics such as `add_mean` and `adds_mean`

Use `-success-rot-deg` and `-success-trans` to make the success gate explicit when the operator wants a stricter or looser pose-success definition.

Example:

```
python3 rtdetr_pose/tools/train_minimal.py \
  --dataset-root data/coco128 \
  --config rtdetr_pose/configs/base.json \
  --use-matcher \
  --depth-mode sidecar \
  --depth-unit metric \
  --depth-scale 1.0
```

10.1.6 Imbalance handling, backbone override, strict data checks

The trainer also supports data-imbalance mitigation and explicit runtime backbone overrides:

- `-imbalance-strategy class_balanced` (+ `-imbalance-gamma`, `-imbalance-min-weight`, `-imbalance-max-weight`, `-imbalance-aggregate`)
- `-backbone-name`, `-backbone-norm`, `-backbone-args '{"...": ...}'`
- `-strict-task-data` for fail-fast validation of required real-data supervision

```
python3 rtdetr_pose/tools/train_minimal.py \
  --dataset-root data/real_multitask_fewshot \
  --split train --val-split val \
  --real-images --strict-task-data \
  --use-matcher --model-config rtdetr_pose/configs/base.json \
  --imbalance-strategy class_balanced --imbalance-gamma 1.0 \
  --backbone-name cspdarknet_s --backbone-norm bn \
  --backbone-args '{"width_mult": 0.5, "depth_mult": 0.34}'
```

10.1.7 Real-image few-shot multitask demo

To run a staged smoke pipeline over `bbox -> segmentation -> keypoints -> depth -> pose6d`:

```
# 0) Download tiny COCO subset manually (review dataset license terms)
python3 scripts/download_coco_instances_tiny.py \
  --out-root data/coco --split val2017 --num-images 8 --seed 0 --force

python3 tools/prepare_real_multitask_fewshot.py \
  --instances-json data/coco/annotations/instances_val2017.json \
  --images-dir data/coco/images/val2017 \
  --out data/real_multitask_fewshot \
  --train-images 6 --val-images 2 --strict-provenance --force

python3 tools/run_real_multitask_finetune_demo.py \
  --dataset-root data/real_multitask_fewshot \
  --out reports/real_multitask_finetune_demo \
  --device cpu \
  --epochs 1 --max-steps 1 --batch-size 2 --image-size 96 \
  --strict-provenance --force
```

The aggregated report is written to `reports/real_multitask_finetune_demo/multitask_finetune_demo_report.json`. Dataset provenance for each supervision field is recorded in `data/real_multitask_fewshot/prepare_summary.json`.

Practical entry points ("real model" path):

- Reference trainer: `python3 rtdetr_pose/tools/train_minimal.py ...` (writes metrics and optional run artifacts).
- Export predictions: `python3 tools/export_predictions.py -adapter rtdetr_pose ... -wrap`

- Torch acceleration knobs on export: `-infer-batch-size`, `-torch-compile*`, `-torch-amp`, `-torch-channels-last`, `-torch-inference-mode`
- No-torch path: use the `precomputed` adapter with a schema-valid `predictions.json`.

10.2 Typical regression heads

The 6DoF-oriented design documents and utilities assume per-detection regression heads such as:

- `log_z`: object center depth in log space
- `rot6d`: 6D rotation representation [28]
- `offsets`: center correction $(\Delta u, \Delta v)$
- `k_delta`: small intrinsics correction (global / per-frame)

See the design spec for clear definitions of frames and outputs: `docs/specs/rt_detr_6dof_geom_mim_spec_en_v0_4.md`.

10.3 Translation recovery (postprocess)

A common pattern is to recover translation from depth + intrinsics: For monocular depth estimation background (if you learn depth from RGB), see [2, 22].

Let bbox center be (u, v) and corrected center (u', v') be:

$$u' = u + \Delta u, \qquad v' = v + \Delta v.$$

Let corrected intrinsics be $K' = (f'_x, f'_y, c'_x, c'_y)$. Then with $Z = z$:

$$X = \frac{u' - c'_x}{f'_x} Z, \qquad Y = \frac{v' - c'_y}{f'_y} Z.$$

The point of this design is to avoid unstable direct regression of (X, Y).

10.4 Commonsense constraints (Stage 4, implemented utilities)

This repository contains a small, test-covered set of **commonsense constraints** for depth/pose postprocess. They are designed as **utilities** (not a full end-to-end model training system) and can be used both as train-time regularizers and as inference-time gates/penalties.

Implementation / configuration:

- Core logic: `yolozu/constraints.py`
- Runtime config: `configs/runtime/constraints.yaml`

Key pieces (using the Stage 4 naming kept elsewhere in this manual):

- Depth prior:
  `depth_prior(bbox_wh, size_wh, (fx,fy))`
  and penalty `depth_prior_penalty(z_pred, z_prior)`.
- Table plane: signed distance via `plane_signed_distance(...)` and below-plane rejection via `is_above_plane(...)`.
- Upright prior: roll/pitch extraction and violation via `upright_violation_deg(R, bounds)`.

Wiring example:

- Candidate evaluation calls constraints from `yolozu/pipeline.py` (`evaluate_candidate`).

Ablations:

- Toggle each constraint: `tools/ablate_constraints.py`

Tests (unit + integration toggles):

- `tests/test_gates_constraints.py`
- `tests/test_inference_constraints.py`

10.5 Handheld camera robustness (Stage 5, implemented utilities)

Stage 5 focuses on robustness to camera perturbations by consistently using corrected intrinsics K' and center offsets $(u + \Delta u, v + \Delta v)$.

Implementation entry points:

- Intrinsics correction: `yolozu/geometry.py` (`corrected_intrinsics`)
- Translation recovery using offsets: `yolozu/geometry.py` (`recover_translation`)
- Jitter profile utilities: `yolozu/jitter.py`

Jitter utilities provide:

- Intrinsics drift sampling (e.g. $\delta f, \delta c$): `sample_intrinsics_jitter(...)`
- Extrinsics jitter sampling: `sample_extrinsics_jitter(...)`
- A baseline profile with jitter disabled: `jitter_off()`

Note: the jitter profile includes a `rolling_shutter` field (`enabled`, `line_delay`), but the current repository code treats it as a reserved configuration field; there is no physics-accurate rolling shutter model wired into the core geometry utilities.

Tests (integration-level expectations for jitter on/off profiles):

- `tests/test_geometry_pipeline.py`
- `rtdetr_pose/tests/test_train_minimal_sim_jitter.py`
- `rtdetr_pose/tests/test_train_minimal_extrinsics_jitter.py`

10.6 Scenario suite (Stage 6, deterministic validation harness)

The validation harness includes a lightweight **scenario suite** that emits a stable JSON report for CI and tooling integration.

Implementation / schema:

- Scenario suite generator: `yolozu/scenario_suite.py` (`build_report`)
- CLI wrapper: `tools/run_scenario_suite.py`
- Schema-style stability test: `tests/test_scenario_suite.py`

Run it manually:

```
python3 tools/run_scenario_suite.py --output reports/scenario_suite.json
```

Important: the current scenario suite produces **deterministic harness metrics** (it is a harness and contract surface), not a substitute for real model evaluation.

10.7 Per-detection refinement: Hessian solver

YOLOZU includes a Gauss–Newton / Levenberg–Marquardt style refinement utility to iteratively refine regression head outputs per detection [13, 16].

Key idea: refine per-detection parameters (depth/rotation/offsets) with bounded iterations; treat it as a research postprocess (not a default real-time feature).

10.7.1 CLI

Refine offsets (example):

```
python3 tools/refine_predictions_hessian.py \
  --predictions reports/predictions.json \
  --dataset data/coco128 \
  --output reports/predictions_refined.json \
  --enable \
  --refine-offsets \
```

```
  --wrap
```

Refine offsets (bounded iterations):

```
python3 tools/refine_predictions_hessian.py \
  --predictions reports/predictions.json \
  --dataset data/coco128 \
  --output reports/predictions_refined.json \
  --enable \
  --refine-offsets \
  --steps 10 \
  --damping 1e-2 \
  --wrap
```

10.7.2 How it differs from calibration

- Hessian refinement is **per detection** (local improvements).
- L-BFGS calibration is typically **dataset-wide** (global scale/intrinsics style parameters).

Use refinement when you have supervision/constraints and you want to improve individual predictions. Avoid it for real-time production unless the added latency is justified.

10.8 Symmetry: why it matters

For symmetric objects, a single canonical rotation label may not be unique. Naïvely training with a standard rotation loss can be ill-posed and can also break template-based verification.

10.9 Symmetry metadata (runtime configuration)

The preferred approach is deterministic, class-level symmetry metadata. The runtime config exists at:

- `configs/runtime/symmetry.json`

The expected shape (from the spec) looks like:

```
{
  "class_id_or_name": {
    "type": "none | Cn | Dn | Cinf | mirror",
    "n": 4,
    "axis": [0, 0, 1],
    "notes": "optional"
  }
}
```

Note: in a fresh checkout, `configs/runtime/symmetry.json` may be empty (\{\}). Populate it for your class set.

10.10 Symmetry-aware loss / metrics (concept)

A symmetry-aware rotation distance is commonly expressed as:

$$L_{rot}^{sym} = \min_{S \in G} d(R_{pred}, R_{gt} \cdot S)$$

where G is the symmetry group for the object.

For context and intended gates (including real-time constraints), see:

- `docs/specs/rt_detr_6dof_geom_mim_spec_en_v0_4.md`
- `docs/roadmaps/symmetry_commonsense_realtime.md` (historical notes)

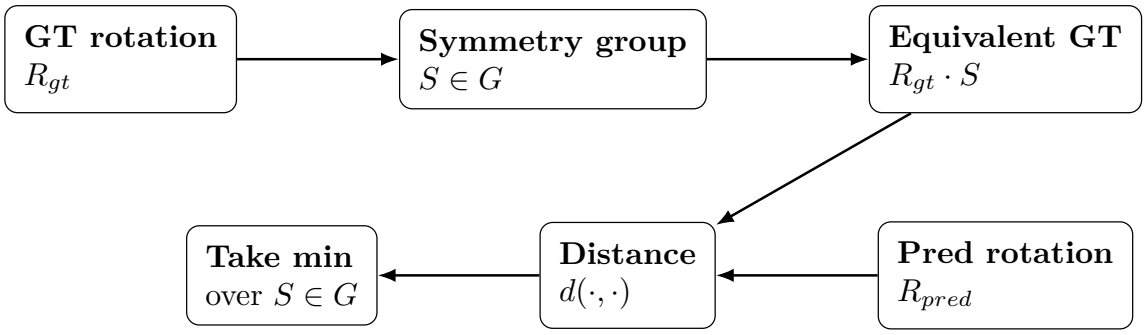


Figure 10.3: Symmetry-aware scoring: evaluate rotation error modulo the object's symmetry group.

10.11 Available evaluation metrics (current implementation)

The pose/depth evaluator (`yolozu/pose_eval.py`) reports these keys:

- Depth: `depth_abs_mean`, `depth_abs_median`
- Pose core: `rot_deg_mean`, `trans_l2_mean`, `pose_success`
- CAD-point pose: `add_mean`, `add_median`, `adds_mean`, `adds_median`

Definitions:

- **ADD**: mean point distance under fixed point correspondence between (R_{pred}, t_{pred}) and (R_{gt}, t_{gt}).
- **ADDS**: mean distance from each transformed predicted point to its nearest transformed GT point.

These pose metrics and the symmetry caveats around them are discussed in standard 6D pose evaluation literature [9, 10]. For an example of a modern end-to-end 6D pose system (useful as conceptual background), see [12].

Input requirements:

- Depth metrics require valid GT translation/depth and predicted depth or translation.
- ADD/ADDS require CAD points in sidecar metadata (`cad_points` / `cad_path` / `cad_points_path`) plus valid GT+pred pose.

10.12 Practical advice

- First, make unit/intrinsics consistency boring and deterministic.
- Next, ensure evaluation metrics treat symmetry correctly for relevant classes.
- Keep symmetry/commonsense logic out of the TensorRT graph when possible; run it in postprocess.

Chapter 11

Real-time vs Batch Inference (Research and Deployment)

This chapter compares batch and real-time operating modes and outlines the canonical TensorRT pipeline with parity and benchmark safeguards. It guides you to high-throughput deployment paths while keeping artifact contracts and drift detection in place. Real-time TensorRT paths are typically Linux with an NVIDIA GPU; parity requires consistent preprocessing and output formats, and expensive extras like TTT and refinement should be explicitly budgeted.

Who should read this chapter: Read this chapter when you need to choose between offline throughput and real-time deployment constraints.

Operator opening.

Goal Choose batch or real-time inference while keeping `predictions.json`, parity, and latency evidence explicit.

When to use Use this when moving from offline evaluation toward ONNX Runtime, TensorRT, or streaming deployment constraints.

Inputs Model/export artifacts, input shape assumptions, preprocessing settings, runtime backend, and hardware notes.

Command `python3 tools/export_trt.py ...`, `python3 tools/build_trt_engine.py ...`, and backend export/parity commands with real paths.

Outputs ONNX or engine artifacts, metadata JSON, predictions artifacts, parity reports, and latency/FPS reports.

How to verify Confirm shape metadata, preprocessing, backend output format, parity tolerance, and hardware-pinned latency are all recorded.

Common failure modes Shape mismatch, hidden preprocessing drift, unsupported GPU/runtime libraries, unbudgeted TTT/refinement, or latency claims without hardware evidence.

11.1 Two operating modes

Batch/offline Run over a dataset split, export `predictions.json`, then evaluate and plot.

Real-time/streaming Optimize end-to-end latency and stability; avoid expensive per-frame extras.

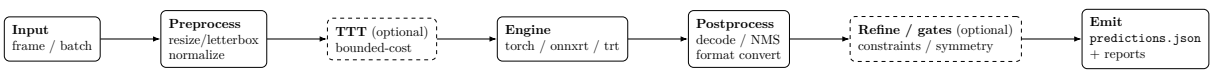


Figure 11.1: Real-time vs batch inference share the same artifact boundary (`predictions.json`); optional extras must be explicitly budgeted.

11.2 Non-goals (operational boundaries)

- TTT is **not** a real-time default. Keep it disabled unless explicitly budgeted and enabled.
- If TTT is enabled in streaming mode, use bounded-cost knobs first (e.g., `-ttt-batch-size 1 -ttt-max-batches 1`).
- Hessian refinement is not part of the TensorRT inference graph; treat it as an optional postprocess step.
- Real-time claims without hardware-pinned latency/FPS evidence are out of scope for this chapter.

11.3 Backend choices

- **Precomputed predictions**: best for integration; works on macOS; use `tools/run_scenarios.py` with `precomputed` adapter.
- **Torch**: flexible for research (TTT, ablations), but slower and less deployable.
- **ONNX Runtime**: CPU-friendly parity reference; useful in CI.
- **TensorRT**: preferred for real-time throughput on Linux+NVIDIA.

References:

- `docs/real_model_interface.md`
- `docs/tensorrt_pipeline.md`

11.4 TensorRT pipeline (canonical)

The typical export route is PyTorch → ONNX → TensorRT (engine build), with parity checks.

Canonical pattern (artifacts-first):

- Export ONNX (+ metadata JSON):
  `python3 tools/export_trt.py -onnx ... -opset 18 -dynamic-hw ...`
- Build engine (+ metadata JSON):
  `python3 tools/build_trt_engine.py -onnx ... -engine ... -precision fp16`
- Export `predictions.json` from TensorRT and run parity vs ONNXRuntime predictions.

The main rule for parity is to keep preprocessing and output formats identical (letterbox/RGB, box format, score scaling), and keep NMS disabled in exported graphs when comparing raw outputs.

Input shape note: Unlike PyTorch eager mode, ONNXRuntime and TensorRT are typically **shape-pinned** unless you explicitly export with dynamic axes / optimization profiles. If an ONNX or engine expects (for example) 64×64, feeding 640×640 will fail. The TensorRT exporter defaults to inferring `imgsz` from the engine input when possible (and accepts an explicit `-imgsz` override for dynamic engines).

11.5 Stage 7: Performance + deployment safeguards (implemented tooling)

Stage 7 is about keeping the real-time path fast and predictable. In this repo, the main safeguards are implemented as **tooling and contracts** rather than as model-internal graph logic.

11.5.1 Keep symmetry/commonsense logic out of TensorRT graphs

The TensorRT export/build pipeline is designed to focus on the model forward pass. Symmetry, template scoring, commonsense constraints, and gates are intended to run in **postprocess** where

possible.

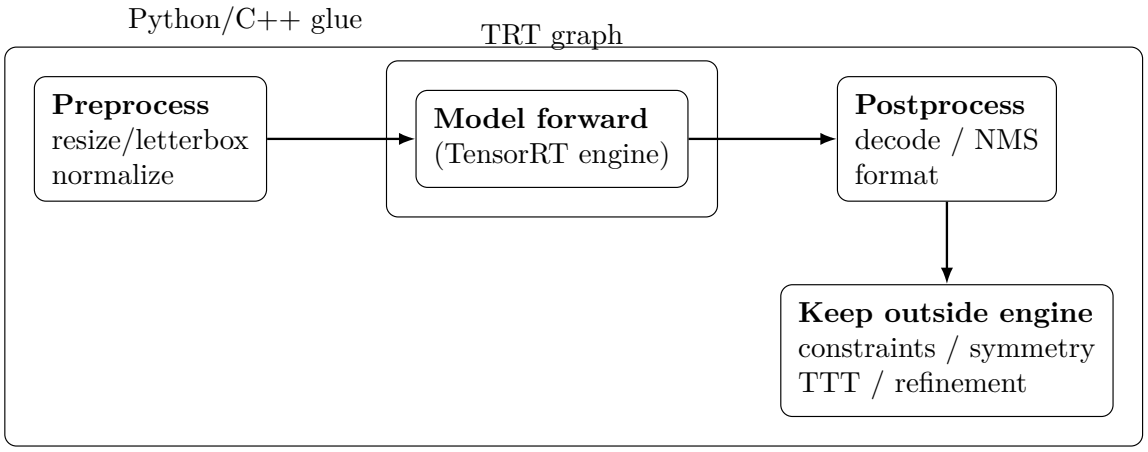


Figure 11.2: Boundary rule: keep non-forward logic out of the TensorRT graph to simplify deployment and make drift diagnosable.

In particular:

- Commonsense constraints live in `yolozu/constraints.py` and are toggled via `configs/runtime/constraints.yaml`.
- Parity comparisons are performed on raw outputs (no-NMS exports) so numerical mismatches are detectable.

11.5.2 Benchmark and regression surfaces

There are two complementary benchmark surfaces:

- A backend-agnostic latency/FPS harness: `tools/benchmark_latency.py`
- A TensorRT engine latency measurement tool (Linux+NVIDIA + TRT python): `tools/measure_trt_latency.py`

The harness supports optional targets and can fail the run if targets are not met:

```
python tools/benchmark_latency.py --config configs/benchmark_latency_example.json \
  --target-fps 30
```

For a full end-to-end TensorRT run on a Linux+NVIDIA box, use: `tools/run_trt_pipeline.py` (export/predictions/parity/eval/latency/benchmark; supports `-dry-run` for CI-style artifact checks).

11.5.3 Gate checks against a baseline

The repo includes a lightweight gate-check script that compares a current scenario suite report with a saved baseline:

- `tools/check_gates.py` (writes `reports/gate_check.json`)
- baseline source: `reports/baseline.json` (generated by `tools/run_baseline.py`)

The current Stage 7 check is a simple `fps >= 30` threshold in the gate output. Note that real FPS validation is inherently **hardware-dependent**; CI runs can validate the harness and contracts, while deployment-relevant benchmarks should be run on reference Linux+NVIDIA hardware.

11.6 Real-time constraints and optional features

11.6.1 TTT in real-time

TTT can improve robustness under domain shift but adds latency. If you use it in streaming mode, keep it bounded:

- prefer `-ttt-preset safe`
- start with `-ttt-batch-size 1 -ttt-max-batches 1`
- prefer `-ttt-reset stream` when you accept cross-image state

For controlled comparisons and ablations, use `-ttt-reset sample` even though it is slower.

11.6.2 Per-detection refinement

Hessian refinement (depth/rotation/offsets) is usually a **post-inference research tool**. It is not typically enabled in production unless the added cost is explicitly budgeted.

11.7 Batch throughput tips

- Increase batch size first (until memory limits).
- Keep dataset subsets deterministic for comparisons.
- Record engine metadata (GPU/driver/TensorRT versions) when benchmarking.

11.8 Mac development vs GPU execution

If you develop on macOS, keep GPU/TensorRT runs on a Linux+NVIDIA box. The repo provides a Runpod-oriented path; see `deploy/runpod/README.md`.

11.8.1 Preflight split for GPU sweep tasks

Before reserving GPU runtime, generate a local preflight split report:

```
python3 tools/gpu_validation_preflight.py --output reports/gpu_validation_preflight.
    json
```

The report separates:

- `summary.local_executable_steps`: tasks you can complete locally
- `summary.gpu_execution_required_steps`: tasks that require Linux+NVIDIA runtime
- `summary.needs_gpu_runtime_steps`: tasks ready to run once GPU runtime is available

Operational runbook: `docs/runpod_gpu_validation_split.md`.

When operating via GitHub Actions machine runners instead of RunPod sessions, use `.github/workflows/gpu_zisn_pipeline.yml` (`workflow_dispatch`) and select stage `zisn1`, `zisn2`, `zisn3`, or `all`. The workflow publishes deterministic DoD summaries to `ci-logs/gpu-zisn`.

11.9 Inference profiler integration

The `yolozu.inference.profiler` module wraps `torch.profiler` to produce kernel-level latency traces that complement wall-clock benchmarks.

```
profiler_available()
profile_inference(adapter, records, *, warmup, active, output_dir)
profile_callable(fn, *args, iterations=10, warmup=3)
```

- `profiler_available()` checks for `torch.profiler.profile`.

- `profile_inference(...)` profiles `adapter.predict()` calls and writes TensorBoard/Chrome-trace output.
- `profile_callable(...)` profiles any callable and returns a text summary table.

```
from yolozu.inference.profiler import profile_inference, profiler_available

if profiler_available():
    summary = profile_inference(adapter, records, output_dir="runs/profile")
    print(summary)
```

11.10 torch.compile and ONNX export helpers

The `yolozu.inference.torch_export` module provides two orthogonal utilities:

- `compile_for_inference(model, backend, mode)` — wraps `torch.compile` with configurable backend (`inductor`, `aot_eager`, etc.) and mode (`reduce-overhead`, `max-autotune`). Falls back gracefully when compilation is unavailable.
- `export_model_onnx(model, sample_input, output_path, opset_version, ...)` — exports a `torch.nn.Module` to ONNX format with configurable opset, dynamic axes, and named I/O.
- `torch_compile_available()`, `torch_export_available()` — feature-detection guards.

The `RTDETRPoseAdapter` integrates `compile_for_inference` via its `compile_model`, `compile_backend`, and `compile_mode` constructor kwargs; when enabled, the model is automatically compiled at first `predict()` call.

```
from yolozu.inference.torch_export import (
    compile_for_inference, export_model_onnx,
)

fast = compile_for_inference(model, mode="reduce-overhead")
export_model_onnx(model, sample_input, "model.onnx", opset_version=17)
```

11.11 What to report in papers/notes

For real-time or deployment-relevant results, always report:

- hardware (GPU, driver, CUDA/TensorRT) and engine build settings
- end-to-end latency / FPS and measurement method
- whether TTT/refinement/gates were enabled and their bounds

Part III

Training and Research Workflows

Chapter 12

Training and the Run Contract

This chapter explains the training stack and the run interface contract: stable artifact layout, strict config resolution, and resumable runs. Following the run interface contract keeps runs reproducible, makes parity checks easier, and keeps outputs predictable across machines and branches. It assumes run-interface-contract mode, such as `-run-id` or `-run-contract`, strict config keys, and the standard training dependencies.

Who should read this chapter: Read this chapter when you are running training jobs and want reproducible artifact layout, resume behavior, and tuning surfaces.

Operator opening.

Goal Run training so checkpoints, summaries, exports, resume state, and evaluation handoff files land in predictable locations.

When to use Use this when YOLOZU owns the reference training run or when an external lane must emit a stable training summary and next-step handoff.

Inputs A normalized dataset root or wrapper, a YAML/JSON training config, and optional resume/export/parity settings.

Command `yolozu train ...` for the public route, or `python3 rtdetr_pose/tools/train_minimal.py ...` when using the low-level reference trainer.

Outputs `runs/<run_id>/checkpoints/`, `runs/<run_id>/reports/`, optional `exports/`, and `training_summary.json`.

How to verify Check that `config_resolved.yaml`, metrics JSONL, checkpoint paths, and optional export/parity reports exist and match the selected run id.

Common failure modes Native dataset layouts passed directly to the reference trainer, missing config keys, stale resume paths, backend-specific export failures, or unexpected training dependencies.

12.1 Model-family policy and reference training stack

YOLOZU interface contracts for predictions, evaluation reports, and scenario outputs are model-family agnostic. The in-repo training reference implementation is the RT-DETR-style adapter under `rtdetr_pose/`. The critical part is the **run interface contract**: stable artifact paths and resumable training runs, independent of the model family that produced them.

Two detector-family lanes are supported at the training surface:

- **RT-DETR / DETR-family**: the in-repo reference lane. Use AdamW, separate backbone/head learning-rate groups, gradient clipping, warmup, EMA, and NMS-free evaluation/export assumptions by default. See `configs/examples/train_rtdetr_stable.yaml`.
- **YOLO-family**: external YOLO lanes such as YOLOX. Preserve letterbox preprocessing, SGD-style optimization defaults, and NMS-applied export/evaluation assumptions. See `configs/examples/finetune_external/yolox_s_finetune_smoke.py`.

Do not copy one family's defaults into the other. A DETR-family run should not silently inherit YOLO letterbox/NMS assumptions, and a YOLO-family run should not silently inherit

DETR AdamW/head-backbone group assumptions unless the backend configuration explicitly says so.

12.1.1 Dataset-format boundary for the reference trainer

The RT-DETR reference trainer does not claim that it can ingest every native dataset layout directly at training time. Its expected input is a dataset root that `build_manifest(...)` can resolve: a YOLO-style root, a YOLO/Ultralytics `data.yaml`, or a `dataset.json` wrapper with `images_dir` and `labels_dir` pointing to image files and YOLO label files. The multi-format user convenience layer therefore lives one step earlier: `yolozu doctor import -dataset-from auto`, `yolozu import dataset -from auto`, and `yolozu migrate dataset -from auto` can auto-detect supported native sources such as COCO, COCO keypoints, Ultralytics/YOLO, and the supported semantic-segmentation roots, then materialize a normalized wrapper for training.

In other words, users do not need to manually classify the incoming dataset family, but the reference trainer itself still consumes a train-ready YOLOZU/YOLO-style result of that preflight/import step. Semantic-segmentation descriptors and SynthGen intake shards are import/evaluation inputs unless they are converted into train-ready image and label records. The `doctor import` report includes `reference_trainer.direct_train_ready` and `reference_trainer.train_ready_after_migration` so automation can distinguish direct training inputs from native sources that need migration first. The train-specific `doctor train-dataset` route performs the same check and emits next commands without launching training. It also recognizes explicit `classification`, `obb`, `depth`, and `pose6d` source families. Classification folders/manifests and OBB labels are recognized intake contracts for external training lanes, not direct RT-DETR reference-trainer inputs. Depth and pose6d are direct only when normalized bbox records include the required sidecars. Through the public CLI, `python3 -m yolozu train <config> -dataset-root <wrapper>` forwards the dataset root to the RT-DETR reference trainer.

For record-based training, `python3 -m yolozu train <config> -records-json <train.json> -val-records-json <val.json>` forwards explicit train and validation records to the reference trainer. Both files may be a JSON list of records or an object with an `images` list. Records may use either `image` or `image_path`; label boxes may use flat `cx`/`cy`/`w`/`h` fields or nested `bbox` fields. OBB records may include `angle` or `obb`; depth and pose6d records may carry sidecars at the record level or on individual labels. Use this path when the dataset cannot be cleanly represented as one YOLO-style split directory.

```
python3 -m yolozu doctor train-dataset \
  --from auto \
  --dataset /path/to/native_dataset_root \
  --split val2017 \
  --output -
python3 -m yolozu doctor import \
  --dataset-from auto \
  --dataset /path/to/native_dataset_root \
  --split val2017 \
  --output -
python3 -m yolozu migrate dataset \
  --from auto \
  --dataset /path/to/native_dataset_root \
  --split val2017 \
  --output reports/native_dataset_wrapper \
  --force
python3 rtdetr_pose/tools/train_minimal.py \
  --dataset-root reports/native_dataset_wrapper \
  --config rtdetr_pose/configs/base.json \
  --max-steps 50
```

COCO keypoints roots can be selected explicitly when needed:

```
python3 -m yolozu doctor train-dataset \
  --from coco-keypoints \
  --dataset /path/to/coco_keypoints_root \
  --split val2017 \
  --output -
python3 -m yolozu import dataset \
  --from coco-keypoints \
  --dataset /path/to/coco_keypoints_root \
  --split val2017 \
  --output reports/coco_keypoints_wrapper \
  --force
```

12.2 The Run Contract: Rationale and implementation

A standardized run directory contains:

- Checkpoints (e.g., `runs/<run-id>/checkpoints/best.pt`)
- Exported models (e.g., ONNX formats)
- Predictions and evaluation reports
- Calibration statistics artifacts (when explicitly enabled)

This layout makes these operations easier:

- Reproducing experiment results
- Running parity checks
- Comparing runs across Git branches or compute nodes

Core rules of the run interface contract, documented here and enforced by the trainer:

- Activation via the `-run-contract` or `-run-id <id>` flags.
- Run-interface-contract mode requires explicit configuration keys and rejects unrecognized keys to prevent silent configuration drift.
- A deterministic artifact layout under `runs/<run_id>/`:
  - `checkpoints/last.pt`, `checkpoints/best.pt`
  - `reports/train_metrics.jsonl`, `reports/val_metrics.jsonl`
  - `reports/config_resolved.yaml`, `reports/run_meta.json`, `reports/training_summary.json`
- Optional export/parity artifacts (when export/parity is enabled): `exports/model.onnx`, `exports/model.onnx.meta.json`, `reports/onnx_parity.json`.
- Full-state resumption is standardized using the `-resume` flag, defaulting to `runs/<run_id>/checkpoints/last.pt`.

12.2.1 Artifact contract: required vs optional

Class	Path	Condition
Required	`runs/<run_id>/checkpoints/last.pt`	Always in contracted runs
Required	`runs/<run_id>/reports/train_metrics.jsonl`	Always in contracted runs
Required	`runs/<run_id>/reports/val_metrics.jsonl`	Always in contracted runs

Required	runs/<run_id> /reports/config_ resolved.yaml	Always in contracted runs
Required	runs/<run_id> /reports/run_meta. json	Always in contracted runs
Optional	runs/<run_id> /exports/model.onnx	Present when ONNX export is enabled
Optional	runs/<run_id> /exports/model.onnx. meta.json	Present when ONNX export metadata is emitted
Optional	runs/<run_id> /reports/onnx_parity. json	Present when ONNX parity check is executed
Optional	runs/<run_id> /reports/training_ summary.json	Present as the backend-neutral top-level summary interface contract

Note (low-level trainer): When invoking `python3 -m rtdetr_pose.train_minimal` directly, specifying `-run-id` implicitly activates run-contract mode and necessitates a `-config` argument (a YAML/JSON file such as `configs/examples/train_contract.yaml`). For rapid smoke tests bypassing the run contract, use explicit values such as `-run-dir runs/exp001` and `-onnx-out runs/exp001/model.onnx`.

12.3 Configuration resolution and strictness

The training entry point reads YAML/JSON configurations via `-config`, then applies explicit CLI overrides. The resolution hierarchy is:

CLI flags > config file > built-in defaults

Configuration keys must correspond exactly to their argparse destination names (e.g., `-weight-decay` maps to `weight_decay`). In run-contract mode (triggered by `run_contract` or `run_id`), any unrecognized keys are immediately rejected, thereby eliminating the risk of silent configuration drift.

12.4 YAML workflows: train / resume / test

A unified project structure supports YAML-driven execution for both training and testing scenarios.

Workflow	Primary config	Reference command
Train (starter config)	configs/examples/train_ setting.yaml	yolozu train <train_setting.yaml>
Train (run contract)	configs/examples/train_ contract.yaml	yolozu train <train_contract.yaml> -run-id <id>
Resume (full state)	Same as contract	yolozu train <train_contract.yaml> -run-id <id> -resume
Test (scenario runner)	configs/examples/test_ setting.yaml	yolozu test <test_setting.yaml> [test_args...]

Example execution commands:

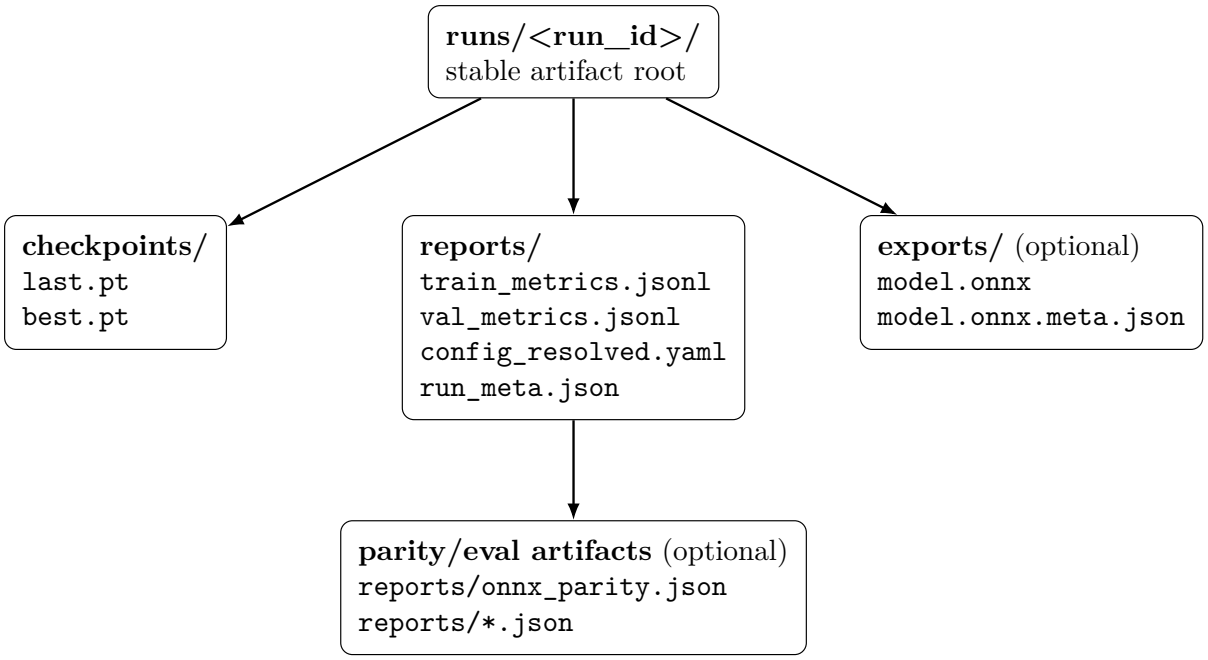


Figure 12.1: Run directory layout: fixed locations make resume, parity, and cross-branch comparisons reproducible.

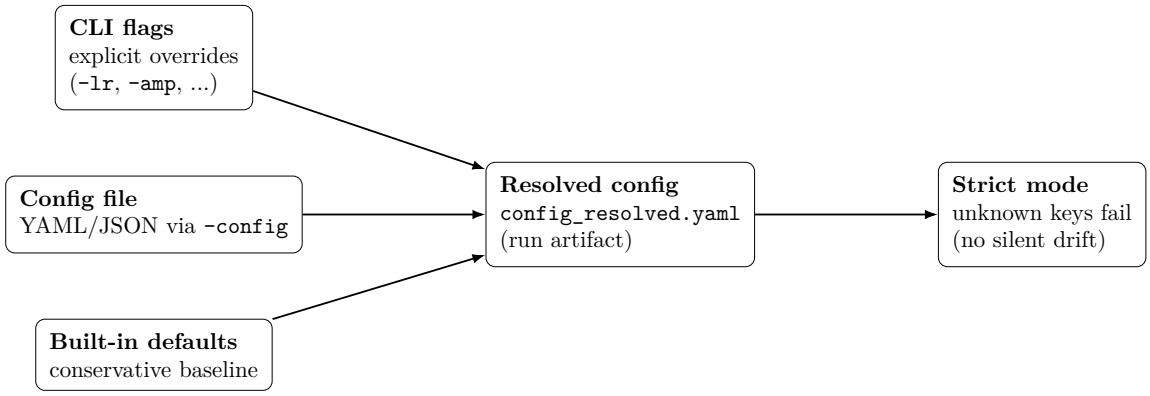


Figure 12.2: Configuration resolution: the run contract enforces explicit keys and produces a resolved config artifact.

```
yolozu train configs/examples/train_setting.yaml
yolozu train configs/examples/train_contract.yaml --run-id exp01
yolozu train configs/examples/train_contract.yaml --run-id exp01 --resume
yolozu test configs/examples/test_setting.yaml --adapter precomputed --predictions
    reports/predictions.json
```

For `yolozu test`, YAML keys are dynamically translated into `-key value` arguments (with booleans converted to flags), allowing subsequent CLI arguments to override them as necessary.

12.5 Backend interface and orchestration

The training platform now separates five concerns:

1. a canonical `TrainConfig` projection,
2. a backend registry,
3. a shared `training_summary.json` style interface contract,

4. a capability matrix, and
5. a lightweight orchestration entrypoint.

This means the reference trainer and the external lanes do not need to share one internal loop, but they do share one top-level description of what ran and what artifacts were promised.

For external lanes, YOLOZU now also standardizes one wrapper-level bundle under `work_dir/`:

- `dataset/`
- `configs/train_config_projection.json`
- `reports/training_summary.json`
- `reports/external_run_meta.json`
- `reports/launcher_plan.json`
- `reports/execution.json`
- `reports/resume_handoff.json`
- `reports/export_handoff.json`
- `reports/eval_handoff.json`
- `reports/parity_handoff.json`
- `reports/training_registry_entry.json`

This is not the same as forcing one backend-native checkpoint layout. It means the wrapper-owned interface contract is now fixed even when the actual trainer remains external.

The external training summary also records `next_steps`: copy-paste follow-up commands for resume, export, evaluation, and parity. The handoff JSONs make those next steps machine-readable, so orchestration and CI do not need to infer backend-specific conventions from prose. In practice, this makes the training report itself the hand-off note for the next operator step while still keeping the resume/export/eval/parity interface contract explicit.

OpenCV DNN and ONNX Runtime do not appear in this training registry because YOLOZU treats them as inference/export runtimes after training, not as training backends in their own right.

The repo-side orchestration command is:

```
yolozu train-orchestrate \
  --spec reports/train_orchestration_spec.json \
  --output reports/training_orchestration_report.json \
  --registry-out reports/training_registry.jsonl
```

When `-registry-out` is set, each executed training run appends one `yolozu_training_registry_entry_v1` record to a shared JSONL file. This is intentionally lightweight: it is an append-only experiment registry for cross-backend auditing, not a full experiment tracking server.

Read next:

- `docs/training_backend_interface.md`
- `docs/training_capability_matrix.md`
- `docs/training_orchestration.md`

12.6 External framework finetune smoke matrix

For user convenience, YOLOZU includes template configs and a unified smoke runner for external finetune entrypoints:

- YOLO-family runtime
- Detectron2
- MMDetection
- MMPose
- MMSeg
- NVIDIA TAO

- RT-DETR (`rtdetr_pose`)

Template paths:

- `configs/examples/finetune_external/yolo_runtime_yolov8n_finetune_smoke.yaml`
- `configs/examples/finetune_external/mmdetection_finetune_smoke.py`
- `configs/examples/finetune_external/mmpose_finetune_smoke.py`
- `configs/examples/finetune_external/mmseg_finetune_smoke.py`
- `configs/examples/finetune_external/tao_finetune_smoke.yaml`
- `configs/examples/finetune_external/detectron2_finetune_smoke.yaml`
- `configs/examples/finetune_external/rtdetr_pose_finetune_smoke.yaml`

Unified smoke command (interface contract report):

```
python3 tools/run_external_finetune_smoke.py \
  --dataset-root data/smoke \
  --split train \
  --output reports/external_finetune_smoke.json
```

Top-level Detectron2 training lane examples:

```
python3 -m yolozu train \
  --external-backend detectron2 \
  configs/examples/finetune_external/detectron2_finetune_smoke.yaml \
  --dataset data/smoke \
  --split val \
  --task-family bbox \
  --dry-run \
  --output reports/train_external_detectron2_bbox.json

python3 -m yolozu train \
  --external-backend detectron2 \
  /path/to/mask_rcnn_config.yaml \
  --dataset /path/to/dataset \
  --split train \
  --task-family segmentation \
  --train-opt DATASETS.TRAIN "(\"my_seg_train\",)" \
  --dry-run \
  --output reports/train_external_detectron2_seg.json

python3 -m yolozu train \
  --external-backend detectron2 \
  /path/to/keypoint_rcnn_config.yaml \
  --dataset /path/to/dataset \
  --split train \
  --task-family keypoints \
  --train-opt DATASETS.TRAIN "(\"my_kpts_train\",)" \
  --dry-run \
  --output reports/train_external_detectron2_keypoints.json
```

Execute real training for selected frameworks:

```
python3 tools/run_external_finetune_smoke.py \
  --dataset-root data/smoke \
  --split train \
  --non-dry-framework yolov \
  --non-dry-framework rtdetr \
  --epochs 1 --max-steps 1 --batch-size 2 --image-size 96 \
  --device cpu \
  --require-training-execution \
  --output reports/external_finetune_smoke.exec.json
```

MMDetection/Detectron2 can be wired to external launchers via `-mmdet-train-script` and `-detectron2-train-script`. RT-DETR non-dry dependency failures are explicit in the report (`failure_code=E_DEP_TORCH_MISSING`). When MMDetection/Detectron2 projection dependencies are absent, train-path audit can still continue with external launchers; the report records `projection_error` and `train_path_audited`.

GPU check (machine.dev style) example:

```
python3 tools/run_external_finetune_smoke.py \
  --dataset-root data/smoke \
  --split train \
  --non-dry-framework rtdetr \
  --device cuda \
  --epochs 1 --max-steps 1 --batch-size 2 --image-size 96 \
  --require-training-execution \
  --output reports/external_finetune_smoke.machine_dev.json
```

12.7 Solver (optimizer) parameters and defaults

The supported optimizers are `adamw` (the default) and `sgd`.

CLI / config key	Type	Default	Notes
`-optimizer` / `optimizer`	str	`adamw`	`adamw`, `sgd`
`-lr` / `lr`	float	`1e-4`	Base learning rate
`-weight-decay` / `weight_decay`	float	`0.01`	Base weight decay
`-momentum` / `momentum`	float	`0.9`	Applicable to SGD only
`-nesterov` / `nesterov`	bool	`false`	Applicable to SGD only
`-use-param-groups` / `use_param_groups`	bool	`false`	Enables separate backbone/head parameter groups
`-backbone-lr-mult` / `backbone_lr_mult`	float	`1.0`	Learning rate multiplier for the backbone
`-head-lr-mult` / `head_lr_mult`	float	`1.0`	Learning rate multiplier for the head
`-backbone-wd-mult` / `backbone_wd_mult`	float	`1.0`	Weight decay multiplier for the backbone
`-head-wd-mult` / `head_wd_mult`	float	`1.0`	Weight decay multiplier for the head
`-wd-exclude-bias` / `wd_exclude_bias`	bool	`true`	Sets bias decay to 0
`-wd-exclude-norm` / `wd_exclude_norm`	bool	`true`	Sets normalization decay to 0

12.8 LR scheduling parameters and defaults

Supported learning rate scheduler configurations include `none` (default), `cosine`, `onecycle`, and `multistep`.

CLI / config key	Type	Default	Notes
`-scheduler` / `scheduler`	str	`none`	`none`, `cosine`, `onecycle`, `multistep`

`-min-lr` / `min_lr`	float	`0.0`	Corresponds to cosine `eta_min`
`-scheduler-milestones` / `scheduler_milestones`	str/list	`""`	Comma-separated list for multistep scheduling
`-scheduler-gamma` / `scheduler_gamma`	float	`0.1`	Decay factor for multistep scheduling
`-lr-warmup-steps` / `lr_warmup_steps`	int	`0`	Number of linear warmup steps
`-lr-warmup-init` / `lr_warmup_init`	float	`0.0`	Initial learning rate during warmup

Note: The `linear` scheduler is not supported in the current codebase.

12.9 LoRA / QLoRA parameters and defaults

Low-Rank Adaptation (LoRA) is activated when `lora_r > 0`. For the plain-language intuition and the difference between LoRA and QLoRA, see Chapter 14 before tuning these knobs.

What these words mean in plain language.

- **LoRA** stands for **Low-Rank Adaptation**. Instead of updating every weight in a large model, LoRA adds a small trainable adapter on top of selected layers. In practice, this means "keep most of the base model fixed, and learn a compact correction."
- **Why people use LoRA**: it reduces memory cost, usually trains faster than full-model finetuning, and makes it easier to keep one base checkpoint while testing several small adaptation variants.
- `lora_r` is the adapter rank. A larger rank means a more expressive adapter, but also more trainable parameters.
- `lora_alpha` is a scaling factor for the adapter contribution. A simple mental model is: rank controls adapter capacity, alpha controls how strongly the adapter is allowed to matter.
- **QLoRA** means "LoRA on top of a quantized base model." In this repository, `-qlora` keeps the trainable LoRA adapters, but stores the frozen base model in a lower-precision quantized form to reduce memory further.
- **Why QLoRA is stricter**: because quantization and adapter training interact. That is why YOLOZU enforces the safe combination explicitly instead of pretending every LoRA run is also a safe QLoRA run.

A simple comparison.

- **Full finetuning**: move almost every parameter.
- **LoRA finetuning**: keep the base model mostly fixed and train small adapters.
- **QLoRA finetuning**: same adapter idea as LoRA, but the frozen base model is additionally quantized to save memory.

CLI / config key	Type	Default	Notes
`-lora-r` / `lora_r`	int	`0`	Values `>0` enable LoRA
`-lora-alpha` / `lora_alpha`	float/null	`null`	A null value implies $\alpha = r$
`-lora-dropout` / `lora_dropout`	float	`0.0`	Specifies LoRA input dropout
`-lora-target` / `lora_target`	str	`head`	Options: `head`, `all_linear`, `all_conv1x1`, `all_linear_conv1x1`

`-lora-freeze-base` / `lora_freeze_base`	bool	`true`	Restricts training exclusively to LoRA parameters
`-lora-train-bias` / `lora_train_bias`	str	`none`	Options: `none`, `all`
`-torchao-quant` / `torchao_quant`	str	`none`	Options: `none`, `int8wo`, `int4wo`
`-torchao-required` / `torchao_required`	bool	`false`	Triggers a failure if the quantization backend is absent
`-qlora` / `qlora`	bool	`false`	Requires LoRA; strictly enforces int4wo quantization and base freezing

12.10 Default configuration list (practical minimum)

The canonical default configurations are maintained within:

- `configs/examples/train_setting.yaml`
- `configs/examples/train_contract.yaml`
- `configs/examples/test_setting.yaml`

Frequently modified default parameters include:

Config key	Default	Meaning
`optimizer`	`adamw`	Optimizer family
`lr`	`1e-4`	Base learning rate
`weight_decay`	`0.01`	Base weight decay
`scheduler`	`none`	Learning rate scheduler mode
`lr_warmup_steps`	`0`	Warmup is disabled by default
`lora_r`	`0`	LoRA is disabled by default
`gradient_accumulation_steps`	`1`	Gradient accumulation is disabled
`clip_grad_norm`	`0.0`	Gradient clipping is disabled
`use_ema`	`false`	Exponential Moving Average (EMA) is disabled
`amp`	`none`	Automatic Mixed Precision (AMP) is disabled
`metrics_jsonl`	`reports/train_metrics.jsonl`	Output stream for training metrics
`metrics_csv`	`reports/train_metrics.csv`	Tabular format for metrics
`export_onnx`	`true`	ONNX export is enabled
`onnx_out`	`reports/rtdetr_pose.onnx`	Designated ONNX output path

12.11 Smoke training

For a repository-wide offline smoke, use:

```
bash scripts/smoke.sh
```

For training-entrypoint coverage across external frameworks, use the dedicated interface-contract smoke runner:

```
python3 tools/run_external_finetune_smoke.py \
```

```
  --dataset-root data/smoke \
  --split train \
  --output reports/external_finetune_smoke.json
```

The first command exercises the main offline smoke lane. The second focuses on finetune-entrypoint audit for YOLOv/MMDetection/Detectron2/RT-DETR style training surfaces and writes a machine-readable report.

12.12 Depth mode in training (current implementation)

The in-repo RT-DETR Pose trainer incorporates optional depth integration, configured with a conservative default:

- `-depth-mode none` (default): Executes the baseline pathway with depth processing disabled.
- `-depth-mode sidecar`: Ingests per-image depth sidecars (`depth_path`/`depth`) and propagates the `depth_valid` flag.
- `-depth-mode fuse_mid`: Implements lightweight depth fusion subsequent to the projector (external to the backbone boundary).

Safety controls:

- `-depth-unit \{unspecified,relative,metric\}` (default: `unspecified`).
- Absolute depth matcher costs are strictly permitted only in metric mode; non-metric modes automatically disable `cost_z`/`cost_t` for safety.
- `-depth-scale` applies a scalar multiplier to sidecar depth values.
- `-depth-dropout` regulates the modality dropout probability during `fuse_mid` training.

12.13 Contracted run example

```
yolozu train configs/examples/train_contract.yaml --run-id exp01

# Resume training from runs/exp01/checkpoints/last.pt
yolozu train configs/examples/train_contract.yaml --run-id exp01 --resume
```

12.14 Backbone modification (adapter-scoped architecture switch)

Backbone-specific configurations are not primarily governed by solver flags; rather, they are derived from the model configuration file specified by `model_config` in the training YAML. This section applies to the in-repo `rtdetr_pose` adapter boundary.

Backbone notes and examples are maintained in `docs/backbones.md`.

The backbone output contract strictly mandates three feature maps: `[P3, P4, P5]` with corresponding strides of `[8, 16, 32]`. Each level is independently projected to the transformer's `d_model` dimension via 1x1 convolutions, then processed by the RT-DETR-style FPN/PAN + hybrid encoder path.

Supported backbone architectures in the current codebase include: `cspresnet`, `tiny_cnn`, `cspdarknet_s`, `resnet50`, and `convnext_tiny`.

While legacy compatibility fields (e.g., `backbone_name`, `backbone_channels`) remain functional, the strongly preferred convention is to use `model.backbone.*` together with `model.projector.d_model`.

Location	Key	Role

Train YAML	`model_config`	References the model JSON (e.g., `rtdetr_pose/configs/base.json`)
Model JSON (`model.backbone.*`)	`name`	Specifies the backbone family (e.g., `cspresnet`, `cspdarknet_s`, `resnet50`, `convnext_tiny`)
Model JSON (`model.backbone.*`)	`norm`	Defines the normalization type (`bn|syncbn|frozenbn|gn`)
Model JSON (`model.backbone.*`)	`args`	Optional, backbone-specific constructor arguments
Model JSON (`model.projector.*`)	`d_model`	Sets the projection/output channel size for transformer input
Solver YAML/CLI	`backbone_lr_mult`	Learning rate multiplier applied to backbone parameters
Solver YAML/CLI	`backbone_wd_mult`	Weight decay multiplier applied to backbone parameters

Normalization guidelines:

- `bn`: The standard default for single-node training.
- `syncbn`: Highly recommended for multi-GPU environments utilizing small per-rank batch sizes.
- `frozenbn`: Provides stability when batch statistics must remain static.
- `gn`: A robust, batch-size-independent alternative.

Recommended workflow for backbone modification:

1. Duplicate `rtdetr_pose/configs/base.json` to create a project-local model JSON file.
2. Modify `model.backbone.name` and `model.backbone.args` as required.
3. Configure `model.projector.d_model` to match the intended transformer hidden size.
4. Update the training YAML's `model_config` to reference the newly created JSON file.
5. Initiate a smoke test with minimal settings (e.g., reduced `image_size` and `max_steps`) before scaling up.

Example implementation:

```
cp rtdetr_pose/configs/base.json rtdetr_pose/configs/my_backbone.json
# Edit model.backbone.name, model.backbone.args, and model.projector.d_model
    accordingly

yolozu train configs/examples/train_setting.yaml   --model-config rtdetr_pose/configs/
    my_backbone.json   --use-param-groups   --backbone-lr-mult 0.1
```

12.15 PyTorch utility wrappers

The `yolozu.training.torch_utils` module provides thin wrappers around core PyTorch APIs for common training and inference tasks:

- `amp_inference_context()` — wraps `torch.amp.autocast` for mixed-precision inference.
- `compile_model()` — wraps `torch.compile` with configurable backend/mode.
- `profile_callable()` — wraps `torch.profiler` and returns structured event summaries.
- `model_info()` — reports parameter counts, dtype breakdown, device, and buffer stats.
- `auto_device()` — auto-detects the best available device (CUDA → MPS → CPU).
- `seed_everything()` — sets all random seeds (Python, NumPy, PyTorch) for reproducibility.

```
from yolozu.training.torch_utils import (
    amp_inference_context, compile_model, model_info,
```

```
    auto_device, seed_everything,
)

seed_everything(42)
device = auto_device()
model = model.to(device)
compiled = compile_model(model, mode="reduce-overhead")
print(model_info(compiled))

with amp_inference_context(device.type):
    output = compiled(images)
```

12.16 AMP utilities for training loops

The `yolozu.training.amp_utils` module provides a thin wrapper around `torch.amp` that standardizes Automatic Mixed Precision setup across SDFT distillation, TTA/TTT loops, and custom training scripts.

- `amp_available()` — returns `True` when `torch.amp.autocast` is present (PyTorch 2.x).
- `make_amp_context(device_type, dtype, enabled)` — creates a paired `(autocast_factory, GradScaler|None)` tuple. A `GradScaler` is only returned for `float16` on CUDA; for `bfloat16`, MPS, or CPU the scaler is `None`.

```
from yolozu.training.amp_utils import make_amp_context

autocast_ctx, scaler = make_amp_context(device_type="cuda", dtype="float16")
with autocast_ctx():
    loss = model(inputs)
if scaler is not None:
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
else:
    loss.backward()
    optimizer.step()
```

12.17 Torchvision v2 transforms bridge

The `yolozu.training.transforms_bridge` module provides composable augmentation pipelines using `torchvision.transforms.v2` (stable since torchvision 0.17). These pipelines jointly transform images, bounding boxes, and keypoints in a single call.

- `transforms_v2_available()` — checks whether `torchvision.transforms.v2` is importable.
- `build_detection_transforms(size, hflip_prob, color_jitter, scale_range, ...)` — constructs a training pipeline (random resize, horizontal flip, colour jitter, normalize).
- `build_eval_transforms(size, ...)` — constructs an evaluation pipeline (resize + normalize only).

Both builders insert `v2.ToImage()` before `v2.Normalize()` so that raw PIL inputs are converted to tensors first, avoiding the common "Normalize requires tensor" error.

```
from yolozu.training.transforms_bridge import (
    build_detection_transforms, build_eval_transforms,
)

train_tfm = build_detection_transforms(size=(640, 640), hflip_prob=0.5)
eval_tfm  = build_eval_transforms(size=(640, 640))
```

Chapter 13

Hessian-based Refinement (Practical)

This chapter introduces per-detection numerical refinement utilizing a Hessian-based Gauss–Newton style solver, applied post-inference. This technique can significantly mitigate errors on challenging samples during offline analysis and controlled studies by applying iterative, localized corrections. It is optimally suited for offline and batch processing workflows where additional computational overhead is permissible. It requires both predictions and dataset context, and relies on configurable convergence and damping parameters.

Who should read this chapter: Read this chapter when you need post-inference local correction and want to understand when Hessian refinement helps more than it hurts.

Operator opening.
Goal Apply bounded post-inference local refinement to an already produced predictions artifact.
When to use Use this for offline analysis or controlled studies where small geometric corrections are worth extra compute.
Inputs A predictions artifact, dataset context, refinement settings, and an evaluation protocol for before/after comparison.
Command `python3 tools/refine_predictions_hessian.py -help` first, then run with explicit input, output, and report paths.
Outputs A refined predictions artifact plus a research report describing iterations, convergence, damping, and changed fields.
How to verify Validate both input and output artifacts, compare before/after metrics, and inspect whether updates stayed within the configured bounds.
Common failure modes Using refinement as a production default, missing dataset sidecars, unstable damping settings, excessive latency, or interpreting local correction as model retraining.

Readiness classification. Hessian refinement is a **research-oriented** path in YOLOZU. Use it as a controlled post-inference study or offline correction stage, not as the default production lane. See `docs/production_readiness.md`.

13.1 Scope and relation to Test-Time Training (TTT)

This chapter is strictly focused on per-detection numerical refinement executed after the primary inference phase. TTT, Tent, and MIM adaptation policies are comprehensively covered in Chapter 15 to prevent redundant guidance.

13.2 Mechanics of the Hessian solver

The Hessian solver refines prediction values on a per-detection basis, employing Gauss–Newton style updates stabilized by Levenberg–Marquardt damping.

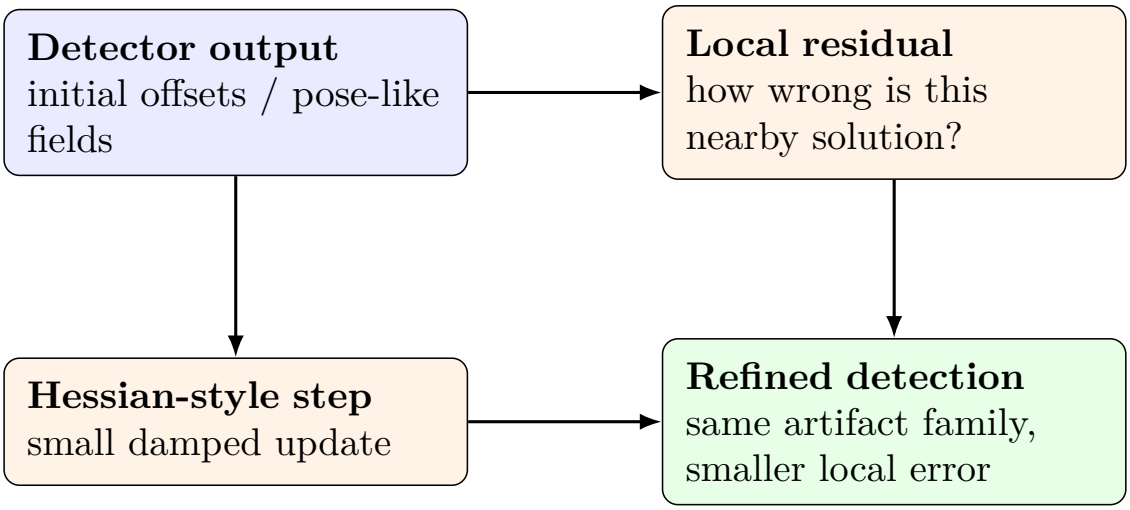


Figure 13.1: The practical picture: the model already produced an answer, and the Hessian solver asks whether a small local correction can make that answer geometrically cleaner.

The current standard CLI implementation operates as an engine-external post-processing step applied directly to predictions JSON artifacts, with bounding box offset refinement serving as the initial target.

At a high level, each iteration computes the residuals and the Jacobian matrix, solves a damped normal equation, updates the parameters, and terminates once predefined convergence criteria are satisfied.

13.3 Plain-language intuition

If gradient descent feels like following the local slope, Hessian-style refinement feels like also paying attention to the shape of the bowl around the current solution.

That is useful when the model is already close, but not quite aligned:

- the center offset is slightly wrong,
- depth is plausible but biased,
- rotation is nearly correct but still misaligned.

In other words, this chapter is about **small local corrections**, not about teaching the model a whole new concept.

13.4 Why pose workflows care

Pose outputs are often more sensitive than plain detection outputs. A box can still count as correct even when it is a little loose. By contrast, pose-style outputs can degrade quickly when:

- the center offset is wrong,
- the depth is slightly biased,
- the rotation is off by a small but important angle.

That is why the solver exists with pose use cases in mind, even though the current public CLI rollout deliberately starts from the safest operational target: **offset refinement**.

The broader design still reflects pose-oriented quantities:

- depth-like variables,
- rotation-like variables,
- offsets,
- intrinsics-related corrections.

So the honest operational summary is:

- **today's standard path**: offsets-first CLI refinement,
- **why it matters**: pose and geometry workflows often benefit from structured local correction.

13.5 Failure → local refinement example

The typical use case is not "the detector missed the object completely." It is the quieter case where the detector has already found the right object, but the local geometric state is still slightly wrong.

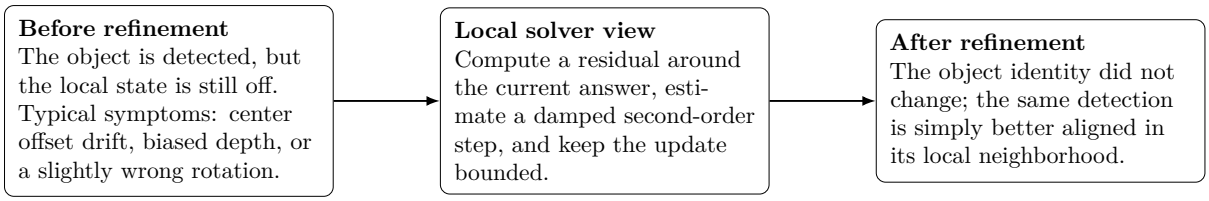


Figure 13.2: Concrete reading guide for Hessian refinement. This is not a "new object found" story; it is a "same object, cleaner local alignment" story. That distinction is why the method is especially relevant for pose-style outputs.

13.6 Optimal use cases

Recommended applications include:

- Offline validation and analysis scenarios where Ground Truth (GT) supervision or strict geometric constraints are accessible.
- Targeted error reduction specifically focused on highly difficult or anomalous samples.
- Post-inference refinement studies conducted subsequent to global calibration efforts.

It is strongly advised to avoid this technique by default in strict, real-time production environments due to the inherent latency overhead it introduces.

13.7 CLI quickstart

To enable refinement (opt-in behavior):

```
python3 tools/refine_predictions_hessian.py \
  --predictions reports/predictions.json \
  --dataset data/smoke \
  --output reports/predictions_refined.json \
  --enable \
  --refine-offsets \
  --wrap
```

To explicitly disable refinement (pass-through behavior):

```
python3 tools/refine_predictions_hessian.py \
  --predictions reports/predictions.json \
  --output reports/predictions_refined.json \
  --disable \
  --wrap
```

To enable refinement with customized solver parameters:

```
python3 tools/refine_predictions_hessian.py \
  --predictions reports/predictions.json \
  --dataset data/smoke \
  --output reports/predictions_refined.json \
  --enable \
  --refine-offsets \
  --steps 10 \
  --damping 1e-2
```

13.7.1 Workflow (misuse-prevention view)

1. Input predictions: `reports/predictions.json`
2. Gate selection: `-disable` → pass-through, `-enable` → iterative refine
3. Output predictions: `reports/predictions_refined.json`
4. Validate output contract: `yolozu validate predictions reports/predictions_refined.json -strict`

13.8 Core configuration parameters

Commonly tuned parameters (which correspond to the documentation in `docs/hessian_solver.md`):

- **Gate/Control**: `-enable`/`-disable`, `steps`, `damping`, `fd_eps`
- **Target toggles**: `refine_offsets` (prioritizing offsets-first rollout)
- **Residual weights**: `w_depth`, `w_mask`, `w_reg`
- **Configuration file**: `-config <yaml|json>` (Note: CLI arguments will override config file values)

13.9 Integration within the broader workflow

A highly practical operational sequence is:

1. Train the core model.
2. Optionally execute dataset-wide calibration (addressing global scale and intrinsics).
3. Apply Hessian refinement exclusively in scenarios where per-detection correction is demonstrably required.

13.10 Pros and Cons

Pros

- engine-external: it does not require changing the main inference graph,
- useful for offline analysis and controlled studies,
- especially intuitive for geometry-heavy predictions such as pose and depth.

Cons

- it adds latency and is not a default real-time production path,
- it needs a meaningful residual to optimize,
- it can become numerically fragile without damping and step caps.

This chapter highlights the most frequently utilized parameters; for an exhaustive detailing of the parameter surface, please consult the tool's `-help` output, which is rigorously maintained in sync with the codebase.

TensorRT Note: TensorRT conversion strictly targets the inference graph execution. Hessian refinement functions as a completely separate, engine-external post-processing operation.

Chapter 14

Research Workflows (End-to-end Loops)

This chapter defines repeatable research loops and the utilities for subsets, sweeps, and benchmarks. It keeps experiments artifact-first so results can be compared and regressed over time. It assumes stable JSON artifacts and, when needed, protocol presets. Keep subsets deterministic so comparisons stay fair and valid.

Who should read this chapter: Read this chapter when you are planning ablations, sweeps, offline distillation, or artifact-first research loops.

Operator opening.
Goal Run repeatable research loops while keeping the evaluation boundary artifact-first and comparable.
When to use Use this for ablations, frozen subsets, protocol-pinned comparisons, sweeps, prediction distillation, and research notes.
Inputs Stable JSON artifacts, a fixed dataset subset or protocol, and explicit output paths for reports and histories.
Command Start with validation and protocol-pinned eval; use the specific sweep, distillation, or benchmark helper only after artifacts are valid.
Outputs Suite reports, sweep histories, distilled predictions, benchmark summaries, and research-note inputs.
How to verify Check that subset seeds, protocol files, metric keys, and artifact paths are recorded before interpreting the result.
Common failure modes Changing subsets mid-study, mixing conceptual and executable commands, skipping validation, or comparing reports with different protocol settings.

14.1 The research loop in YOLOZU

A canonical research iteration proceeds as follows:

1. Build or acquire a model (either via internal training or external sourcing).
2. Export predictions that follow the stable JSON interface contract.
3. Validate artifacts immediately and fail fast.
4. Evaluate the artifacts under a pinned protocol (generating suite reports).
5. Run backend parity checks and compare historical runs with JSONL/CSV/MD histories.
6. (Optional) Apply post-hoc calibration and then re-evaluate.
7. (Optional) Execute hyperparameter sweeps and benchmark latency metrics.

The discipline is simple: **treat artifacts as first-class entities**. Every run must generate JSON files that support reproduction and comparison.

14.2 Freezing a dataset subset for ablations

Rapid ablation studies frequently require a deterministic subset of a larger dataset. YOLOZU provides a dedicated utility for this purpose:

```
python3 tools/make_subset_dataset.py   --dataset data/coco128   --split train2017   --
    n 50   --seed 0   --out reports/coco128_50
```

This command writes:

- A dedicated subset dataset directory.
- A frozen image list for reproducible sampling.
- A subset metadata JSON file for clear provenance.

14.3 Protocol-pinned evaluation (YOLO26 example)

Protocols pin evaluation settings so metrics remain comparable over time. For YOLO26, the protocol is defined in `protocols/yolo26_eval.json`; the evaluation tool reads this file and embeds its parameters into the final suite reports.

To execute a suite evaluation across multiple prediction artifacts:

```
python3 tools/eval_suite.py   --protocol yolo26   --dataset /path/to/yolo-dataset   --
    predictions-glob '/path/to/pred_*.json'   --output reports/eval_suite.json
```

The resulting report includes protocol metadata, such as split definitions and bounding box formats. This metadata is required for valid research comparisons.

14.4 Validating contracts as a research quality gate

Before investing time in metric interpretation, you must validate your artifacts:

```
python3 tools/validate_dataset.py --dataset /path/to/yolo --strict
python3 tools/validate_predictions.py /path/to/predictions.json --strict
```

This step is particularly critical when ingesting predictions exported from external pipelines.

14.5 Backend parity (Torch vs. ONNX Runtime vs. TensorRT)

When evaluating different inference backends, output alignment comes first. Use the parity tooling to compare two `predictions.json` artifacts and identify systematic differences, such as preprocessing, coordinate formatting, or numerical drift.

Conceptual (not copy-paste as-is): parity command shape. Use real artifacts and validate them first as shown in the quality-gate section above.

```
yolozu parity   --reference reports/pred_torch.json   --candidate reports/pred_onnxrt.
    json
```

14.6 Sweeps (Hyperparameter search harness)

Research workflows often need parameter sweeps that run outside the primary training loop. The sweep harness runs external commands, aggregates metrics, and writes summary tables.

Quick start:

```
python3 tools/hpo_sweep.py --config docs/hpo_sweep_example.json --resume
```

Default outputs include:

- `reports/hpo_sweep.jsonl`
- `reports/hpo_sweep.csv`
- `reports/hpo_sweep.md`

Use the `-resume` flag to make sweeps incremental and resilient to interruptions.

Critical configuration fields (detailed in `docs/hpo_sweep.md`):

- **Required**: `base_cmd`, alongside either `param_grid` or `param_list`.
- **Optional**: `run_dir`, `metrics.path`, `metrics.keys`.
- **Optional outputs**: `result_jsonl`, `result_csv`, `result_md`.
- **Optional runtime controls**: `env`, `shell`.

The harness is intentionally framework-agnostic; it makes no assumptions about the underlying training stack.

14.7 Prediction distillation workflow

YOLOZU includes a beginner-friendly *offline prediction distillation* helper at `tools/distill_predictions.py`. This section is intentionally written in the same spirit as the TTT chapter: start from the goal, then follow the concrete procedure, then inspect the resulting artifacts.

14.7.1 What this workflow is

Prediction distillation in this chapter means:

- take a **student** `predictions.json`,
- take a **teacher** `predictions.json`,
- compare them image-by-image,
- blend matched detections, optionally inject selected teacher-only detections,
- write a new distilled `predictions.json` plus a compact report.

The important point is that this is an **artifact-level** workflow. It does *not* update model weights. It is a fast research helper for answering:

> "If I trust this teacher more than the student on this subset, is there enough signal to justify a heavier training-time distillation setup?"

14.7.2 What this workflow is not

This section's *prediction distillation* is narrower than the self-distillation discussed in the continual-learning chapter and narrower than TTT:

- it is **not** checkpoint-based self-distillation during training,
- it is **not** an anti-forgetting method by itself,
- it is **not** changing model parameters at inference time,
- it is **not** a substitute for the full evaluator when reporting final numbers.

So the separation is:

- **Prediction distillation**: rewrite one prediction artifact using another prediction artifact.
- **Self-distillation**: train a checkpoint while regularizing it toward another checkpoint.
- **TTT / CTTA**: adapt model parameters in memory during inference-time operation.

14.7.3 A friendlier mental model

If you need to explain this workflow to a new teammate, the most useful sentence is:

> The student artifact is the first draft; the teacher artifact is the reviewer; distillation applies only a limited set of corrections and writes a new artifact.

In practice, each candidate object lands in one of three buckets:

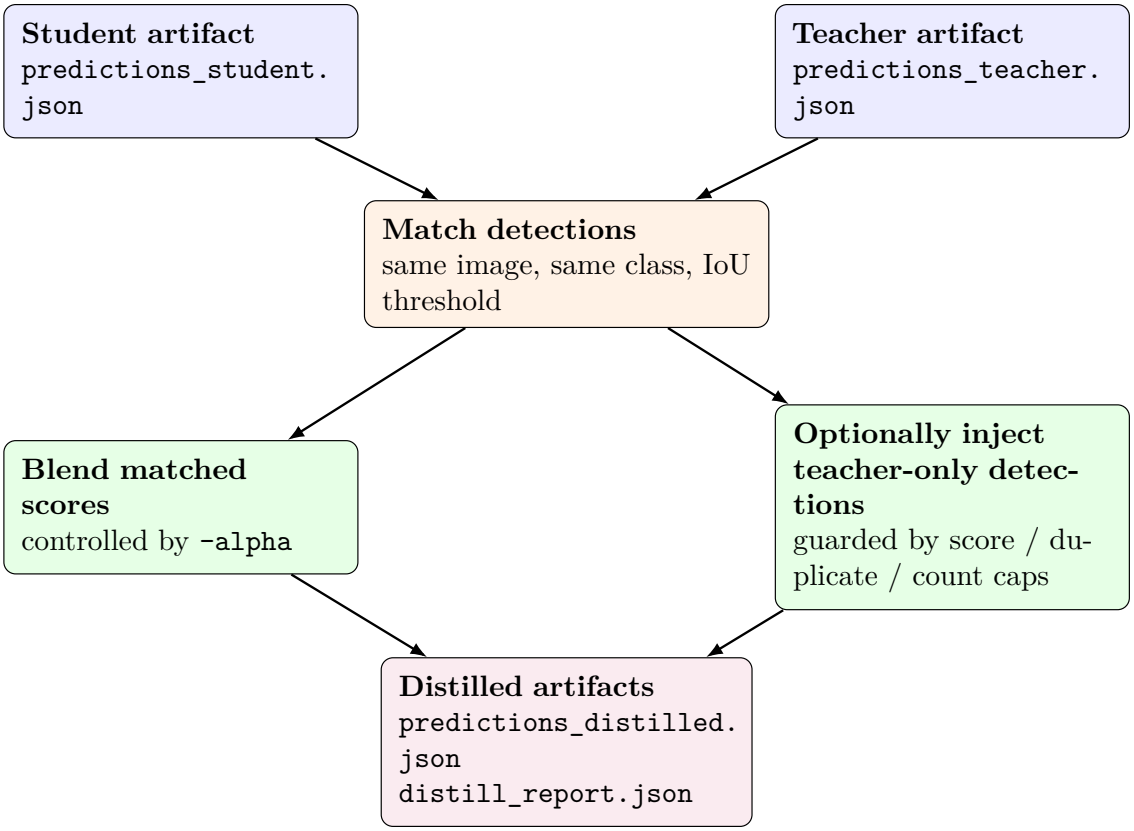


Figure 14.1: Prediction distillation flow in YOLOZU: keep the student artifact as the base, borrow teacher signal under explicit rules, then emit a new distilled artifact instead of mutating the originals.

- **Matched**: both student and teacher found the same object. This usually leads to a safer score update.
- **Teacher-only**: the teacher found an object that the student missed. This can improve recall, but it is also where false positives enter.
- **Student-only**: the student found an object that the teacher did not. The helper keeps the student artifact as the base.

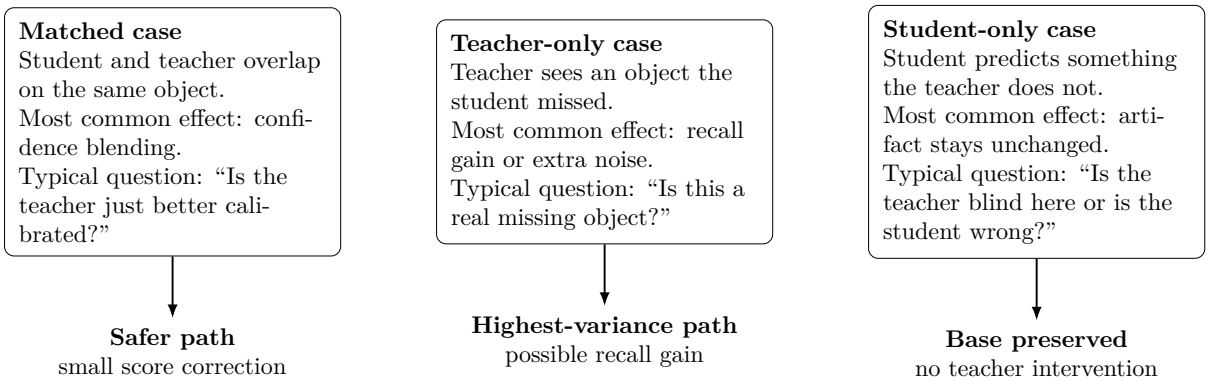


Figure 14.2: The three cases to look for when reading a distillation result. For beginners, this is often easier to reason about than starting directly from the final metric table.

14.7.4 Step-by-step procedure

Step 1: Prepare two valid artifacts You need one student predictions artifact and one teacher predictions artifact. Before distilling, validate both:

```
python3 tools/validate_predictions.py reports/predictions_student.json --strict
python3 tools/validate_predictions.py reports/predictions_teacher.json --strict
```

For operator convenience, `tools/distill_predictions.py` also accepts wrapped payloads with partial `meta` blocks and supplements missing stable keys (for example `adapter`, `timestamp`, `tta`, `ttt`) before validation. This makes checked-in research artifacts easier to reuse without hand-editing metadata first.

Step 2: Decide how aggressive the blend should be The most important knobs are:

- `-alpha`: how strongly matched detections move toward the teacher score.
- `-iou-threshold`: how much overlap teacher/student detections need before they count as the same object.
- `-add-missing`: whether teacher-only detections may be injected.
- `-teacher-min-score`: minimum confidence required before a teacher-only detection may be injected.
- `-max-added-per-image`: cap on teacher-only additions.
- `-add-duplicate-iou-threshold`: prevents near-duplicate teacher injections.

Safe-first reading strategy When in doubt, use this order:

1. run a conservative pass first,
2. inspect whether `matched` dominates `added`,
3. only then enable teacher-only injection,
4. if `added` dominates but the visual result looks noisy, treat the gain as fragile.

Step 3: Run a conservative first pass Start with a conservative run before enabling teacher-only injection:

```
python3 tools/distill_predictions.py \
  --student reports/predictions_student.json \
  --teacher reports/predictions_teacher.json \
  --dataset data/coco128 \
  --split val2017 \
  --alpha 0.5 \
  --iou-threshold 0.7 \
  --output reports/predictions_distilled.json \
  --output-report reports/distill_report.json
```

Step 4: Run an exploratory ablation if needed If you want to test whether teacher-only detections help, enable guarded injection:

```
python3 tools/distill_predictions.py \
  --student reports/predictions_student.json \
  --teacher reports/predictions_teacher.json \
  --dataset data/coco128 \
  --split val2017 \
  --alpha 0.5 \
  --iou-threshold 0.7 \
  --add-missing \
  --teacher-min-score 0.25 \
  --max-added-per-image 20 \
  --add-duplicate-iou-threshold 0.9 \
  --output reports/predictions_distilled.json \
  --output-report reports/distill_report.json
```

Step 4b: Prefer a checked-in YAML config when the CLI gets too long The helper also accepts `-config` in JSON or YAML format. For day-to-day use, the shortest recommended boilerplate is the checked-in YAML file: `configs/examples/distill_predictions.yaml`.

```
python3 tools/distill_predictions.py \
  --student reports/predictions_student.json \
  --teacher reports/predictions_teacher.json \
  --dataset data/coco128 \
  --split val2017 \
  --config configs/examples/distill_predictions.yaml \
  --output reports/predictions_distilled.json \
  --output-report reports/distill_report.json
```

This is the preferred operator path when the long explicit flag list becomes hard to remember or copy correctly.

Step 5: Read the outputs in the right order The recommended reading order is:

1. `distill_report.json`
2. `predictions_distilled.json`
3. a full evaluator report if the result looks promising

14.7.5 How the algorithm behaves

Matched detections When teacher and student detections overlap enough under `-iou-threshold`, the helper treats them as corresponding detections and blends scores using `-alpha`. This is usually the safer path because both artifacts already agree that there is probably an object there.

Teacher-only detections With `-add-missing`, the helper may inject detections that only the teacher predicted. This is where gains and risks both tend to come from.

Typical upside:

- recover missed objects,
- reveal useful teacher signal quickly.

Typical downside:

- propagate teacher false positives,
- grow duplicate boxes when guardrails are too loose,
- overstate benefit on a tiny subset.

14.7.6 How to read the report

The compact report is enough for first-pass interpretation. Inspect:

- `losses.distill_score_gap`
- `metrics.student.map50`
- `metrics.student.map50_95`
- `metrics.distilled.map50`
- `metrics.distilled.map50_95`
- `meta.matched`
- `meta.added`
- `meta.distill.*`

Interpretation hints:

- **high matched, low added**: mostly confidence refinement on objects both models found.
- **high added**: the result depends heavily on teacher-only injection.
- **metrics up with low added**: usually a healthier sign.
- **metrics up with very high added**: inspect duplicates and false positives carefully.

14.7.7 Pros and Cons

Workflow	Pros	Cons
Prediction distillation	Fast CPU-friendly ablation; no retraining required; preserves the predictions interface contract workflow; easy side-by-side artifact comparison	Post-hoc artifact rewrite only; gains can be fragile when injection dominates; does not solve forgetting by itself

14.7.8 Practical guardrails

Recommended defaults for safer use:

- keep `-teacher-min-score >= 0.2`,
- keep `-max-added-per-image` finite,
- keep `-add-duplicate-iou-threshold` high (for example, `0.85`–`0.95`),
- specify `-split` when the dataset root has multiple candidate splits,
- treat the lightweight `simple_map` result as a *quick signal*, not the final claim.

14.7.9 When to use prediction distillation vs TTT vs self-distillation

Goal	Recommended workflow
Quick offline teacher/student ablation	Prediction distillation

Adapt at inference time under domain shift	TTT / CTTA
Mitigate forgetting while training across tasks/domains	Continual learning with self-distillation / replay / PEFT
Learn a new checkpoint that actually absorbs teacher behavior	Training-time distillation

14.7.10 Background references

This helper is an artifact-level utility, not a claim of faithful reproduction of any single paper. The conceptual lineage still comes from the broader distillation literature:

- knowledge distillation: [8],
- self-distillation / Born-Again Networks: [3].

Within research loops, prediction distillation is most useful as a **fast feasibility check**: keep the dataset subset fixed, keep the downstream evaluator fixed, and use the distilled artifact to answer whether there is teacher signal worth pursuing further.

14.8 Latency/FPS benchmarks with historical tracking (JSONL)

Benchmark reports must be treated as formal artifacts. The latency harness generates a stable JSON report and can append results to a JSONL history file:

```
python3 tools/benchmark_latency.py --iterations 200 --warmup 20 --output reports
    /benchmark_latency.json --history reports/benchmark_latency.jsonl --notes '
    baseline on target HW'
```

This dual-output pattern (a single JSON report coupled with a JSONL history) is invaluable for systematically tracking performance regressions across successive commits and varying hardware configurations.

14.9 Run contract artifacts (Training research)

When utilizing the in-repository trainer, it is strongly recommended to employ the contracted run mode to ensure the generation of consistent artifacts. This methodology is comprehensively detailed in Chapter 7 (Training and the Run Contract).

Minimal example:

```
yolozu train configs/examples/train_contract.yaml --run-id exp01
```

Contracted outputs are systematically organized under `runs/exp01/` and encompass checkpoints, metrics JSONL files, ONNX exports, and parity reports.

14.10 Notes on long-tail calibration experiments

Post-hoc calibration frequently serves as a primary research axis. Utilize `yolozu calibrate` to generate calibrated prediction artifacts, enabling rigorous side-by-side comparisons of different methodologies. This process is thoroughly summarized in Chapter 6 (Score Calibration and Long-tail Adjustments).

Chapter 15

Continual Learning (Anti-forgetting)

This chapter covers continual fine-tuning across tasks and domains, and the evaluation of forgetting using recorded run artifacts. It helps mitigate catastrophic forgetting and makes per-task metrics comparable over a training sequence. It requires continual configs and run artifacts (for example `continual_run.json`); optional replay and LoRA settings change compute and forgetting behavior.

Who should read this chapter: Read this chapter when you are training across tasks or domains and need to reason about forgetting, replay, self-distillation, LoRA, or QLoRA.

Operator opening.

Goal Evaluate staged model updates across tasks or domains while measuring forgetting before any checkpoint promotion.

When to use Use this when training happens in phases and old-task behavior must remain visible alongside new-task learning.

Inputs Continual configs, task datasets or wrappers, prior checkpoint paths, optional replay buffers, and run/eval JSON artifacts.

Command Run `train_continual.py`, then `eval_continual.py` and `continual_decide.py` with explicit report paths.

Outputs `continual_run.json`, per-stage metrics, `continual_eval.json`, and a promotion decision report.

How to verify Confirm old-task and new-task metrics are present, forgetting thresholds are recorded, and promotion gates explain promote/review/hold.

Common failure modes Treating online updates as approved continual learning, missing old-task eval, weak replay provenance, stale teacher checkpoints, or unclear LoRA update scope.

Readiness classification. Continual learning in YOLOZU is a **research-oriented** lane with explicit promotion gates. It is supported for reproducible experiments and governed checkpoint promotion, but it is not the first production lane for most adopters. See `docs/production_readiness.md`.

15.1 What YOLOZU means by "continual"

Continual learning (CL) here means fine-tuning across a sequence of tasks/domains while mitigating catastrophic forgetting. In this repository, the reference implementation targets the in-repo `rtdetr_pose` training stack.

15.2 Core approach (baseline)

The default strategy combines:

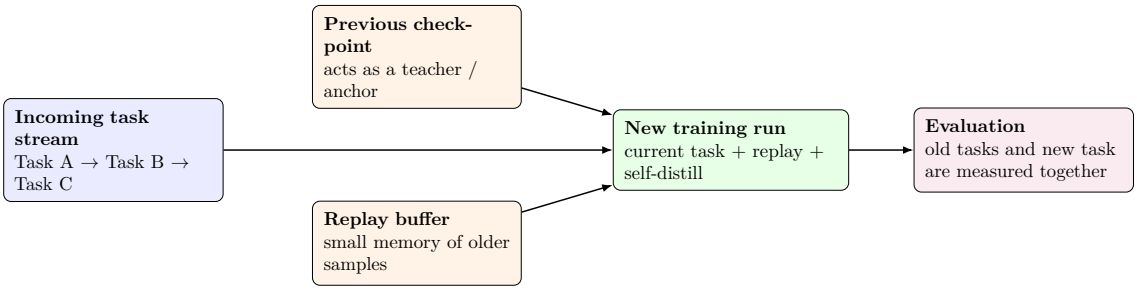


Figure 15.1: Continual learning in YOLOZU: the new run does not train on the latest task alone; it is anchored by the previous checkpoint and may also replay a small memory of older examples.

- **Memoryless self-distillation**: keep the new model close to the previous checkpoint while learning the new task.
- **Optional replay buffer**: add a small buffer of past samples (e.g., reservoir sampling) and train on *(current task + replay)*.
- **Optional parameter-efficiency**: restrict trainable parameters (e.g., LoRA) to reduce forgetting pressure.

15.3 Online learning, continual learning, and TTT are not the same thing

These three ideas are often grouped together because all of them react to distribution shift. Operationally, however, they solve different problems and carry different risk.

Approach	What changes	Main goal	Main operational risk
Online learning	model weights are updated close to inference time	adapt immediately to fresh data	bad labels or drift can corrupt production behavior quickly
Continual learning	model weights are updated in staged retraining runs	learn new tasks/domains while limiting forgetting	forgetting old behavior or promoting a weak checkpoint
TTT (test-time training / adaptation)	temporary inference-time adaptation or state adjustment	stabilize predictions on the current batch / scene	mixing short-lived adaptation with long-lived training decisions

The short version is:

- **online learning** is the fastest to react, but also the easiest to destabilize,
- **continual learning** is slower, but much easier to audit and govern,
- **TTT** is best thought of as *inference-time correction*, not as the main mechanism for long-term knowledge accumulation.

15.4 Safe design pattern in YOLOZU

For this repository, the safest default is **not** "train the production model while it is serving". Instead, use YOLOZU as a governed loop with explicit boundaries:

1. keep production inference separate from retraining,
2. collect inputs, predictions, and review signals as artifacts,

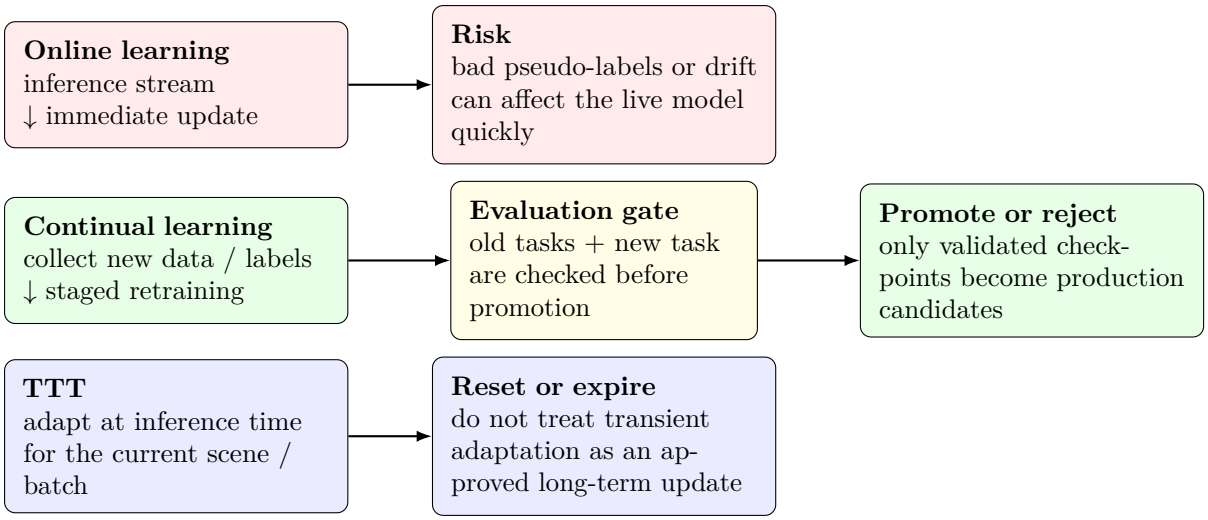


Figure 15.2: The practical difference: online learning changes production weights quickly, continual learning inserts an explicit evaluation gate before promotion, and TTT is a short-horizon inference-time adaptation path rather than a long-horizon training loop.

3. select candidate retraining data from those artifacts,
4. run continual-learning jobs off the serving path,
5. evaluate old-task retention and new-task adaptation before promotion.

This is where the repository fits naturally:

- **prediction capture**: keep outputs and settings under a predictions interface contract,
- **candidate selection**: choose low-confidence, failed, or newly labeled examples for retraining,
- **continual training**: use replay and/or self-distillation to reduce catastrophic forgetting,
- **evaluation**: compare old tasks and new tasks before a checkpoint is promoted.

15.4.1 Recommended control-plane split

The practical split is:

- **serving plane**: inference only,
- **data plane**: collect predictions, labels, and failure cases,
- **training plane**: run continual-learning jobs,
- **promotion plane**: gate model release on retention + adaptation metrics.

This separation matters because it prevents a weak pseudo-labeling step or a bad domain shift from mutating the live model immediately.

15.4.2 What to do with TTT

TTT is still useful in YOLOZU, but for a different reason:

- use it to stabilize predictions under a temporary shift,
- log what happened,
- do *not* automatically treat that short-lived adaptation as an approved training update.

In other words:

> TTT can help the current prediction, but continual learning should decide the next checkpoint.

15.4.3 A safe operator checklist

If you want to introduce adaptation without destabilizing production, the safest order is:

1. start by logging predictions and failures,
2. build a reviewed retraining set or a tightly filtered pseudo-label set,
3. run a separate continual-learning job,
4. compare retention on older tasks and adaptation on the new task,
5. promote only if the checkpoint passes the gate,
6. keep TTT separate from automatic checkpoint promotion.

For most teams, this means:

> prefer continual learning as the default production-update loop, and use online learning or TTT only where the risk is well understood and explicitly bounded.

15.4.4 Automating the promotion decision in YOLOZU

The repository can automate the *decision layer* even when the data curation and retraining steps remain separate. The intended pattern is:

1. run continual training,
2. run `eval_continual.py`,
3. feed the resulting eval report into `continual_decide.py`,
4. promote only if the decision report says `promote`.

Example:

```
python3 tools/continual_decide.py \
  --eval-json runs/continual/<run>/continual_eval.json \
  --run-json runs/continual/<run>/continual_run.json \
  --max-forgetting 0.05 \
  --min-new-task-score 0.40 \
  --min-old-task-final 0.40 \
  --min-reviewed-labels 20 \
  --min-highconf-pseudo-labels 50 \
  --min-total-curated-examples 60
```

For CI or batch ops, the same pattern is:

```
python3 tools/eval_continual.py \
  --run-json runs/continual/<run>/continual_run.json \
  --device cpu \
  --max-images 50

python3 tools/continual_decide.py \
  --eval-json runs/continual/<run>/continual_eval.json \
  --run-json runs/continual/<run>/continual_run.json \
  --curation-json reports/continual_curation.json \
  --max-forgetting 0.05 \
  --min-new-task-score 0.40 \
  --min-old-task-final 0.40 \
  --min-reviewed-labels 20 \
  --min-highconf-pseudo-labels 50 \
  --min-total-curated-examples 60
```

This writes a decision report (by default `runs/continual/<run>/continual_promotion_decision.json`) with one of three outcomes:

- `hold`: hard gate failed,
- `review`: hard gates passed, but a soft review gate still requires operator confirmation,
- `promote`: the checkpoint is eligible for promotion under the configured policy.

The soft-gate idea is deliberate. For example, reviewed labels and trusted pseudo-label counts can support a promotion decision, but they should not replace actual old-task/new-task evaluation.

Minimal curation summary:

```
{
  "counts": {
    "samples_total": 1200,
    "candidate_images": 90,
    "reviewed_labels": 48,
    "pseudo_labels_high_confidence": 120
  }
}
```

Recommended batch/CI curation summary:

```
{
  "counts": {
    "samples_total": 1200,
    "candidate_images": 90,
    "reviewed_labels": 48,
    "pseudo_labels_high_confidence": 120
  },
  "sources": {
    "review_queue": "reports/review_queue.json",
    "pseudo_label_run": "reports/pseudo_labels.json"
  },
  "serving": {
    "ttt_active": false
  }
}
```

If TTT was active on the serving path, the safe default is still to require review:

> use TTT to help the current prediction, but let continual-learning evaluation decide the next checkpoint.

15.5 LoRA / QLoRA in plain language

LoRA and QLoRA are not separate continual-learning algorithms. They are **ways to make the parameter updates smaller and cheaper**.

15.5.1 LoRA

The easiest mental model is:

> freeze most of the big model, then train a much smaller correction on top.

That is why LoRA is attractive in continual learning:

- a smaller trainable surface can reduce forgetting pressure,
- it is often easier to fit within a limited GPU budget,
- it gives a more controlled ablation than full-model fine-tuning.

What actually gets updated. This is the part that is easiest to miss if you only hear the slogan. LoRA usually keeps the original pretrained weight W frozen and learns a small low-rank correction:

$$W' = W + \Delta W, \qquad \Delta W = BA.$$

Here, A and B are the trainable adapter matrices. Operationally, that means:

- the backbone knowledge remains mostly anchored in the frozen weight,
- the new task/domain signal is expressed through a smaller correction,
- replay and self-distillation still do the anti-forgetting work; LoRA only narrows the update surface.

15.5.2 QLoRA

QLoRA keeps the same adapter idea, but stores the frozen base model in a more memory-efficient quantized form.

In plain language:

- **LoRA**: freeze the large model, train small adapters.
- **QLoRA**: do the same thing, but keep the frozen base cheaper to hold in memory.

So the update logic is still LoRA-style:

- the adapters move,
- the frozen base remains the reference,
- quantization changes storage/runtime cost, not the continual-learning objective.

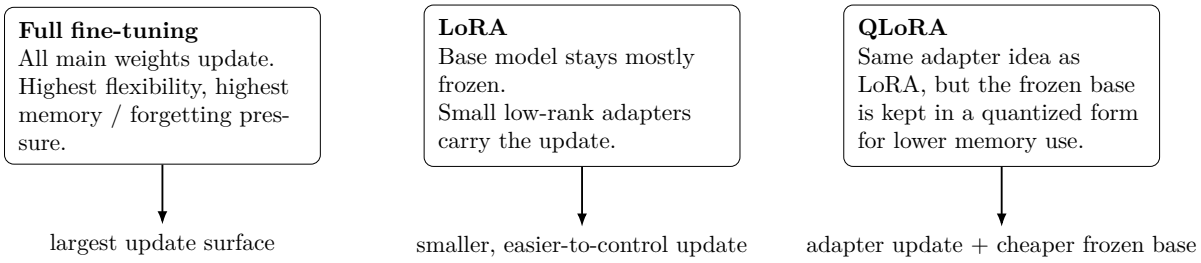


Figure 15.3: LoRA and QLoRA are best understood as parameter-efficiency tools. They change *how much* of the model moves during continual learning, not the underlying anti-forgetting objective itself.

15.5.3 Pros and Cons

LoRA

- Pros: simpler mental model than QLoRA; often cheaper than full fine-tuning; integrates naturally with replay and self-distillation.
- Cons: still introduces extra knobs; can underfit if the adapter is too small; may hide the fact that the base model is the real bottleneck.

QLoRA

- Pros: lowers memory pressure further; keeps the adapter-style workflow; attractive when the frozen base dominates footprint.
- Cons: adds quantization complexity; depends on backend support; is more experimental in this repository than plain LoRA.

15.5.4 A practical choice example

Imagine a domain-incremental warehouse-camera project:

- the previous checkpoint is already usable,
- the main concern is forgetting older camera views,
- GPU memory is limited, but not critically constrained.

In that situation, **LoRA is usually the better first experiment**. It keeps the story simpler: replay + self-distillation still do the anti-forgetting work, while the adapter merely reduces how much of the model moves.

Now change one assumption:

- the frozen base is the real memory bottleneck,
- batch size collapses unless the base becomes cheaper to hold.

That is when **QLoRA becomes the practical option**. You keep the same adapter-style update logic, but trade some simplicity for a smaller memory footprint.

So the operational rule is:

- **LoRA first when clarity and debugging speed matter**,
- **QLoRA when memory is the real blocker**.

15.6 Main entry points

15.6.1 Train

Run continual fine-tuning using a config (start from the example):

```
python3 rtdetr_pose/tools/train_continual.py \
  --config configs/continual/rtdetr_pose_domain_inc_example.yaml
```

To run **memoryless** (no replay), set replay size to 0 in the config.

Note on "distillation" terminology This chapter's *distillation* refers to **checkpoint-based self-distillation during training** (e.g., `-self-distill-from`) as an anti-forgetting mechanism. The separate tool `tools/distill_predictions.py` performs **offline prediction blending** (rewriting a predictions JSON) and does not by itself mitigate catastrophic forgetting.

15.7 How to add SDFT to an RT-DETR / YOLO-style image model

For detector-style image models, SDFT does **not** mean inventing a new architecture. It means adding two pieces to an existing training loop:

- a **frozen teacher checkpoint** from the previous stage,
- a **distillation loss** that keeps the current student close to that teacher while learning the new task.

15.7.1 The roles of student and teacher

The simplest mental model is:

- **student**: the model we are updating on the new task/domain,
- **teacher**: the previous checkpoint, loaded once and kept frozen.

So one training step becomes:

1. run the current image batch through the student,
2. run the same batch through the frozen teacher,
3. compute the usual task loss against labels,
4. compute a distillation loss between student outputs and teacher outputs,
5. add them together and update the student only.

In compact form:

$$\mathcal{L}_{total} = \mathcal{L}_{task} + \lambda \mathcal{L}_{distill}.$$

Here, λ controls how strongly the new run is pulled toward the previous checkpoint.

15.7.2 What to distill in detector-style models

For RT-DETR / YOLO-like models, the most practical distillation targets are:

- **classification logits**: keep class confidence behavior close to the previous checkpoint,
- **bounding boxes**: keep localization from drifting too far,
- optional task heads such as **segmentation**, **depth**, **keypoints**, or **pose**, when those outputs exist in both teacher and student.

The repository's default continual path describes the common baseline as:

logits + bbox distillation, with reverse-KL on logits by default.

That is a good first configuration because it directly targets the two outputs most likely to regress during detector fine-tuning: classification behavior and localization behavior.

15.7.3 What changes in the training loop

Conceptually, the existing training loop stays the same. The only difference is that the batch now has two forward passes:

```
student_outputs = student(images)

with torch.no_grad():
    teacher_outputs = teacher(images)

task_loss = compute_task_loss(student_outputs, targets)
distill_loss = compute_distill_loss(student_outputs, teacher_outputs)
total_loss = task_loss + lambda_distill * distill_loss

optimizer.zero_grad()
total_loss.backward()
optimizer.step()
```

Three practical rules matter:

- the teacher checkpoint is loaded once and kept in `eval()` mode,
- the teacher is never optimized,
- teacher and student must expose compatible output keys for the chosen distillation targets.

15.7.4 How this maps to YOLOZU

In this repository, the operator-facing hook is the frozen-teacher flag:

```
--self-distill-from /path/to/previous_stage.ckpt
```

That means the practical recipe is:

1. train Task A or Domain A normally and save a checkpoint,
2. start Task B / Domain B fine-tuning from the current initialization,
3. also pass the previous checkpoint as the frozen teacher,
4. keep the usual detection loss, and add distillation on logits / bbox (plus optional extra heads).

At the CLI level, the continual runner does this through the sequential training path:

```
python3 rtdetr_pose/tools/train_continual.py \
  --config configs/continual/rtdetr_pose_domain_inc_example.yaml
```

And the manual low-level mental model is:

- `train_minimal.py`: the per-stage trainer,
- `-self-distill-from`: the frozen teacher checkpoint input,
- continual config: controls replay, distillation keys, and distillation weight.

15.7.5 Why this helps

Without SDFT, fine-tuning on the new task can move detector outputs too aggressively toward the new data distribution. In practice, that often shows up as:

- older classes becoming miscalibrated,
- box quality drifting on previous domains,
- task-specific heads regressing even when the backbone still looks reasonable.

SDFT does not stop the model from learning the new task. Instead, it adds a second pressure:

learn the new data, but do not move so far that the previous detector behavior collapses.

15.7.6 A minimal operator recipe

If you already have an existing RT-DETR / YOLO-style checkpoint, the operational sequence is:

1. keep the previous checkpoint,
2. fine-tune on the new dataset/domain,
3. pass the previous checkpoint as `-self-distill-from`,
4. start with logits + bbox distillation,
5. only later add replay, LoRA, or extra task-head distillation if needed.

This is usually the cleanest first experiment because it answers the central question quickly:

> Can the model adapt to the new data without forgetting too much of the previous detector behavior?

15.7.7 Measured smoke run: naive sequential fine-tune vs memoryless SDFT

To make the workflow concrete, we ran a tiny two-task continual-learning smoke experiment in this repository. This was not intended as a benchmark. Its purpose was to verify that the SDFT path really executes, records the teacher checkpoint, and emits the expected continual-learning artifacts.

Setup. We created two tiny task roots from `data/coco128`:

- Task A: 8 train images + 4 val images,
- Task B: 8 train images + 4 val images.

Both runs used the same detector config and the same tiny budget:

- device: CPU,
- image size: 160,
- batch size: 1,
- max steps per task: 2,
- replay: disabled (`replay_size: 0`).

We then executed two runs:

- **naive sequential fine-tune**: Task B resumes from Task A, but uses no teacher checkpoint,
- **memoryless SDFT**: Task B resumes from Task A and also receives `-self-distill-from` pointing to the Task A checkpoint.

Commands. The measured runs were produced with:

```
python3 rtdetr_pose/tools/train_continual.py \
  --config /tmp/yolozu_sdft_manual_naive.yaml \
  --run-dir runs/continual/manual_sdft_smoke_naive

python3 rtdetr_pose/tools/train_continual.py \
  --config /tmp/yolozu_sdft_manual_sdft.yaml \
  --run-dir runs/continual/manual_sdft_smoke_sdft
```

And evaluated with:

```
python3 tools/eval_continual.py \
  --run-json runs/continual/manual_sdft_smoke_naive/continual_run.json \
  --device cpu \
  --image-size 160 \
  --max-images 4 \
  --metric simple_map \
```

```
  --force

python3 tools/eval_continual.py \
  --run-json runs/continual/manual_sdft_smoke_sdft/continual_run.json \
  --device cpu \
  --image-size 160 \
  --max-images 4 \
  --metric simple_map \
  --force
```

Observed results. The important thing to notice is not a claimed accuracy gain, but that the SDFT path became visible in the recorded outputs:

Run	Task B teacher	Task B final loss	Extra recorded loss terms
Naive	none	2.673	none
SDFT	Task A checkpoint	1.975	`loss_sdft`=0.187, `loss_sdft_bbox`=0.057, `loss_sdft_logits`=0.130

More specifically:

- `runs/continual/manual_sdft_smoke_sdft/task01_task_b/run_record.json` recorded `self_distill_from` pointing to the Task A checkpoint.
- `runs/continual/manual_sdft_smoke_sdft/task01_task_b/metrics.json` contained `loss_sdft`, `loss_sdft_bbox`, and `loss_sdft_logits`, which do not appear in the naive run.
- Both runs wrote the expected continual-learning outputs: `continual_run.json`, per-task checkpoints, per-task metrics, and `continual_eval.json`.

What this smoke run does *not* prove. Because this validation run used only two optimization steps per task and only four validation images per task, the proxy mAP matrix stayed at zero for both runs. That means this example should be read as a *wiring and artifact validation*, not as evidence that SDFT improved accuracy in this tiny setting.

The right interpretation is:

> the teacher path executed correctly, the distillation loss was active on Task B, and the continual-learning artifacts were emitted in a form that can be inspected and compared.

15.7.8 Evaluate

Evaluate forgetting and per-task metrics from the recorded run JSON:

```
python3 tools/eval_continual.py \
  --run-json runs/continual/<run>/continual_run.json \
  --device cpu \
  --max-images 50
```

If your dataset provides pose sidecar metadata (e.g., under `labels/<split>/*.json`), you can request pose metrics:

```
python3 tools/eval_continual.py \
  --run-json runs/continual/<run>/continual_run.json \
  --device cpu \
  --max-images 50 \
  --metric pose \
  --metric-key pose_success
```

Available `-metric-key` values in pose mode:

- `pose_success`, `rot_success`, `trans_success`, `match_rate`, `iou_mean`
- `depth_abs_mean`, `depth_abs_median`
- `add_mean`, `add_median`, `adds_mean`, `adds_median`

Notes:

- Depth metrics require GT depth/translation and predicted depth/translation fields.
- ADD/ADDS metrics require CAD points in sidecar metadata (`cad_points` / `cad_path` / `cad_points_path`) and valid GT+pred pose.

Examples for metric-key switching:

```
# depth error summary
python3 tools/eval_continual.py \
  --run-json runs/continual/<run>/continual_run.json \
  --metric pose \
  --metric-key depth_abs_mean

# ADD / ADDS (requires CAD points)
python3 tools/eval_continual.py \
  --run-json runs/continual/<run>/continual_run.json \
  --metric pose \
  --metric-key add_mean

python3 tools/eval_continual.py \
  --run-json runs/continual/<run>/continual_run.json \
  --metric pose \
  --metric-key adds_mean
```

15.8 Run artifacts (what to version / what to compare)

`train_continual.py` writes a run folder under `runs/continual/`. Key outputs:

- `continual_run.json`: task list, checkpoints, config hash, and run record.
- `replay_buffer.json`: buffer summary (when replay is enabled).
- Per-task subfolders, typically including:
  `checkpoint.pt`, `metrics.jsonl`, `metrics.csv`, `run_record.json`.

For reporting and plots, treat `continual_run.json` as the single source of truth.

15.9 Selected config knobs (high leverage)

Commonly tuned items include:

- Replay: `replay_size`, `replay_fraction`, `replay_per_task_cap`.
- Distillation: `distill.enabled`, `distill.keys`, `distill.weight`, `distill.temperature`.
- Replay distillation (DER++-style): `derpp.enabled` and related keys (requires teacher outputs stored in records).
- Regularizers across tasks (optional): EWC (`ewc.*`) and SI (`si.*`).

15.10 Research workflow guidance

- **Pin the evaluation subset**: use a deterministic subset (same images each run) before comparing CL runs.
- **Log runtime cost**: replay and distillation change throughput; report accuracy vs cost.
- **Keep baselines simple**: compare (A) naive fine-tune, (B) memoryless self-distill, (C) replay+distill.

15.11 Notes

- Some continual comparisons use a CPU-friendly proxy metric (e.g., a lightweight mAP-style summary) on a pinned subset to keep iteration fast.
- For "real" mAP, switch your evaluation workflow to a full COCO evaluator when appropriate.

15.12 References (self-distillation background)

This repository's checkpoint-based self-distillation is a pragmatic anti-forgetting regularizer. For background reading:

- Knowledge distillation (original): [8].
- Self-distillation (classic reference): [3].
- Continual learning via self-distillation (SDFT): [23].
- LoRA as parameter-efficient tuning: [11].
- QLoRA as quantized adapter tuning: [1].

Chapter 16

Test-Time Training (TTT): Tent, MIM, CoTTA, EATA, SAR

This chapter explains controlled test-time adaptation (Tent/MIM/CoTTA/EATA/SAR), presets, guard rails, and reset policies for fair comparisons. It can improve robustness under domain shift by adapting weights in memory, while keeping cost bounded and results reproducible. It is most relevant when there is clear domain shift; comparisons require fixed image sets, bounded `-ttt-max-batches`, and careful seed and reset policy control.

Who should read this chapter: Read this chapter when you are adapting models under domain shift and want a method-by-method guide to Tent, MIM, CoTTA, EATA, and SAR.

Operator opening.

Goal Run bounded test-time adaptation experiments and compare before/after artifacts under a fixed target shift.

When to use Use this when the model faces domain shift and you want controlled Tent, MIM, CoTTA, EATA, or SAR comparisons.

Inputs A fixed target image set, baseline export settings, TTT preset or method knobs, seed/reset policy, and output report paths.

Command `bash scripts/ttt_compare.sh -help` for the compare route, or `yolozu export -ttt ...` with bounded TTT flags.

Outputs Baseline and adapted predictions, compare JSON/Markdown, TTT logs, and optional method-specific research reports.

How to verify Check changed image counts, final losses, guard warnings, reset policy, runtime cost, and before/after metrics on the same target set.

Common failure modes No real domain shift, broad update filters, missing reset policy, unbounded streaming adaptation, or mixing TTA with TTT claims.

Readiness classification. TTT is a **research-oriented** lane in YOLOZU. Use it as an inference-time adaptation mode with bounded cost and explicit review, not as the default production update loop. See `docs/production_readiness.md`.

16.1 TTT Quickstart (operator first)

This is the shortest safe route through the chapter:

1. freeze a small target subset,
2. run one baseline export and one adapted export,
3. open the before/after compare markdown,
4. only then inspect raw TTT logs or method internals.

Start with the compare wrapper instead of hand-building a long export command:

```
bash scripts/ttt_compare.sh \
  --boilerplate tent \
  --dataset data/smoke \
  --split val \
  --checkpoint /path/to.ckpt \
  --run-dir reports/ttt_compare/tent \
  --device cpu
```

Expected outputs:

- `reports/ttt_compare/tent/baseline_predictions.json`
- `reports/ttt_compare/tent/tent_predictions.json`
- `reports/ttt_compare/tent/tent_before_after_compare.md`
- `reports/ttt_compare/tent/tent_ttt_log.json`

How to verify the run:

- open the compare markdown first,
- check that `steps_run > 0`,
- check `changed_images` and the before/after metric fields,
- confirm the same dataset split, checkpoint, seed, and reset policy were used.

Common failures:

- the target subset has no real domain shift, so predictions do not change,
- the checkpoint path is missing or incompatible with the config,
- the update filter is too broad for a first run,
- stream reset is used without an explicit batch cap.

16.2 Scope and support

YOLOZU supports controlled test-time training (TTT) for the `rtdetr_pose` adapter via `tools/export_predictions.py` or the canonical `yolozu export` / `python3 -m yolozu export` surface.

Authoritative references:

- `docs/ttt_protocol.md`
- `docs/ttt_integration_plan.md`
- `docs/mim_inference.md`

TTT updates weights *in memory* using unlabeled test data before (or during) inference [24, 26]. This makes comparisons sensitive to domain shift, random seeds, and reset policy.

16.2.1 Plain-language glossary for this chapter

Many readers first get stuck not on the code, but on the names. The short meanings below are enough to read the rest of this chapter:

- **TTA** (**Test-Time Augmentation**): run inference several times on simple transformed versions of the same image, then merge the predictions. This does *not* update model weights.
- **TTT** (**Test-Time Training**): update some model parameters in memory at inference time using unlabeled test data.
- **CTTA** (**Continual Test-Time Adaptation**): a streaming form of TTT where adaptation continues over many incoming batches instead of resetting after every image.
- **LoRA**: a small trainable adapter added to selected layers, used when you want limited, safer parameter updates instead of moving the full model.
- **Norm-only update**: adapt only normalization-layer affine parameters (for example BatchNorm or LayerNorm scale/bias). This is one of the safest update scopes.

- **EMA** (**Exponential Moving Average**): a smoothed teacher copy of model weights. It changes more slowly than the student and is often used to stabilize adaptation.
- **Reset policy**: when and how the adapted weights are restored. `sample` means "reset after each image/subset"; `stream` means "carry updates forward across the stream."
- **Guard rail**: a safety rule that limits adaptation, for example restricting which parameters may update or stopping updates when sample selection looks unreliable.

What the method names mean at a glance.

- **Tent**: minimize prediction entropy so the model becomes more confident on shifted inputs.
- **MIM**: use a masked-reconstruction objective so the model must infer hidden structure, not only sharpen confidence.
- **CoTTA**: a continual TTT method that combines student updates, an EMA-style teacher, augmentation agreement, and partial restoration so the model does not drift too far.
- **EATA**: an efficiency/reliability-oriented TTT method that adapts only on selected samples instead of updating on everything.
- **SAR**: a sharpness-aware robustness method; in practical terms, it tries to adapt in directions that are locally more stable instead of overreacting to noisy gradients.

How to read the rest of the chapter. If you only want the shortest reading path, use this order:

1. understand whether you need **TTA** or **TTT**,
2. understand the **update scope** (norm-only, LoRA, or full-model),
3. then compare the methods (Tent, MIM, CoTTA, EATA, SAR).

This order matters because many failures blamed on the method are actually caused by an overly broad update scope or an unsafe reset policy.

16.2.2 Operator quick guide (read first)

- **Default OFF**: TTT is opt-in and only activates when `-ttt` is set.
- **Default update scope**: start with safe presets (norm-only / adapter-safe). Keep CoTTA/EATA/SAR constrained to LoRA/Norm-safe subsets first.
- **Reset recommendation**: for controlled comparisons, prefer `-ttt-reset sample`. For streaming, use `-ttt-reset stream` only with an explicit batch cap.

16.3 When TTT is meaningful

On clean COCO-style validation, TTT can be neutral or harmful. To demonstrate improvements, you typically need a clear domain shift (e.g., corruptions, style shift, different camera).

Practical baseline:

- Baseline domain: clean COCO `val2017`
- Target domain: shifted dataset or corrupted copy of COCO images

16.3.1 Deterministic shift-target recipe

For reproducible TTT evidence, generate a deterministic corrupted target split with:

```
python3 scripts/prepare_ttt_domain_shift_target.py \
  --dataset-root data/smoke \
  --split val \
  --out reports/domain_shift/smoke_gaussian_blur_s2 \
  --corruption gaussian_blur \
  --severity 2 \
  --seed 2026 \
```

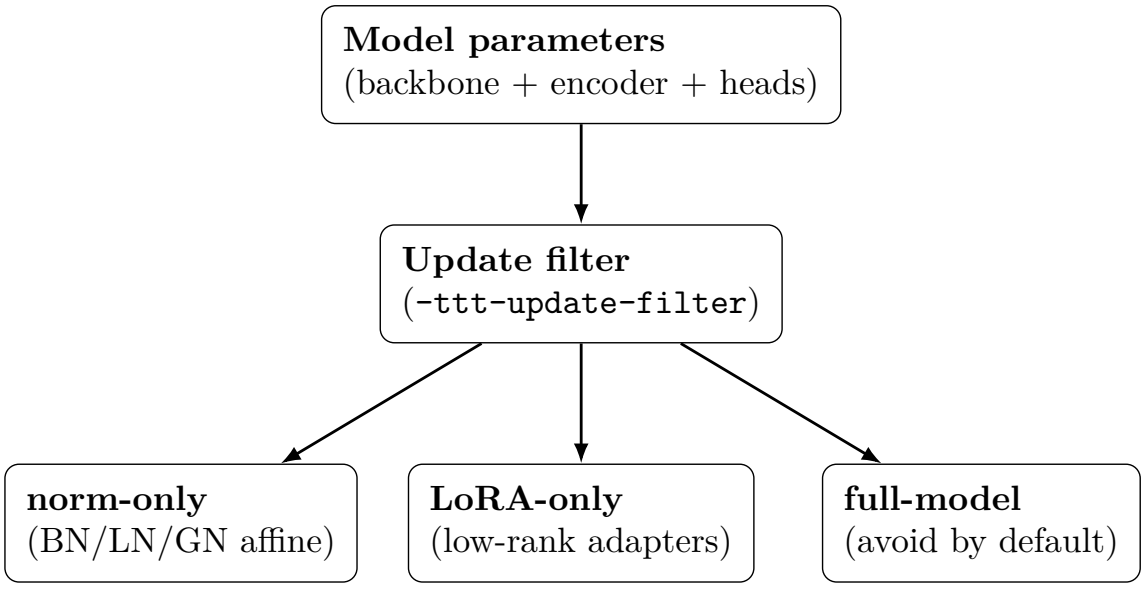


Figure 16.1: TTT update scope: start with constrained update filters (norm/LoRA) and keep full-model updates opt-in.

```
  --force
```

This produces `domain_shift_recipe.json` containing `export_settings.domain_shift_target` with fixed (`corruption`, `severity`, `seed`, `images_sha256`).

To bind that recipe into prediction artifacts:

```
python3 tools/export_predictions.py \
  --adapter dummy \
  --dataset reports/domain_shift/smoke_gaussian_blur_s2 \
  --split val \
  --wrap \
  --domain-shift-recipe reports/domain_shift/smoke_gaussian_blur_s2/
    domain_shift_recipe.json \
  --output reports/pred_shift_target.json
```

With `-wrap`, outputs explicitly include:

- `meta.export_settings.domain_shift_target`
- `meta.export_settings.domain_shift_recipe` (`path`, `sha256`)

16.3.2 Real result summary: use this first

The first thing beginners need is not a noisy overlay, but a controlled *result summary*. Figure 16.2 is therefore the primary onboarding figure for this chapter. It uses the same few-shot checkpoint, the same ten shifted images, and the same seed for every method. That means the bar chart is a controlled comparison, not a one-off anecdote. The checked-in PNG figures are regenerated with `tools/render_ttt_manual_figures.py`.

Command summary (deterministic target + same checkpoint, with and without TTT):

```
# 1) Prepare a deterministic shifted target split (1 image for illustration).
python3 scripts/prepare_ttt_domain_shift_target.py \
  --dataset-root data/smoke \
  --split val \
  --out reports/domain_shift/smoke_gaussian_blur_s3_1img \
  --corruption gaussian_blur \
  --severity 3 \
```



```
  --seed 2026 \
  --max-images 1 \
  --force

# 2) Inference overlays without TTT.
python3 -m yolozu predict-images \
  --backend torch --device cpu \
  --config rtdetr_pose/configs/base.json \
  --checkpoint reports/rtdetr_pose_ckpt_coco128_gpu_matcher.pt \
  --image-size 320 --score-threshold 0.05 --max-detections 20 \
  --input-dir reports/domain_shift/smoke_gaussian_blur_s3_1img/images/val \
  --max-images 1 \
  --overlays-dir reports/ttt_demo/no_ttt_overlays \
  --output reports/ttt_demo/pred_no_ttt.json \
  --force

# 3) Same inference but with TTT enabled (Tent safe preset, reset per sample).
python3 -m yolozu predict-images \
  --backend torch --device cpu \
  --config rtdetr_pose/configs/base.json \
  --checkpoint reports/rtdetr_pose_ckpt_coco128_gpu_matcher.pt \
  --image-size 320 --score-threshold 0.05 --max-detections 20 \
  --input-dir reports/domain_shift/smoke_gaussian_blur_s3_1img/images/val \
  --max-images 1 \
  --overlays-dir reports/ttt_demo/ttt_overlays \
  --output reports/ttt_demo/pred_ttt.json \
  --ttt --ttt-preset safe --ttt-reset sample --ttt-seed 2026 \
  --ttt-log-out reports/ttt_demo/ttt_log.json \
  --force
```

16.3.3 Metric micro-demo: show a delta (recommended)

The visual example above shows *what changes*. For onboarding and regression checks, it is also useful to show *a small metric delta* on a deterministic shift target.

YOLOZU provides a self-contained micro-demo that:

- trains a tiny few-shot checkpoint on clean `data/smoke` first (so no external weights download is required),
- applies a deterministic corruption to create a shifted target split,
- evaluates a simple mAP proxy for **no TTT** vs **with TTT** (and writes overlay PNGs).

Listing 16.1: TTT improvement micro-demo: few-shot train + deterministic shift + metric delta.

```
# pip users:
python3 -m pip install -U 'yolozu[demo]'
yolozu demo ttt

# repo checkout equivalent:
python3 -m yolozu demo ttt
```

Outputs include:

- `demo_output/ttt/<utc>/ttt_improvement_report.json` (with `metrics.delta`),
- `demo_output/ttt/<utc>/overlay_no_ttt.png` and `overlay_ttt.png`.

Example stdout (CPU, deterministic seeds; values are intentionally tiny, the point is the reproducible delta):

- `map500.00326797->0.00392157`
- `map50_950.000326797->0.000392157`

Figure 16.2: Primary TTT result figure. Real fixed-subset result summary from the ten-image shifted probe in `reports/ttt_improvement_probe`. Tent, CoTTA, EATA, and SAR all improve from `map50=0.00326797` to `0.00392157` and from `map50_95=0.000326797` to `0.000392157` under the same checkpoint, subset, and seed. MIM is shown separately as execution evidence because the current repo-backed MIM smoke fixture proves the masked-reconstruction path runs (`steps_run=2`, `mean_final_loss=0.461853`) but does not change exported detections on its tiny two-image smoke subset.

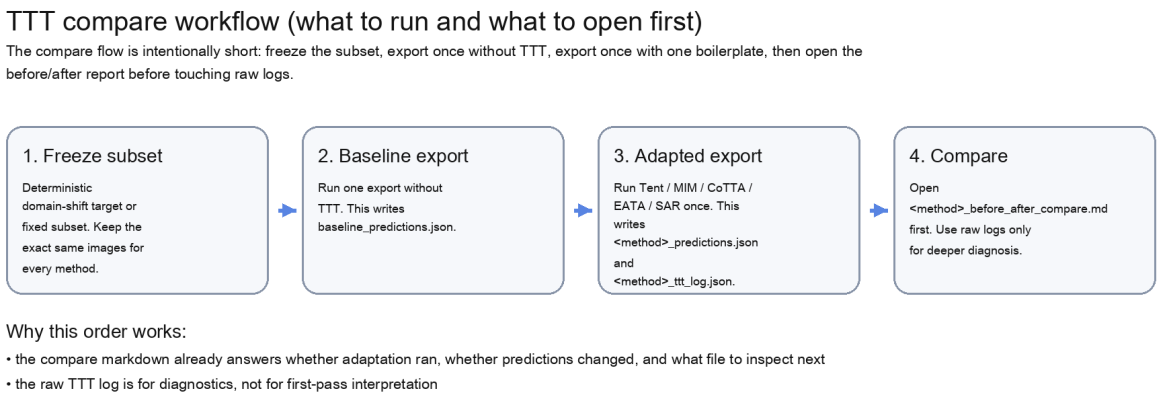


Figure 16.3: Beginner-oriented processing flow for every TTT/CTTA method. Freeze the subset, export once without TTT, export once with one boilerplate, then open `<method>_before_after_compare.md` before reading raw logs. This is the recommended reading order because the compare markdown answers whether adaptation actually ran, whether predictions changed, and which artifact to inspect next.

16.3.4 Per-image probe grid: concrete, but still secondary to the metric chart

Figure 16.4 uses the same fixed ten-image shifted subset as the metric summary. It is intentionally secondary. The goal of this figure is not to prove quality with a single image, but to make the result concrete: for each shifted image, it shows the top baseline detection, the top TTT detection, and the score movement. This makes it much easier to explain what the detector is trying to localize and how the prediction changed, while the actual proof of improvement still comes from Figure 16.2.

16.4 Presets (start here)

Both CLIs expose `-ttt-preset`:

- `safe`: Tent + BN-affine only (`update_filter=norm_only`)
- `adapter_only`: Tent + adapter/head only
- `mim_safe`: MIM + adapter/head only
- `cotta_safe`: CoTTA with conservative update/restore guard rails
- `eata_safe`: EATA selective adaptation with anti-forgetting defaults
- `sar_safe`: SAR LoRA-first sharpness-aware adaptation defaults
- `pose_safe`/`keypoints_safe`/`depth_safe`/`seg_safe`: task-aware Tent defaults
- `pose_mim`: pose-aware MIM defaults

Presets override core knobs (method, steps, lr, update filter, max batches) and set conservative guard rails unless you override them. Use `-ttt-sdft-task \{pose,keypoints,depth,seg,full\}` and `-ttt-aux-*` weights to steer multi-task adaptation losses.

16.5 Method Catalog with concrete examples

The currently implemented parameter-updating TTT/CTTA methods are: `tent`, `mim`, `cotta`, `eata`, and `sar`. In practice, the fastest way to compare them fairly is:

1. freeze the image subset,
2. run the same adapter/checkpoint on the same shifted images,
3. emit wrapped predictions for each method,
4. compare method-specific logs and benchmark artifacts.

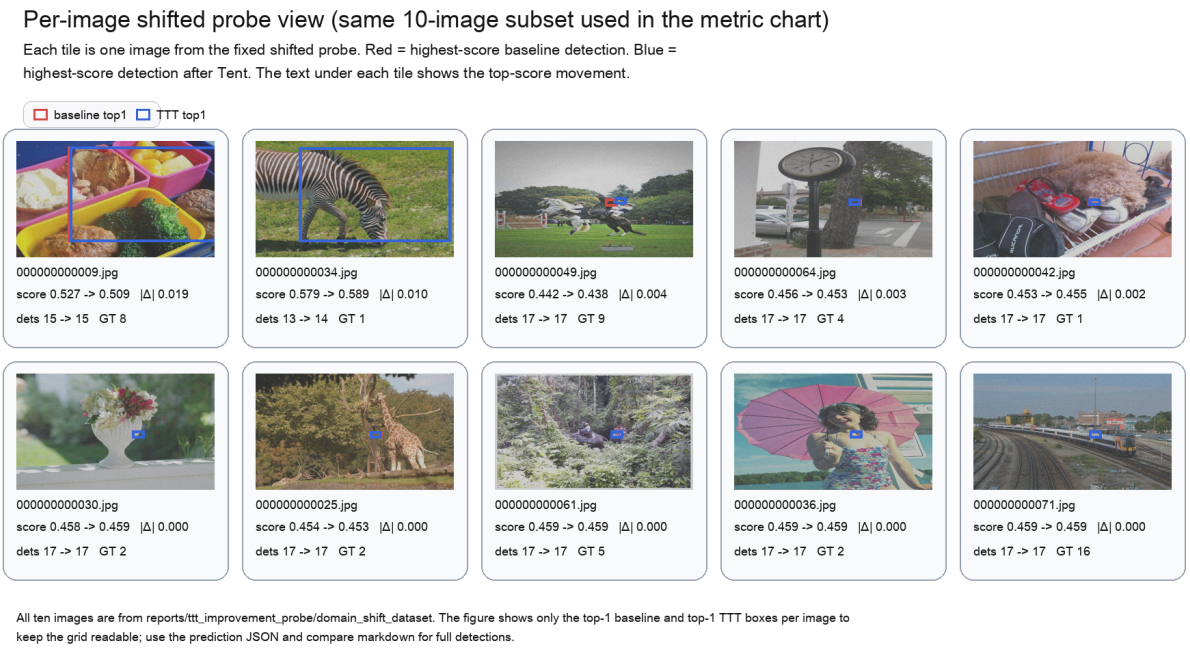


Figure 16.4: Per-image result grid from the same fixed ten-image shifted probe used in Figure 16.2. Each tile is one actual shifted image. Red marks the highest-score baseline detection without TTT; blue marks the highest-score detection after Tent. The text under each tile reports the top-score movement plus detection-count and ground-truth counts. This figure answers "what was the detector trying to localize, and how did the top detection move on each image?" The quantitative claim still comes from the fixed-subset metric chart in Figure 16.2.

16.5.1 Recommended operator workflow: boilerplate compare

Instead of retyping a long raw export command for each method, use the boilerplate compare wrapper:

```
bash scripts/ttt_compare.sh \
  --boilerplate tent \
  --dataset data/smoke \
  --split val \
  --checkpoint /path/to.ckpt \
  --run-dir reports/ttt_compare/tent \
  --device cuda
```

This workflow writes:

- `plan.json`
- `baseline_predictions.json`
- `<method>_predictions.json`
- `<method>_ttt_log.json`
- `<method>_before_after_compare.json`
- `<method>_before_after_compare.md`

The compare report provides the actual **before/after** summary: baseline vs adapted prediction counts, TTT loss/runtime summary, and a compact drift summary (`changed_images`, `missing_match_failures`, `value_mismatch_failures`).

16.5.2 Which files to compare for each method

Method	Boilerplate	Baseline	Adapted	Primary before/after artifact
Tent	`tent`	`reports/ttt_compare/tent/baseline_predictions.json`	`reports/ttt_compare/tent/tent_predictions.json`	`reports/ttt_compare/tent/tent_before_after_compare.md`
MIM	`mim`	`reports/ttt_compare/mim/baseline_predictions.json`	`reports/ttt_compare/mim/mim_predictions.json`	`reports/ttt_compare/mim/mim_before_after_compare.md`
CoTTA	`cotta`	`reports/ttt_compare/cotta/baseline_predictions.json`	`reports/ttt_compare/cotta/cotta_predictions.json`	`reports/ttt_compare/cotta/cotta_before_after_compare.md`
EATA	`eata`	`reports/ttt_compare/eata/baseline_predictions.json`	`reports/ttt_compare/eata/eata_predictions.json`	`reports/ttt_compare/eata/eata_before_after_compare.md`
SAR	`sar`	`reports/ttt_compare/sar/baseline_predictions.json`	`reports/ttt_compare/sar/sar_predictions.json`	`reports/ttt_compare/sar/sar_before_after_compare.md`

Recommended reading order:

1. `plan.json` to confirm the frozen subset and boilerplate,
2. `<method>_before_after_compare.md` for the compact summary,
3. `<method>_ttt_log.json` for adaptation loss/runtime details,
4. method-specific follow-up tools only when deeper diagnostics are needed.

Important note. The shipped `mim` and `sar` boilerplates expand the safe defaults directly and force `-ttt-update-filter norm_only` so they run against the repo-shipped checkpoint without requiring LoRA-specific weights. If a team owns a dedicated adapter/LoRA checkpoint, it is better to copy the JSON boilerplate and switch the update filter there than to fall back to a long raw CLI invocation. The `mim` boilerplate also injects the repo-backed config `configs/examples/ttt_compare/rtdetr_pose_mim_compare.json` into both the baseline and adapted export commands so the MIM branch is enabled without a manual config override.

16.5.3 Smoke compare snapshot (repo-shipped checkpoint)

We also validated the boilerplates against the repo-shipped checkpoint `reports/rtdetr_pose_ckpt_coco128_gpu_matcher.pt` on `data/smoke`, `split=val`, `max-images=2`, `device=cpu`, with `-skip-eval`.

This is a workflow validation snapshot, not a quality claim. The tiny smoke subset produced zero detections in both baseline and adapted runs, so the useful signal here is method completion and the TTT log behavior.

For the completed runs, the compact summaries are:

Method	Run dir	Real compare status	Steps	Mean final loss
Tent	reports/ttt_compare/tent_smoke_cpu	completed	2	4.214431
MIM	reports/ttt_compare/mim_smoke_cpu	completed	2	0.461853
CoTTA	reports/ttt_compare/cotta_smoke_cpu	completed	1	4.243579
EATA	reports/ttt_compare/eata_smoke_cpu	completed	0	null
SAR	reports/ttt_compare/sar_smoke_cpu	completed	2	4.211009

Table 16.1: Smoke compare snapshot for the repo-shipped checkpoint on `data/smoke`, `split=val`, `max-images=2`, `device=cpu`, with `-skip-eval`. This is a workflow-validation table, not a quality leaderboard. Warning notes are intentionally moved out of the table for readability: MIM uses the repo-backed compare config `configs/examples/ttt_compare/rtdetr_pose_mim_compare.json`; EATA reports `eata_empty_selected_set` on both smoke images; Tent, CoTTA, and SAR complete without warnings on this smoke subset.

- `reports/ttt_compare/tent_smoke_cpu/tent_before_after_compare.md`
- `reports/ttt_compare/mim_smoke_cpu/mim_before_after_compare.md`
- `reports/ttt_compare/cotta_smoke_cpu/cotta_before_after_compare.md`
- `reports/ttt_compare/eata_smoke_cpu/eata_before_after_compare.md`
- `reports/ttt_compare/sar_smoke_cpu/sar_before_after_compare.md`

16.5.4 How to read the smoke compare table

The smoke table above is a **workflow validation table**. It tells you whether the method ran safely and emitted reproducible artifacts. It does *not* claim that one method is globally better than another.

Read the columns in this order:

1. **Real compare status**: did the whole compare workflow finish?
2. **Steps**: how many adaptation updates actually ran?
3. **Mean final loss**: what was the final adaptation objective value?

Two common beginner mistakes:

- treating `mean_final_loss` as a quality score across methods,
- treating `steps=0` as a failure.

Warning notes are now kept in the table caption rather than in a dedicated wide column. For example, EATA can legitimately report `steps=0` with `eata_empty_selected_set`. That means the method decided *not* to adapt on those smoke images, which is conservative behavior, not a crash.

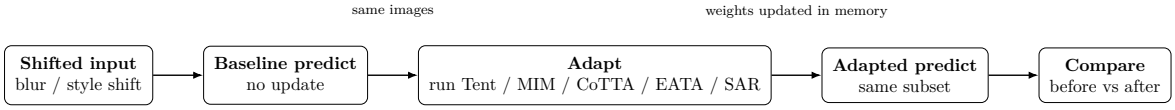


Figure 16.5: Common TTT workflow in YOLOZU: run the baseline and the adapted variant on the same deterministic image subset, then compare their artifacts.

16.5.5 Operator workflow: what to do first

For every method, the safest operator workflow is:

1. choose a fixed dataset subset or a deterministic domain-shift target,
2. run `scripts/ttt_compare.sh` with one boilerplate,
3. open `<method>_before_after_compare.md`,
4. only then inspect `<method>_ttt_log.json`.

This order matters. The compare markdown is the onboarding artifact. It answers the first three questions operators usually have:

- Did adaptation actually run?
- Did predictions change?
- Which file should I inspect next?

16.5.6 Tent

What the name means. Tent is short for **fully Test-time adaptation by ENTropy minimization**. The name is useful because it already tells you the two essential ideas:

- adaptation happens at *test time*,
- entropy is the signal being minimized.

Principle. Tent minimizes prediction entropy online using a restricted parameter subset [26]. In plain language, it pushes the model toward making sharper, more confident predictions on shifted inputs, but only lets a small subset of parameters move.

What entropy means here. If a prediction spreads probability mass across many classes, the entropy is high. If the same prediction becomes concentrated on one class, the entropy is low. For a single prediction vector p, the usual entropy term is:

$$H(p) = -\sum_{c} p_c \log p_c.$$

Tent adapts a small subset of parameters by reducing the average prediction entropy over the selected test-time samples:

$$\min_{\theta_{\text{adapt}}} \; \mathrm{E}_{x \sim \text{test stream}} \left[H(p_\theta(y \mid x)) \right].$$

In YOLOZU's safe presets, θ_{adapt} is intentionally small, usually norm-affine parameters or a restricted adapter/head subset.

Why it can work. Under mild domain shift, the model often still attends to roughly the right object, but its output distribution becomes softer and less decisive. Tent exploits that situation: it does not ask the model to discover a brand-new concept, it only asks whether a more confident local decision boundary gives a better answer.

What it is not.

- It is not supervised fine-tuning.
- It does not tell the model which class is correct.
- It assumes "become more confident" is a useful direction on the current shifted sample.

When to use it. Tent is the best first method. Choose it when you want the smallest operational jump away from baseline inference and the easiest debug path.

Concrete procedure.

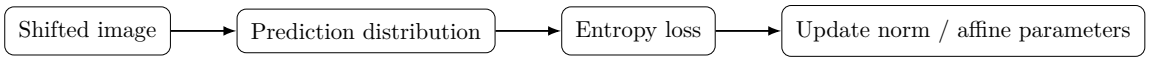


Figure 16.6: Tent: use prediction entropy as the adaptation signal and keep updates constrained to a safe parameter subset.

```
bash scripts/ttt_compare.sh \
  --boilerplate tent \
  --dataset data/smoke \
  --split val \
  --checkpoint /path/to.ckpt \
  --run-dir reports/ttt_compare/tent \
  --device cuda
```

How to read the result.

- Open `tent_before_after_compare.md` first.
- Check whether `steps_run > 0`.
- If `changed_images=0`, adaptation completed but exported predictions did not change on this subset.
- Open `tent_ttt_log.json` only if you need guard or runtime detail.

Concrete repo result. On the fixed ten-image shifted probe in Figure 16.2, Tent improves `map50` from `0.00326797` to `0.00392157` and `map50_95` from `0.000326797` to `0.000392157`. The effect is small, but it is reproducible under the same checkpoint, subset, and seed.

Pros.

- easiest method to explain,
- cheapest adaptation cost,
- safest default when using norm-only updates.

Cons.

- weakest self-supervised signal,
- can sharpen wrong predictions under severe shift,
- often less expressive than reconstruction-based methods.

16.5.7 MIM

What the name means. MIM stands for **Masked Image Modeling**. In plain language, we deliberately hide part of the visual signal and ask the model to reconstruct or predict what is missing. That is why MIM feels more structural than Tent: it is not only asking for confidence, it is asking for internal consistency.

Principle. MIM uses masked reconstruction and optional entropy terms to adapt from unlabeled shifted inputs. Instead of only asking the model to become more confident, it also asks the model to reconstruct hidden structure from partially masked internal features. The lineage is broader than MAE alone: denoising autoencoders already used corruption-plus-reconstruction as a representation-learning signal in 2008 [25], image inpainting-style reconstruction was used as a visual self-supervised task in Context Encoders [20], and modern masked-image-modeling references include Masked Autoencoders (MAE) [7] and SimMIM [27]. For direct test-time usage, representative references are Test-Time Training with Masked Autoencoders [4] and TTT-MIM [15]. YOLOZU uses this family of masked-image-modeling ideas as an inference-time auxiliary signal rather than as a standalone pretraining recipe.

Minimal objective view. Let M denote a mask and R_θ denote the reconstruction path. A simplified masked-reconstruction loss can be written as:

$$L_{\text{MIM}} = \left\| R_\theta((1 - M) \odot x) - M \odot x \right\| .$$

Many practical test-time variants combine this reconstruction term with an entropy term:

$$L_{\text{adapt}} = \lambda_{\text{mim}} L_{\text{MIM}} + \lambda_{\text{ent}} H(p_\theta(y \mid x)) \,.$$

The exact target differs across implementations, but the operational story stays the same:

- hide part of the signal,
- ask the model to infer the hidden structure,
- use that self-supervised error as the adaptation signal.

Research lineage and IP note. This manual cites representative prior-art waypoints so that operators can place the method in a broader technical lineage instead of treating MIM-style adaptation as a single new idea. The broad pattern—corrupt or mask part of the input/feature state, reconstruct hidden structure, and use that self-supervised loss to improve robustness—predates the current wording of test-time masked adaptation. This note is meant to support technical due diligence and prior-art positioning only; it is **not** legal advice and it is **not** a non-infringement opinion. Any deployment-specific IP or patent review belongs to qualified counsel.

Why it can work. Entropy-only adaptation can be too weak when the shift is geometric, structural, or pose-sensitive. MIM adds a denser self-supervised objective: the model is rewarded not merely for being decisive, but for reconstructing a plausible hidden structure from incomplete evidence. That is why MIM is often easier to justify in pose-heavy workflows than plain confidence sharpening.

What it is not.

- It is not just "MAE pretraining at inference time".
- It is not guaranteed to improve every exported detection.
- It is usually more model-specific than Tent because the masked-reconstruction branch must exist and stay numerically stable.

Figure 16.7: MIM: use masked reconstruction to create a stronger adaptation signal than entropy alone.

When to use it. Choose MIM when you need a stronger adaptation signal than Tent, especially for pose-heavy or geometry-sensitive runs.

Concrete procedure.

```
bash scripts/ttt_compare.sh \
  --boilerplate mim \
  --dataset data/smoke \
  --split val \
  --checkpoint /path/to.ckpt \
  --run-dir reports/ttt_compare/mim \
  --device cuda
```

For pose-heavy runs, switch to `-ttt-preset pose_mim`. YOLOZU now ships two practical MIM entrypoints:

- `mim`: a repo-backed smoke fixture for verifying that the masked-reconstruction path is wired correctly
- `mim_probe`: a fixed real probe that demonstrates an actual before/after metric delta on the bundled shifted ten-image probe

```
bash scripts/ttt_compare.sh \
  --boilerplate mim \
  --dataset data/smoke \
  --split val \
```

```
  --checkpoint reports/rtdetr_pose_ckpt_coco128_gpu_matcher.pt \
  --run-dir reports/ttt_compare/mim_smoke_cpu \
  --device cpu \
  --max-images 2 \
  --skip-eval \
  --force
```

Fixed real-probe compare.

```
bash scripts/ttt_compare.sh \
  --boilerplate mim_probe \
  --dataset reports/ttt_improvement_probe/domain_shift_dataset \
  --split val \
  --checkpoint reports/ttt_improvement_probe/checkpoint.pt \
  --run-dir reports/ttt_compare/mim_probe_cpu \
  --device cpu \
  --max-images 10
```

How to read the result.

- Open `mim_before_after_compare.md` first.
- Then inspect `mim_ttt_log.json`; the MIM loss trace is more informative than Tent's entropy-only loss.
- The repo-backed smoke example proves execution: `steps_run=2`, `mean_final_loss=0.461853`, and `changed_images=0 / 2`.
- The fixed real probe proves operator-visible impact: `steps_run=10`, `mean_final_loss=0.0791513`, `changed_images=10 / 10`, `map50 0.00326797 \rightarrow 0.00392157`, and `map50_95 0.000326797 \rightarrow 0.000392157`.
- When `pycocotools` is not installed in the current runtime, the compare falls back to `simple_map_proxy` instead of silently skipping evaluation.

Concrete repo results.

- repo-backed smoke compare: `steps_run=2`, `mean_final_loss=0.461853`, `changed_images=0 / 2`
- fixed real probe compare: `steps_run=10`, `mean_final_loss=0.0791513`, `changed_images=10 / 10`
- fixed real probe metrics: `map50 0.00326797 \rightarrow 0.00392157`, `map50_95 0.000326797 \rightarrow 0.000392157`

Pros.

- stronger self-supervised signal than Tent,
- useful for geometry- and pose-sensitive adaptation,
- easier to justify when confidence-only adaptation is too weak.

Cons.

- more model-specific than Tent,
- more complex to interpret,
- loss values are not directly comparable to Tent values.

16.5.8 CoTTA

What the name means. CoTTA is short for **Continual Test-Time Adaptation**. The name matters because it signals that this method is designed for a stream, not just for a single isolated image.

Principle. CoTTA combines augmentation-averaged predictions, EMA teacher updates, and stochastic restoration. The main idea is to adapt continuously in a stream while preventing the model from drifting too far.

Minimal objective view. CoTTA can be read as a consistency objective between an online student and an EMA teacher across augmented views:

$$L_{\text{CoTTA}} = \sum_{a \in \mathcal{A}} d(p_\theta(y \mid a(x)),\ p_{\theta^{\text{EMA}}}(y \mid x)) .$$

The exact distance term $d(\cdot, \cdot)$ and restoration schedule vary by implementation, but the operational story is stable:

- smooth the target with an EMA teacher,
- average over augmented views,
- periodically restore part of the student so drift does not grow unchecked.

Why it can work. Pure online adaptation can slowly walk away from a good solution in long streams. CoTTA adds three safeguards against that:

- augmentation averaging,
- EMA teacher smoothing,
- partial restoration of older weights.

What it is not.

- It is not the simplest first TTT baseline.
- It is not stateless; stream order matters more than in per-sample adaptation.
- It makes the most sense when stream behavior itself is part of the problem.

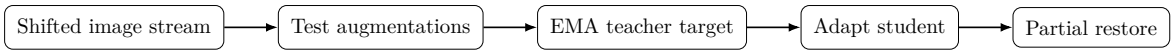


Figure 16.8: CoTTA: smooth predictions with a teacher model and periodically restore part of the original model to control drift.

When to use it. Use CoTTA when stream-mode adaptation matters and you care about long-run behavior, not just a single-image update.

Concrete procedure.

```
bash scripts/ttt_compare.sh \
  --boilerplate cotta \
  --dataset data/smoke \
  --split val \
  --checkpoint /path/to.ckpt \
  --run-dir reports/ttt_compare/cotta \
  --device cuda
```

Follow-up tool.

```
python3 tools/eval_cotta_drift.py \
  --baseline reports/ttt_compare/tent/tent_predictions.json \
  --cotta reports/ttt_compare/cotta/cotta_predictions.json \
  --output-json reports/ttt_compare/cotta/cotta_drift.json \
  --output-md reports/ttt_compare/cotta/cotta_drift.md
```

How to read the result.

- Open `cotta_before_after_compare.md` first.
- Then inspect `cotta_ttt_log.json` to confirm stream updates and runtime.
- Use `cotta_drift.md` when you need to reason about long-run stability.

Concrete repo result. On the same fixed ten-image shifted probe as Tent, CoTTA reaches the same `map50` and `map50_95` values shown in Figure 16.2 (`0.00392157` and `0.000392157` respectively). The practical takeaway is that CoTTA can recover the same small metric gain while using a more stateful adaptation recipe.

Pros.

- best suited to continual / streaming adaptation,

- teacher smoothing helps suppress noisy updates,
- restoration reduces irreversible drift.

Cons.

- more moving parts than Tent or MIM,
- state evolves across images, so debugging is harder,
- higher runtime cost.

16.5.9 EATA

What the name means. EATA is commonly read as **Efficient Test-Time Adaptation**. In practice, the key idea is not only efficiency but *selective* efficiency: adapt only when the current samples look trustworthy enough.

Principle. EATA adds selective sample filtering plus anchor regularization. It adapts only when the batch looks trustworthy enough, and it regularizes the update so important weights are less likely to drift.

Minimal objective view. EATA first selects a subset of samples that pass entropy/reliability gates, then adapts only on that selected subset:

$$L_{\text{EATA}} = \sum_{x \in S_{\text{selected}}} H(p_\theta(y \mid x)) + \lambda_{\text{anchor}} R(\theta, \theta_0).$$

The exact selection test varies, but the operational rule is simple:

- no trustworthy subset,
- no update.

Why it can work. One bad batch can make online adaptation worse instead of better. EATA reduces that risk by refusing to adapt when the current evidence does not look informative enough. That is why its "skip" behavior is a feature, not a failure.

What it is not.

- It is not broken when it skips updates.
- It is not trying to adapt on every sample.
- It deliberately trades aggressiveness for safety.

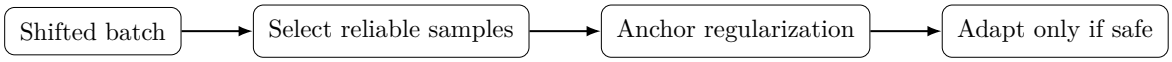


Figure 16.9: EATA: filter out weak adaptation samples first, then regularize the remaining update.

When to use it. Use EATA when false-positive adaptation is a real risk and you prefer the method to skip doubtful samples rather than force every batch to adapt.

Concrete procedure.

```
bash scripts/ttt_compare.sh \
  --boilerplate eata \
  --dataset data/smoke \
  --split val \
  --checkpoint /path/to.ckpt \
  --run-dir reports/ttt_compare/eata \
  --device cuda
```

Follow-up tool.

```
python3 tools/benchmark_eata_stability.py \
  --baseline reports/ttt_compare/tent/tent_predictions.json \
  --eata reports/ttt_compare/eata/eata_predictions.json \
  --output-json reports/ttt_compare/eata/eata_benchmark.json \
  --output-md reports/ttt_compare/eata/eata_benchmark.md
```

How to read the result.

- Open `eata_before_after_compare.md` first.
- If `steps_run=0`, check the warning field before assuming anything failed.
- On tiny smoke subsets, `eata_empty_selected_set` is expected conservative behavior.

Concrete repo result. EATA has two useful examples in this repo. On the tiny smoke fixture, it intentionally skips adaptation (`steps_run=0`, `eata_empty_selected_set`), which is conservative behavior. On the larger fixed ten-image shifted probe, it does adapt and reaches the same small metric gain as Tent/CoTTA/SAR, as shown in Figure 16.2.

Pros.

- safest method when many test samples are low quality,
- explicit skip behavior is easy to justify operationally,
- anti-forgetting regularization is built into the method.

Cons.

- may appear inactive on small subsets,
- harder to explain than Tent,
- more thresholds and diagnostics to inspect.

16.5.10 SAR

What the name means. SAR refers to a **Sharpness-Aware** adaptation rule. The key idea is to prefer updates that remain good even after a small perturbation, rather than trusting only the immediate entropy gradient.

Principle. SAR performs sharpness-aware entropy minimization using a perturb-and-restore style update. Instead of trusting only the immediate entropy gradient, it prefers updates that stay stable after a small perturbation.

Minimal objective view. The sharpness-aware view can be written as:

$$\min_{\theta} \max_{\|\epsilon\| \le \rho} L_{\text{entropy}}(\theta + \epsilon).$$

The inner perturbation asks whether the current update is fragile in a small neighborhood. The outer step then prefers a flatter, more stable region rather than a sharp local win.

Why it can work. Some shifts are noisy enough that plain entropy minimization is too eager. SAR slows the optimizer down and asks whether the local improvement is robust, not just immediate.

What it is not.

- It is not the cheapest method.
- It is not needed when Tent already behaves well.
- It is more about update geometry than about a stronger self-supervised signal.

Figure 16.10: SAR: prefer updates that remain stable after a small perturbation, not only updates that reduce the immediate entropy value.

When to use it. Use SAR when the shift is noisy and you want a more conservative optimizer geometry than Tent, while accepting extra compute per update.

Concrete procedure.

```
bash scripts/ttt_compare.sh \
  --boilerplate sar \
  --dataset data/smoke \
  --split val \
  --checkpoint /path/to.ckpt \
```

```
  --run-dir reports/ttt_compare/sar \
  --device cuda
```

Follow-up tool.

```
python3 tools/benchmark_sar_robustness.py \
  --cotta reports/ttt_compare/cotta/cotta_predictions.json \
  --eata reports/ttt_compare/eata/eata_predictions.json \
  --sar reports/ttt_compare/sar/sar_predictions.json \
  --output-json reports/ttt_compare/sar/sar_robustness.json \
  --output-md reports/ttt_compare/sar/sar_robustness.md
```

How to read the result.

- Open `sar_before_after_compare.md` first.
- Compare runtime in `sar_ttt_log.json` against Tent to understand the extra adaptation cost.
- Use `sar_robustness.md` when comparing SAR against CoTTA and EATA.

Concrete repo result. On the fixed ten-image shifted probe, SAR reaches the same measured metric gain as Tent/CoTTA/EATA in Figure 16.2. For onboarding, the important point is not that SAR wins the chart, but that it lands in the same place while using a sharper, more conservative update geometry.

Pros.

- more conservative update geometry than Tent,
- useful for noisy shifts,
- a good middle ground between Tent simplicity and CoTTA complexity.

Cons.

- slower than Tent,
- conceptually harder for beginners,
- can be unnecessary for small or clean shifts.

16.5.11 Choosing a method quickly

Method	Best first use	Main trade-off
Tent	first ablation, safest first comparison	weakest adaptation signal
MIM	geometry- or pose-sensitive shift	more model-specific and harder to interpret
CoTTA	streaming / continual adaptation	stateful and more expensive
EATA	conservative adaptation with explicit skipping	may do nothing on small subsets
SAR	noisy shift with stability concerns	slower than Tent

16.5.12 Task-aware examples

The methods above are shared, but the safest default preset depends on the task emphasis. These examples keep the method explicit and only change the task-aware preset/auxiliary weights.

Listing 16.2: Task-aware TTT examples using the same export surface.

```
# pose-heavy Tent
python3 -m yolozu export \
  --backend torch --dataset reports/smoke_50 --split val \
  --checkpoint /path/to.ckpt --device cuda \
```

```
  --ttt --ttt-method tent --ttt-preset pose_safe \
  --ttt-sdft-task pose --ttt-aux-pose-weight 0.5 \
  --ttt-log-out reports/ttt_pose_safe.json \
  --output reports/pred_pose_safe.json

# keypoints-heavy Tent
python3 -m yolozu export \
  --backend torch --dataset reports/smoke_50 --split val \
  --checkpoint /path/to.ckpt --device cuda \
  --ttt --ttt-method tent --ttt-preset keypoints_safe \
  --ttt-sdft-task keypoints --ttt-aux-keypoints-weight 0.5 \
  --ttt-log-out reports/ttt_keypoints_safe.json \
  --output reports/pred_keypoints_safe.json

# depth-heavy Tent
python3 -m yolozu export \
  --backend torch --dataset reports/smoke_50 --split val \
  --checkpoint /path/to.ckpt --device cuda \
  --ttt --ttt-method tent --ttt-preset depth_safe \
  --ttt-sdft-task depth --ttt-aux-depth-weight 0.5 \
  --ttt-log-out reports/ttt_depth_safe.json \
  --output reports/pred_depth_safe.json

# segmentation-heavy Tent
python3 -m yolozu export \
  --backend torch --dataset reports/smoke_50 --split val \
  --checkpoint /path/to.ckpt --device cuda \
  --ttt --ttt-method tent --ttt-preset seg_safe \
  --ttt-sdft-task seg --ttt-aux-seg-weight 0.5 \
  --ttt-log-out reports/ttt_seg_safe.json \
  --output reports/pred_seg_safe.json
```

Rule of thumb. Start from the task-aware Tent preset, then escalate to MIM/CoTTA/EATA/SAR only when the shift evidence justifies the extra complexity and latency.

16.6 Implementation Notes: YOLOZU rollout priority (high → low)

For YOLOZU (detection/pose), the recommended operational rollout order is:

1. **CoTTA (highest priority)**: standardize *teacher=EMA(model)*, use *augmentation-averaged predictions* (e.g., flips) as prediction stabilization, and enable *stochastic restoration* to suppress long-run drift. Start safely by limiting restoration and updates to **LoRA/Norm** subsets only.
2. **EATA (second)**: add *active sample selection* (only adapt on trusted samples) plus *anti-forgetting regularization* (Fisher/EWC-style protection of important weights). In YOLOZU, constrain updates to **Norm/LoRA/selected head** parameters, and compute selection signals from detection/pose aggregates (e.g., confidence/entropy summaries across queries).
3. **SAR (third)**: use *sharpness-aware* entropy minimization for stability in wild mixed-shift / small-batch settings. Start with **LoRA-only** sharpness-aware updates to control cost/side effects, and prefer **GN/LN** normalization when possible (BN can be unstable under small-batch online adaptation).

16.7 CoTTA: operational design for YOLOZU (phase 1)

This section is the phase-1 CoTTA integration scope for YOLOZU. The goal is to suppress long-run drift/forgetting under online test-time adaptation while keeping rollout risk low.

16.7.1 Update scope (safe-by-default)

Phase 1 restricts which parameters can change.

Allowed trainable targets:

- LoRA parameters.
- normalization affine parameters (LN/GN/BN affine where used).

Explicitly disallowed in phase 1:

- full backbone weight adaptation,
- unrestricted full-model optimizer updates.

16.7.2 Teacher model: EMA(student)

CoTTA uses a teacher defined as an exponential moving average (EMA) of the student parameters:

$$\theta_{teacher} \leftarrow m\,\theta_{teacher} + (1-m)\,\theta_{student}.$$

Teacher EMA updates run after each adaptation step, with configurable momentum m. Teacher parameters are never directly optimized by gradient descent.

16.7.3 Multi-augmentation prediction averaging

Phase-1 default augmentations:

- identity,
- horizontal flip.

Aggregation behavior:

- Run the model on each augmentation branch.
- Map predictions back to a common image coordinate system.
- Aggregate with a deterministic (seed/config-pinned) confidence-aware averaging rule, then apply the standard postprocess/NMS.

16.7.4 Stochastic restoration (safe mode)

Stochastic restoration is applied only to the allowed trainable targets. For each eligible parameter element, restore from a source snapshot with probability $p_{restore}$. The source snapshot defaults to the initial pre-adaptation model state for the session. Restoration executes on a configurable cadence (e.g., every N adaptation steps).

16.7.5 Safety boundaries and guardrails

Phase-1 rollout requires guardrails to prevent runaway adaptation:

- gradient norm clipping (`max_grad_norm`),
- per-step update norm cap (`max_update_norm`),
- cumulative update norm cap (`max_total_update_norm`),
- divergence stop condition via loss-ratio threshold (`max_loss_ratio`).

Failure behavior should be explicit and logged: abort the adaptation step on hard breach, emit a diagnostics event, and restore model state according to the configured fallback policy.

16.7.6 Minimum configuration knobs (phase 1)

To keep comparisons consistent, phase-1 CoTTA should record at least:

- `ttt.method=cotta`
- `ttt.cotta.ema_momentum`
- `ttt.cotta.augmentations` (initial: identity, hflip)
- `ttt.cotta.aggregation` (initial default: confidence-weighted mean)
- `ttt.cotta.restore_prob` and `ttt.cotta.restore_interval`
- `ttt.update_filter` (must cover norm-only, lora-only, and combined safe subset)
- guardrails: `ttt.max_grad_norm`, `ttt.max_update_norm`, `ttt.max_total_update_norm`, `ttt.max_loss_ratio`

16.8 EATA: operational design for YOLOZU (phase 1)

This section defines a production-safe phase-1 EATA scope for detection and pose in YOLOZU. The goal is to reduce unstable updates by (1) adapting only on informative samples, (2) limiting update scope to safe parameter subsets, and (3) adding anti-forgetting regularization.

16.8.1 Update scope (safe-by-default)

Allowed trainable targets:

- normalization affine parameters (LN/GN/BN affine where used),
- LoRA parameters,
- optional detection/pose head subset (explicit allowlist only; opt-in).

Explicitly disallowed in phase 1:

- full backbone updates,
- unrestricted full-model adaptation.

Phase-1 presets should start with `norm_only` or `lora_norm_only` and keep head updates opt-in.

16.8.2 Active sample selection (detection/pose aware)

EATA adaptation steps consume only selected samples from the incoming stream. For each image, compute selection features from prediction outputs (examples):

- detection confidence aggregate (e.g., `mean_topk_conf`),
- detection entropy aggregate (e.g., `mean_topk_entropy`),
- pose confidence aggregate (e.g., `mean_kpt_conf` when keypoints exist),
- optional agreement score across light augmentation branches (phase-1 optional).

Phase-1 selection rule (conservative default): select a sample only when all are satisfied:

- confidence lower bound: `conf \ge conf_min`,
- entropy is within band: `entropy_min \le entropy \le entropy_max`,
- valid detection count floor: `num_valid \ge min_valid_dets`.

16.8.3 Anti-forgetting regularization (phase 1)

Phase-1 regularization anchors online updates to the pre-adaptation snapshot. Use a parameter-anchor penalty on the same trainable subset:

$$L_{anchor} = \sum_i \lambda_i \left\| \theta_i - \theta_i^{(0)} \right\|_2^2 .$$

Total phase-1 objective is:

$$L = L_{adapt} + \lambda_{anchor} L_{anchor},$$

where L_{adapt} is an entropy-based adaptation objective over selected samples.

16.8.4 Safety boundaries and guardrails

Reuse the standard TTT guardrails:

- `max_grad_norm`
- `max_update_norm`
- `max_total_update_norm`
- `max_loss_ratio`

and rollback-on-stop behavior.

Additional EATA-specific safety checks:

- minimum selected-sample ratio per step (`selected_ratio_min`),
- skip-step when the selected set is empty,
- max consecutive skipped steps (`max_skip_streak`) to trigger early stop with warning.

16.8.5 Minimum configuration knobs (phase 1)

Phase-1 EATA should record at least:

- `ttt.method=eata`
- `ttt.update_filter` (`norm_only`, `lora_only`, `lora_norm_only`, optional head allowlist)
- thresholds:
  `ttt.eata.conf_min`,
  `ttt.eata.entropy_min`,
  `ttt.eata.entropy_max`,
  `ttt.eata.min_valid_dets`
- anchor: `ttt.eata.anchor_lambda`
- selection safety: `ttt.eata.selected_ratio_min`, `ttt.eata.max_skip_streak`
- guardrails: `ttt.max_grad_norm`, `ttt.max_update_norm`,
  `ttt.max_total_update_norm`, `ttt.max_loss_ratio`

16.9 SAR: operational design for YOLOZU (phase 1)

This section defines a production-safe phase-1 SAR rollout scope for YOLOZU. The goal is to improve robustness under challenging distribution shifts by introducing sharpness-aware entropy minimization with controlled update scope.

16.9.1 Update scope (LoRA-only by default)

Allowed update targets:

- LoRA parameters only (default).
- optional norm affine subset in controlled experiments (not default).

Explicitly disallowed in phase 1:

- full-backbone sharpness-aware updates,
- unrestricted full-model SAM-style optimization.

16.9.2 Optimization behavior (sharpness-aware entropy)

Phase-1 SAR uses a two-stage sharpness-aware update over the selected trainable parameters:

1. compute gradient on the entropy objective,
2. perturb weights along a normalized gradient direction,
3. compute a second loss at the perturbed point,
4. apply the optimizer update on the original weights.

Conceptually:

$$\min_{\theta} \max_{\|\epsilon\| \le \rho} L_{entropy}(\theta + \epsilon),$$

with ρ bounded conservatively for phase 1.

16.9.3 Normalization policy guidance

If SAR experiments require normalization updates:

- prefer GN/LN-first configurations,
- avoid BN-stat-heavy behavior in tiny/unstable batches,
- keep BN running-stat side effects monitored and rollback-capable.

Default rollout remains LoRA-only.

16.9.4 Safety boundaries and expected costs

Expected cost profile: approximately ∼2x forward/backward cost per adaptation step versus single-step entropy update, plus extra memory traffic due to perturb-and-restore.

Required safeguards: strict gradient/update caps, immediate rollback on guardrail breach, explicit stop-reason logging, and conservative default `steps=1` and `max_batches=1`.

16.9.5 Minimum configuration knobs (phase 1)

Phase-1 SAR should record at least:

- `ttt.method=sar`
- `ttt.update_filter` defaulting to `lora_only`
- `ttt.sar.rho`, `ttt.sar.adaptive`, `ttt.sar.first_step_scale`
- guardrails: `ttt.max_grad_norm`, `ttt.max_update_norm`, `ttt.max_total_update_norm`, `ttt.max_loss_ratio`

16.10 Evaluation and Failure Modes

Evaluate TTT as a before/after artifact comparison, not as a live-model update. The minimum acceptance check is:

- baseline and adapted predictions use the same target subset,
- the report records method, preset, seed, reset policy, and update scope,
- runtime and warning fields are inspected before claiming an improvement,
- unchanged predictions are reported as such instead of treated as a failure.

Failure modes to check first:

- **No domain shift**: the method runs but has no useful signal.
- **Unsafe update scope**: too many parameters move for a first comparison.
- **State leakage**: stream adaptation carries state into the next comparison.
- **Cost drift**: adaptation latency exceeds the budget for the target workflow.

16.11 Guard rails (safety limits)

If you do not explicitly set these, defaults are applied by preset:

- `max_grad_norm`
- `max_update_norm`
- `max_total_update_norm`
- `max_loss_ratio`

These bounds are there to prevent runaway adaptation and to keep comparisons reproducible.

16.12 Reset policy: stream vs sample

`-ttt-reset stream` Adapt once (up to `-ttt-max-batches`) then reuse adapted weights for subsequent images.

`-ttt-reset sample` Restore base state per image, adapt on that image/batch, then predict. Slower but cleaner ablations.

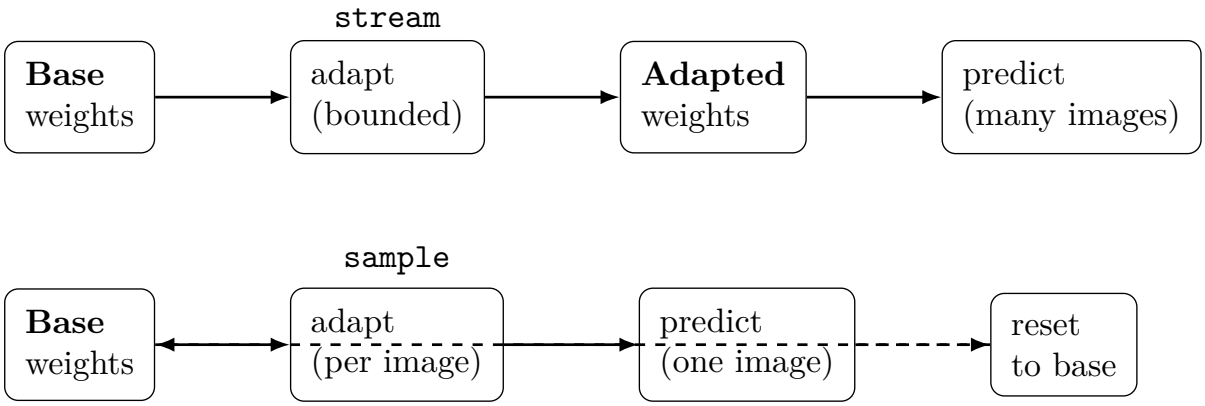


Figure 16.11: Reset policies: `stream` keeps adapted state across images; `sample` resets per image for clean ablations.

For plots and controlled comparisons, start with `-ttt-reset sample`.

16.13 Bounded-cost knobs

Two knobs matter most for runtime and fairness:

- `-ttt-batch-size N`: images per adaptation step.
- `-ttt-max-batches K`: hard cap on adaptation batches consumed.

Suggested smoke-test settings: `-ttt-batch-size 1 -ttt-max-batches 1`.

16.14 Method switch (manifest-aligned)

TTT method can be selected with `-ttt-method`:

- `tent`
- `mim`
- `cotta`
- `eata`
- `sar`

The current canonical CLI surface is reflected in `tools/manifest.json` for `tools/export_predictions.py` and the top-level `yolozu` CLI; `tools/yolozu.py` remains the repo-checkout compatibility wrapper.

16.15 Make a fixed evaluation subset

TTT comparisons must be run on the exact same images. Use `tools/make_subset_dataset.py` to create a deterministic subset dataset root:

```
python3 tools/make_subset_dataset.py \
  --dataset data/smoke \
  --split val \
  --n 50 \
```

```
  --seed 0 \
  --out reports/smoke_50
```

This writes `subset.json` (including an image hash) and a frozen image list.

16.16 Example: baseline vs TTT

Baseline export:

```
python3 -m yolozu export \
  --backend torch \
  --dataset reports/smoke_50 \
  --split val \
  --checkpoint /path/to.ckpt \
  --device cuda \
  --max-images 50 \
  --output reports/pred_baseline.json
```

TTT export (safe preset, per-sample reset, with a log):

```
python3 -m yolozu export \
  --backend torch \
  --dataset reports/smoke_50 \
  --split val \
  --checkpoint /path/to.ckpt \
  --device cuda \
  --max-images 50 \
  --ttt \
  --ttt-preset safe \
  --ttt-reset sample \
  --ttt-log-out reports/ttt_log_safe.json \
  --output reports/pred_ttt_safe.json
```

16.17 MIM-based adaptation (high level)

The geometry-aligned MIM branch (when enabled in the model) can be used for test-time adaptation. Conceptually:

- Use geometry-derived features as a teacher (mask + normalized depth).
- Reconstruct masked features (MIM/MAE-style) [7, 27] and optionally minimize entropy [5].
- Update a restricted parameter subset (e.g., norm or adapter/head) under guard rails.

Conceptually, MIM-style inference adds an auxiliary loss (computed at inference time) to adapt parameters or latent state under distribution shift. In this repo, the important operational points are:

- keep the loss bounded and the update schedule explicit,
- record adaptation settings into run artifacts,
- compare against a non-adapted baseline under the same pinned evaluation protocol.

16.18 Implementation Notes: operational notes

- TTT adds latency. Always report throughput and the chosen `batch_size/max_batches`.
- Keep the evaluation subset fixed and record the exact preset and guards.
- For deployment, treat TTT as an optional mode; default inference should not require it.

16.19 Method-specific evaluation tools

Manifest-registered benchmark tools for TTT phase validation:

- `tools/eval_cotta_drift.py` (baseline vs CoTTA drift/stability)
- `tools/benchmark_eata_stability.py` (baseline vs EATA stability/efficiency)
- `tools/benchmark_sar_robustness.py` (SAR vs CoTTA/EATA go/no-go impact)

These tools emit reproducible JSON/Markdown evidence artifacts and should be preferred over ad-hoc notebook-only comparisons.

Part IV

Maintainer, Automation, and Integration Appendices

Chapter 17

Tool Registry and Automation (Research Use)

This chapter elucidates the machine-readable tool registry (comprising the manifest and schemas) and its role in facilitating safe discovery and execution for both human operators and autonomous agents. It underpins reproducible automation and Continuous Integration (CI)-style workflows by strictly pinning tool I/O interface contracts and validating the registry's surface area. This documentation assumes that tools are invoked exclusively via their declared entrypoints and validated using `python3 tools/validate_tool_manifest.py`. Path-resolution rules are critical for robust automation.

Who should read this chapter: Read this chapter when you need to discover declared tools, understand their inputs and outputs, or build automation from the registry.

17.1 Boundary With the Maintenance Chapter

This chapter explains how to *read and use* the tool registry: what the manifest contains, how to query it, and how automation should treat declared entrypoints. Chapter 18 owns the *maintenance workflow*: changing tool behavior, updating `tools/manifest.json`, syncing packaged copies, and keeping docs/manual examples aligned.

17.2 The Rationale for a Tool Registry

Research workflows frequently accumulate disparate scripts. YOLOZU addresses this by treating the `tools/` directory as a stable, scriptable interface layer, augmented by a machine-readable registry:

- **Tool Manifest**: `tools/manifest.json`
- **Manifest Schema**: `docs/schemas/tools_manifest.schema.json`
- **Validator**: `python3 tools/validate_tool_manifest.py`

This architecture is explicitly designed to serve:

- **Human Operators**: Providing a clear answer to "What command do I run to achieve X?"
- **Agentic Automation**: Enabling safe, programmatic discovery of entrypoints and I/O interface contracts.
- **CI-Style Reproducibility**: Ensuring stable commands and deterministic artifact generation.

17.3 Authority for Discovery and Execution

For CLI capabilities and tool I/O behavior, `tools/manifest.json` serves as the **authoritative source of truth**. Treat prose examples as derived guidance: use the manifest and the tool's

`-help` output when wiring automation.

When discovery surfaces disagree:

- **Behavioral Truth**: Resides in the code and the manifest.
- **Operator Action**: Prefer the declared manifest entry and verify the tool's `-help` output.
- **Maintainer Action**: Use Chapter 18 to update the manifest, packaged copy, docs, and manual together.
- **Quality Gate**: Re-run the manifest validator prior to publishing any changes.

Execute the following command to verify compliance:

```
python3 tools/validate_tool_manifest.py --manifest tools/manifest.json --require-
    declarative
```

17.4 Declarative Fields: Mandatory Tool Metadata

The manifest functions as a declarative interface contract. At a practical minimum, every tool entry must expose:

- **Identity and Execution**: `id`, `entrypoint`, `runner`, `summary`, and `maturity`.
- **Environment**: `platform` constraints and optional `requires` dependencies.
- **Interface**: Explicit `inputs[]` and `outputs[]` definitions.
- **Side Effects**: Declared `effects.writes[]` and `effects.fixed_writes[]`.
- **Reproducibility**: At least one functional `examples[].command`.
- **Typing and Compatibility**: `contracts.consumes` and `contracts.produces`, alongside optional `contract_outputs` and `docs[]` references.

For comprehensive field-level policies and boundaries, consult `docs/manifest_declarative_spec.md`.

17.5 Contracts Referenced by the Manifest

The manifest references stable JSON schemas located under `docs/schemas/`. Prominent schemas include:

- **Predictions**: `docs/schemas/predictions.schema.json`
- **Evaluation Suite Reports**: `docs/schemas/eval_suite_report.schema.json`
- **COCO Evaluation Reports**: `docs/schemas/coco_eval_report.schema.json`
- **Segmentation**: Schemas for both semantic and instance segmentation predictions and evaluation reports.

In a research context, these schemas are invaluable as they drastically reduce the overhead required to:

- Parse and aggregate results across numerous experimental runs.
- Construct dashboards and visualizations from JSONL histories.
- Implement assertions (regression gates) against strictly typed outputs.

17.6 The Discovery Loop

A recommended practical loop when onboarding to the repository (or returning after an absence):

1. Review the documentation index: `docs/README.md`.
2. Scan the tool index: `docs/tools_index.md`.
3. Query the manifest by `tags`, `id`, and `contracts`.
4. Execute the validator to ensure the registry remains trustworthy.

Practical Manifest Queries

```
# List all tool IDs and their corresponding entrypoints
python3 - <<'PY'
import json
with open("tools/manifest.json", "r", encoding="utf-8") as f:
    manifest = json.load(f)
for tool in manifest.get("tools", []):
    print(f"{tool.get('id')}\t{tool.get('entrypoint')}")
PY

# Identify tools that consume the 'predictions_json' contract
python3 - <<'PY'
import json
with open("tools/manifest.json", "r", encoding="utf-8") as f:
    manifest = json.load(f)
for tool in manifest.get("tools", []):
    consumes = (tool.get("contracts") or {}).get("consumes", [])
    if "predictions_json" in consumes:
        print(tool.get("id"))
PY
```

17.7 AI-First Entrypoints for Autonomous Agents

For autonomous or semi-autonomous operations, the recommended stable entrypoints are:

- **Canonical CLI**: `yolozu -help` (or `python3 -m yolozu -help`)
- **Tool Registry**: `tools/manifest.json`
- **Contracts and Schemas**: `docs/schemas/*.schema.json`
- **Usage Guide**: `docs/ai_first.md`

This represents an explicit design objective of YOLOZU: AI agents must be capable of discovering, validating, executing, and extending workflows utilizing stable, well-defined interface contracts.

17.8 When You Need to Change the Registry

Do not treat this chapter as the maintenance checklist. If you add a flag, change an output path, add a public tool, or alter manifest metadata, follow Chapter 18. That chapter is the canonical update procedure for:

- changing tool behavior and `-help` output,
- updating `tools/manifest.json` and the packaged manifest copy,
- syncing docs/manual examples, and
- running the manifest and docs-reference quality gates.

SynthGen intake tools (registry additions)

The registry now exposes explicit intake utilities for externally generated synthetic shards:

- `validate_synthgen_contract`: validate required SynthGen fields and tensor/range constraints.
- `render_synthgen_overlay`: generate semantic/instance/keypoint debug overlays.
- `eval_synthgen`: compute keypoint, segmentation, and depth summaries for SynthGen predictions.
- `smoke_synthgen`: run a deterministic interface-contract+overlay+eval smoke pass on `data/smoke/synthgen_minishard`.

These tools reference the canonical SynthGen interface contract at `docs/synthgen_contract.md` and `schemas/synthgen_sample.schema.json`.

TTT figure renderer (registry addition)

The registry also exposes `render_ttt_manual_figures`, which regenerates:

- `docs/assets/ttt_method_results_summary.png`
- `docs/assets/ttt_compare_pipeline.png`
- `docs/assets/ttt_probe_example_panel.png`

This keeps the TTT chapter's beginner-facing PNG figures reproducible and synchronized with fixed result artifacts rather than hand-edited screenshots.

17.9 Release Automation: Manual PDF DOI as a Separate Record

For release operations, YOLOZU intentionally keeps software DOI and manual DOI lifecycles independent:

- Software release and PyPI publish are handled by `.github/workflows/publish.yml`.
- Manual PDF DOI publishing is handled by `.github/workflows/manual_doi.yml`.

The software publish workflow now enforces release metadata consistency before PyPI publish:

- release-triggered runs validate that tag `vX.Y.Z` matches `yolozu/__init__.py`
- manual `workflow_dispatch` runs require `expected_version`
- optional manual input `release_tag` can re-check tag/version alignment during recovery
- `CHANGELOG.md` must contain a matching `#### [X.Y.Z] - YYYY-MM-DD` section

Operational helper script (no required options):

```
bash release.sh
```

Versioning policy is automatic:

- Current `X.Y.Z`: SemVer bump based on release size.
- Current `YYYY.MM.DD.MICRO`: CalVer bump with same-day micro increment and UTC-day rollover reset.

Dry-run preview:

```
bash release.sh --dry-run --allow-dirty --allow-non-main --output reports/
    release_report.dry_run.json
```

When operators need to recover a failed publish without cutting a new tag, the supported path is the `publish.yml` manual trigger with:

- `expected_version = X.Y.Z`
- optional `release_tag = vX.Y.Z`

This keeps the PyPI publish path explicit instead of allowing a loosely-scoped manual rerun.

The manual DOI workflow builds `manual/build/yolozu_manual.pdf` and publishes it to Zenodo either as:

- a new version (`actions/newversion`) when a manual `conceptrecid` is configured, or
- a first deposition when no prior concept record exists.

To preserve machine-readable linkage between records, manual metadata sets:

- `related_identifiers[].relation = isSupplementTo`
- `related_identifiers[].scheme = doi`
- `related_identifiers[].identifier` = software concept DOI

Operationally, configure repository variables:

- `YOLOZU_SOFTWARE_CONCEPT_DOI` (required)
- `YOLOZU_MANUAL_CONCEPTRECID` (recommended for versioned manual updates)

17.10 Path Resolution Behavior (Critical for Automation)

Many tools adhere to consistent path resolution rules (summarized in `docs/tools_index.md` and `docs/ai_first.md`):

- CLI-relative input paths resolve from the current working directory.
- Config-relative paths (where supported) resolve from the directory containing the configuration file.
- Relative outputs are written relative to the current working directory.

17.11 Safety Guardrails for Agentic Write Actions

Prior to writing artifacts or mutating state, agents and users should prioritize:

- Utilizing **dry-run modes** wherever available.
- Applying **explicit caps or ranges** (e.g., `-max-images`) to limit execution scope.
- Directing outputs to **deterministic locations**, such as under `reports/` or a dedicated run directory.

In a research environment, these practices ensure that batch operations run reliably across different working directories and enhance the portability of scripted executions.

Chapter 18

Manifest-Driven Operations and Manual Synchronization

This chapter establishes `tools/manifest.json` as the canonical operational source for commands, input/output (I/O) contracts, and write-side effects. It mitigates documentation drift by ensuring that manual updates remain strictly traceable to machine-readable tool metadata. Consult this chapter when introducing new tools, modifying CLI flags, or revising workflows that rely on specific artifact paths and contracts.

Who should read this chapter: Read this chapter when changing code and documentation together, especially if command surfaces, examples, or write effects are involved.

18.1 Relationship to the Tool Registry Chapter

Chapter 12 explains how users and automation discover declared tools from the registry. This chapter is different: it is the maintainer checklist for changing that registry. Use it when a code, CLI, manifest, docs, or manual change must land together in the same pull request.

18.2 The Necessity of a Dedicated Chapter

As the repository's tool surface expands, command examples embedded within narrative chapters are prone to becoming obsolete. A dedicated, manifest-driven workflow guarantees that:

- Command discovery originates from a single, authoritative source.
- I/O contracts and side effects undergo consistent, systematic review.
- Manual chapter revisions are executed synchronously with codebase modifications.

18.3 Canonical Sources and Hierarchy

When resolving discrepancies, adhere to the following priority hierarchy:

1. `tools/manifest.json` (The canonical, machine-readable tool metadata).
2. Tool `-help` output (The runtime source of truth for CLI flags).
3. Chapter prose and examples (Derived, human-readable documentation).

Relevant supplementary references include:

- `docs/manifest_declarative_spec.md`
- `docs/tools_index.md`
- `docs/production_readiness.md`
- `docs/evaluation_protocol_template.md`
- `docs/schema_governance.md`

- `docs/repo_governance_audit.md`
- `tools/validate_tool_manifest.py`

18.4 Mandatory Update Procedure

When altering tool behavior, you must follow this sequential procedure:

1. Implement the requisite code changes.
2. Update the corresponding tool entry within `tools/manifest.json`. Ensure that all new flags, inputs, and outputs are explicitly declared.
3. Validate the manifest structure and constraints:

```
python3 tools/validate_tool_manifest.py --manifest tools/manifest.json --require-
    declarative
```

4. Revise the impacted manual chapters located under `manual/chapters/`.
5. Cross-reference and verify all documentation examples against the manifest's `examples[].command` fields.

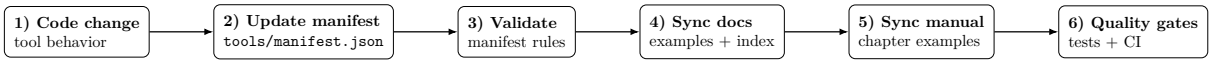


Figure 18.1: Manifest-driven update loop: `tools/manifest.json` is the canonical source, and docs/manual must follow it.

18.5 Verification Criteria for Manual Updates

For each affected command family, rigorously verify the following:

- The referenced command exists within the manifest under `id` and `entrypoint`.
- All required input flags accurately reflect the manifest's `inputs[]` array.
- Output declarations and write-path statements align with the `outputs[]` and `effects` definitions.
- Contract claims strictly match the `contracts.consumes` and `contracts.produces` specifications.

18.6 Understanding Manifest Capabilities

The manifest is not merely a list of scripts; it is a declarative governance tool. Chapter 12 covers day-to-day discovery. In this chapter, inspect `tools/manifest.json` to decide what must be updated when behavior changes:

- **Platform Compatibility**: The `platform` object indicates whether a tool requires a GPU (`gpu_required`) or supports macOS/Linux.
- **Dependencies**: The `requires` object lists necessary Python packages and system-level dependencies (e.g., TensorRT).
- **Data Contracts**: The `contracts` object defines what standard JSON schemas a tool consumes or produces (e.g., `predictions_json`, `metrics_report_json`), enabling automated pipeline construction.

18.7 Quick Audit Commands

To list all registered tools and their entrypoints:


```
python3 - <<'PY'
import json
with open("tools/manifest.json", "r", encoding="utf-8") as f:
    manifest = json.load(f)
for tool in manifest.get("tools", []):
    print(f"{tool.get('id')} -> {tool.get('entrypoint')}")
PY
```

To identify tools that produce a specific contract (e.g., `eval_suite_report_json`):

```
python3 - <<'PY'
import json
with open("tools/manifest.json", "r", encoding="utf-8") as f:
    manifest = json.load(f)
for tool in manifest.get("tools", []):
    produces = (tool.get("contracts") or {}).get("produces", [])
    if "eval_suite_report_json" in produces:
        print(tool.get("id"))
PY
```

18.8 Chapter Ownership and Routing

Utilize this heuristic mapping to determine which chapters require updates following a tool modification:

- General CLI changes: Chapter 4
- Training, backbone, or run-contract modifications: Chapter 7
- Evaluation protocols and benchmark adjustments: Chapter 9
- Test-Time Training (TTT) methods and benchmarks: Chapter 15
- Registry, contract metadata, and governance changes: Chapter 12 and this chapter

This mapping is intentionally lightweight. If a tool modification spans multiple domains, update all affected chapters within the same pull request to maintain documentation coherence.

18.9 Governance snapshots and external repository settings

Some operational guarantees cannot be derived from source code alone. Branch protection, review enforcement, and certain Scorecard/badge signals depend on GitHub settings or repository history.

To keep this boundary explicit, capture settings snapshots and audit them locally:

```
mkdir -p reports/github_governance
gh api repos/ToppyMicroServices/YOLOZU \
  > reports/github_governance/repo.json
gh api repos/ToppyMicroServices/YOLOZU/branches/main/protection \
  > reports/github_governance/branch_protection_main.json
python3 tools/check_repo_governance.py \
  --repo-json reports/github_governance/repo.json \
  --branch-protection-json reports/github_governance/branch_protection_main.json \
  --output reports/repo_governance_check.json
```

This keeps repository-local facts and maintainer-process facts separate:

- repository-local evidence: workflows, docs, fuzz harnesses, manifest coverage
- snapshot-backed settings evidence: branch protection and repository security settings
- manual followups: reviewed PR history, cadence-based maintenance, external badge enrollment

18.10 Ownership split: GitHub docs vs PDF manual

To avoid double maintenance:

- Keep `docs/README.md` as an **index/entry page** with links only.
- Keep procedural depth and operational nuance in chapter docs and `manual/chapters/`.
- When overlap is unavoidable, prefer a single canonical page and link to it from all other surfaces.

18.11 Technical credibility artifacts

Keep the technical credibility tables in the GitHub docs and link to them from the manual instead of copying the same matrix into multiple chapters:

- `docs/production_readiness.md`: version compatibility and production readiness matrices.
- `docs/evaluation_protocol_template.md`: reusable evaluation protocol template for benchmark or customer-facing reports.
- `docs/schema_governance.md`: schema browser coverage for predictions, dataset, training, SynthGen, and research artifacts.

Chapter 19

LLM MCP Integrations (Gemini / Claude / Copilot / OpenAI)

Scope: Unified MCP-first integration strategy across major LLM clients.

Who should read this chapter: Read this chapter when integrating YOLOZU with MCP clients, Actions APIs, or LLM-facing automation layers.

19.1 Principle: one backend, many clients

All clients should share the same implementation backend:

```
python3 tools/run_mcp_server.py
```

Core exposed tools:

- `doctor`
- `generate_config`
- `review_config`
- `validate_predictions`
- `validate_dataset`
- `eval_coco`
- `run_scenarios`
- `convert_dataset` (optional)

Return payload policy (for all clients):

- machine-readable JSON
- short textual summary in `summary`
- stable top-level keys: `ok`, `tool`, `summary`, `exit_code`
- optional metadata: `meta.git_sha`, `meta.manifest_sha256`, runtime details

19.2 Layer model (fixed)

1. Core layer: I/O envelope, auth/log hooks, error shaping.
2. YOLOZU API layer: CLI/Python wrapper, argument normalization, manifest resolution.
3. Job layer: long-running operations with `job_id` tracking.
4. Artifact layer: run artifacts and metadata return (git SHA / manifest hash / environment).

19.3 Quick start (MCP + Actions API)

19.3.1 MCP server

```
python3 tools/run_mcp_server.py
```

Official MCP Lite surface (deterministic, intended for AI-safe clients):

- `doctor`
- `generate_config`
- `review_config`

Copy-paste sanity checks:

```
python3 tools/run_mcp_server.py --print-tools > reports/mcp_tools.json
python3 tools/run_mcp_server.py --sample-generate-config > reports/ai_generate_config.
    json
python3 tools/run_mcp_server.py --sample-review-config reports/ai_generate_config.json
     > reports/ai_review_config.json
python3 tools/check_mcp_settings.py --output reports/mcp_settings_check.json
```

19.3.2 Actions API (optional route)

```
python3 tools/run_actions_api.py --host 127.0.0.1 --port 8080 --workers 1
curl -sS http://127.0.0.1:8080/healthz
```

19.3.3 Minimal end-to-end check

```
python3 -m yolozu doctor --output reports/doctor.json
python3 -m yolozu validate predictions data/smoke/predictions/predictions_dummy.json --
    strict
python3 -m yolozu eval-coco --dataset data/smoke --predictions data/smoke/predictions/
    predictions_dummy.json --dry-run --output reports/mcp_eval_coco_dry.json
```

19.4 Client routing

19.4.1 Gemini

Use MCP first. Optionally use API tool-calling via OpenAPI endpoint.

19.4.2 Claude

Use MCP first with thin prompt wrappers; avoid duplicating tool logic.

19.4.3 Copilot

Use Copilot Extensions / VS Code participant that calls MCP backend (or API fallback).

19.4.4 OpenAI

Use MCP first; add GPT Actions when OpenAPI registration is required.

Optional API route:

```
python3 tools/run_actions_api.py --host 127.0.0.1 --port 8080 --workers 1
# OpenAPI: http://127.0.0.1:8080/openapi.json
```

19.4.5 Ollama (local LLM)

Ollama runs models locally and can be used as the reasoning engine behind an MCP-capable client. The YOLOZU backend does not change: keep using the same YOLOZU MCP server.

Start (or confirm) Ollama is running:

```
ollama serve
ollama pull llama3.1
```

Then configure your client runtime to use Ollama as an **OpenAI-compatible** endpoint:

```
# Typical values (client-dependent)
OPENAI_BASE_URL=http://127.0.0.1:11434/v1
OPENAI_API_KEY=ollama

# Example model identifiers are Ollama-local names
OPENAI_MODEL=llama3.1
```

Optional sanity checks:

```
# Native Ollama API
curl -sS http://127.0.0.1:11434/api/tags

# OpenAI-compatible API
curl -sS http://127.0.0.1:11434/v1/models
```

Notes:

- MCP tool execution still routes to `python3 tools/run_mcp_server.py`; only the LLM provider/model changes.
- Environment variable names are client-specific; some clients use `OPENAI_API_BASE` instead of `OPENAI_BASE_URL`.
- Model capability varies by local weights; if tool selection is unstable, prefer a stronger local model and keep prompts/tool wrappers thin.
- Verify the endpoint is reachable via `http://127.0.0.1:11434/v1/models` (OpenAI-compatible) in environments that support it.

19.5 Operational recommendation

Maintain feature parity by implementing behavior once in backend runner and exposing it through MCP/API adapters.

Operational guards in v2 layer implementation:

- Allowlist is manifest-first (`tools/manifest.json`) with conservative fallback defaults.
- Path tokens are guarded (`..` rejected; absolute paths outside workspace rejected).
- Job state persistence to `runs/mcp_jobs/*.json` for restart-safe status retrieval.
- Stale in-flight states (queued/running after restart) are surfaced as `unknown`.
- CLI execution timeout and output-size caps with explicit truncation metadata.

19.6 Tool map (actual MCP/Actions surface)

19.6.1 Official support boundary (1.0.x)

- Guaranteed (deterministic/lightweight): `doctor`, `generate_config`, `review_config`, `validate_predicti`
- Best-effort (environment dependent): long training jobs, TensorRT build/export, OpenCV CUDA/OpenVINO backends

19.6.2 Synchronous tools

- Data/validation: `doctor`, `validate_predictions`, `validate_dataset`, `convert_dataset`
- Inference/calibration: `predict_images`, `parity_check`, `calibrate_predictions`
- Evaluation: `eval_coco`, `eval_instance_seg`, `eval_long_tail`, `run_scenarios`

19.6.3 Asynchronous job tools

- `train_job`, `export_predictions_job`, `test_job`, `ttt_job`, `ctta_job`
- compatibility alias: `export_onnx_job` (same behavior as `export_predictions_job`)
- control/query: `jobs.list`, `jobs.status`, `jobs.cancel`, `runs.list`, `runs.describe`

19.7 AI operation pitfalls (high-impact)

1. Always pass `dataset` as a YOLO root and specify `split` explicitly.
2. Prefer workspace-relative paths to avoid API-layer path guard rejections.
3. Treat `ok/tool/summary/exit_code` as the canonical status contract.
4. For long tasks, do not poll stdout; use `job_id` with `jobs.status`.
5. Assume post-restart in-flight jobs become `unknown`; requeue deterministically if needed.
6. Set `dry_run`/`strict`/`force` explicitly to avoid client-default drift.

19.8 Efficiency backlog for MCP hardening

High-value follow-ups that reduce AI ambiguity and integration drift:

- Generated MCP/Actions reference now lives at:
  - `docs/generated/mcp_actions_tool_reference.json`
  - `docs/generated/mcp_actions_tool_reference.md`
- Regenerate/check command: `python3 tools/generate_integration_tool_reference.py -check`
- Add explicit error-code taxonomy for allowlist/path/timeout failures in integration responses.

19.9 CI no-abandon policy

For MCP-related changes:

1. Run local validation (manifest + tests).
2. Push and monitor latest CI result.
3. If failed, patch immediately and rerun until green.

19.10 Cross references

- OpenAI details: `docs/openai_mcp_actions.md`
- Copilot details: `docs/copilot_mcp_integration.md`
- Shared overview: `docs/llm_integrations.md`

Chapter 20

SynthGen Repo Integration

This chapter defines the repo-to-repo handoff between an external generator such as `YOLOZU-synthgen` and YOLOZU. It keeps the integration boundary small: the generator owns rendering and shard creation, while YOLOZU owns validation, overlay inspection, and evaluation. The practical goal is to make a synthetic dataset handoff reproducible, CI-friendly, and easy to verify before training work begins.

Who should read this chapter: Read this chapter when you are connecting `YOLOZU-synthgen` or another external synthetic-data generator to the YOLOZU intake and evaluation path.

Operator opening.

Goal Validate and evaluate synthetic-data handoffs without letting generator details leak into YOLOZU's evaluation boundary.

When to use Use this when an external renderer or generator emits shards, assets, and predictions that YOLOZU must inspect and score.

Inputs A generated dataset root, shard JSONL files, asset paths that still resolve, and optional `predictions_synthgen.json`.

Command `python3 tools/smoke_synthgen.py ...` for the bundled check, or run validate, overlay render, and eval helpers separately.

Outputs Validation results, overlay PNGs, SynthGen eval JSON, summary JSON, and any failed sample diagnostics.

How to verify Open at least one overlay, confirm schema validation passed, and check that renderer-derived labels and asset paths are still consistent.

Common failure modes Broken relative paths, schema-version drift, generator-owned labels mixed with YOLOZU outputs, or treating appearance generation as ground truth.

Readiness classification. This chapter belongs to the **experimental** lane in YOLOZU. The intake and evaluation path is reproducible, but the external generator environment still needs qualification and path discipline before production use. See `docs/production_readiness.md`.

20.1 Boundary and ownership

The shared boundary is the SynthGen sample interface contract:

- Spec: `docs/synthgen_contract.md`
- JSON Schema: `schemas/synthgen_sample.schema.json`
- Runtime validator: `yolozu/contracts/synthgen.py`

Ownership split:

- External generator repo: prompts, asset selection, rendering, shard creation, generator metadata
- YOLOZU: interface-contract validation, shard loading, overlay rendering, evaluation, smoke checks

Each record in `shards/*.jsonl` must satisfy `schema_version="1"` and provide the required tensors and metadata fields used by the intake adapters.

20.2 One-page handoff order

1. Export a YOLOZU-compatible dataset root from the generator repo.
2. Keep shard rows and asset paths explicit.
3. Place `predictions_synthgen.json` where its relative paths still resolve.
4. Run `validate_synthgen_contract.py`.
5. Run `render_synthgen_overlay.py`.
6. Run `eval_synthgen.py`.
7. Run `smoke_synthgen.py` if you want one bundled acceptance probe.

20.3 How samples are generated in YOLOZU-synthgen

The generator is intentionally **split-responsibility**, not "gen AI does everything":

- Renderer-owned: geometry, object count, pose, camera, base lighting, depth, instance ids, semantic ids, keypoints, and the authoritative `scene_spec`
- Gen-AI-owned: recipe drafting assistance, appearance recipe drafting, appearance editing, visual QA assistance, and captions/metadata assistance

In other words, **ground truth is renderer-derived**; gen AI may improve realism or diversity, but it is not the source of labels.

20.3.1 Where CAD / mesh assets fit

Current generator rules allow object geometry to come from:

- `primitive_box` entries for early geometry bootstrapping
- `mesh_file` assets for real object geometry

This means CAD-derived assets fit naturally once they are exported into mesh files and referenced from the generator-side asset catalogs. The renderer still owns object pose, camera projection, depth, and id passes, so labels remain consistent even when the visual source is a CAD or mesh asset.

20.3.2 Where Generative AI fits

`YOLOZU-synthgen` is designed around **gen-AI-assisted appearance**, not text-to-image-as-ground-truth.

Supported modes are:

- `render_only`: the renderer output is the final image
- `bg_only_inpaint`: only the background outside the foreground union mask is edited
- `appearance_only_conditioned`: conditioned appearance editing may change object interiors while preserving object count and silhouette

Intentionally unsupported as the main labeled-data path:

- full image regeneration (`full_regen`)
- gen-AI-predicted masks or keypoints as ground truth
- text-to-3D as the main geometry source

This policy is what makes the downstream handoff to YOLOZU reliable: the final image may look more realistic, but labels still come from the renderer and remain tied to the same scene geometry.

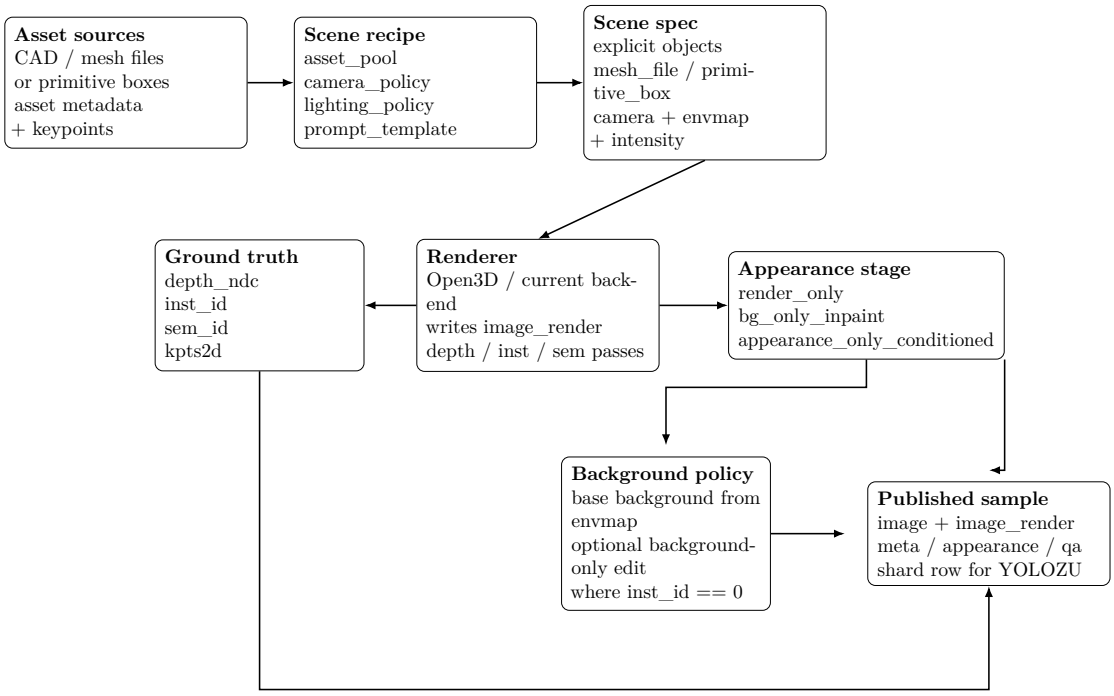


Figure 20.1: SynthGen path: assets become a renderer-ready scene, renderer outputs remain the authoritative source of labels, and appearance edits stay within label-safe stages.

20.4 System pipeline

The generator-side pipeline is:

1. Create `scene_recipe`
2. Materialize `scene_spec`
3. Render `image_render` plus ground-truth passes
4. Create `appearance_recipe`
5. Materialize `appearance_spec`
6. Apply appearance editing, if enabled by mode
7. Run QA gate
8. Emit accepted dataset sample or shard entry

The important handoff idea is that `scene_spec` is fully materialized before rendering, and the renderer-produced passes remain authoritative throughout the rest of the pipeline.

20.5 How images and labels are handled

20.5.1 Canonical sample fields

The generator runtime emits a canonical sample record with fields such as:

- `image`
- `image_source`
- `render_backend`
- `depth_ndc`
- `inst_id`
- `sem_id`
- `kpts2d`
- `prompt`
- `scene_spec`
- `appearance_spec`

- `qa_report`
- `schema_id`, `schema_version`
- `asset_ids`, `inst_map`

20.5.2 Image handling

There are two distinct image concepts:

- `image_render`: the direct renderer output before any optional appearance editing
- `image`: the final training/evaluation image after the selected mode has been applied

For `render_only`, `image == image_render`. For appearance-edited modes, `image` may differ visually, but the labels remain anchored to the renderer-owned geometry and passes.

20.5.3 Label handling

Labels are not hand-annotated after rendering and are not guessed from the final image. Instead:

- `depth_ndc` comes from the renderer depth pass
- `inst_id` comes from the renderer instance-id pass
- `sem_id` comes from the renderer semantic-id pass
- `kpts2d` is projected from object-space keypoints through the scene camera and then refined against depth and instance-id where available

This is why `YOLOZU-synthgen` can support synthetic depth/segmentation/keypoint workflows without a manual labeling stage: labels are generated from the same scene state that produced the render.

20.5.4 Per-sample files on disk

At shard-export time, each sample directory typically stores:

- `image.png`
- `depth_ndc.npy`
- `inst_id.npy`
- `sem_id.npy`
- `kpts2d.npy`
- `meta.json`
- `appearance.json`
- `qa.json`
- `image_render.png` when the final image is not `render_only`

When exported for YOLOZU intake, these assets are summarized into shard rows plus referenced sidecar files. YOLOZU then validates and loads them through the SynthGen sample interface contract instead of re-deriving labels itself.

20.6 How backgrounds are generated and selected

20.6.1 Base background at render time

The base background is not chosen by a free-form text-to-image model. It is first determined by the renderer-side scene settings:

- `scene_recipe.lighting_policy.envmap_ids` provides the candidate environment maps
- the compiler samples one `envmap_id` from that list
- `scene_recipe.lighting_policy.env_intensity_range` controls the sampled background / ambient intensity

So the first background decision is a **lighting/environment selection problem**, not a relabeling problem. This keeps the object labels stable because the background is chosen before the authoritative render passes are produced.

20.6.2 Background choice policy

Background selection is currently policy-driven through the recipe:

- choose the allowed `envmap_ids` set for the target domain
- control brightness variation with `env_intensity_range`
- keep camera and object placement within recipe bounds so the chosen background remains compatible with the target task

If the goal is domain coverage, the practical lever is to diversify the recipe's environment-map pool and its difficulty/coverage tags, rather than letting an unconstrained model hallucinate arbitrary backgrounds.

20.6.3 Optional background editing

After rendering, `YOLOZU-synthgen` may optionally modify the image appearance using `appearance_spec`. For background-safe generation the important mode is:

- `bg_only_inpaint`: edit only the background, leaving labeled object regions untouched

The current contract for `appearance_spec` makes this explicit:

- `mode = bg_only_inpaint`
- `mask_scope = background_only`
- controls can include renderer-derived guides such as `depth_ndc`, `inst_boundary`, and `foreground_union_mask`

Operationally, the background edit should be guided by renderer-owned masks. In the current background-only implementation, this is approximated by changing pixels where `inst_id == 0`. In a fuller provider-backed pipeline, the same rule would be enforced with masks and QA checks rather than with direct pixel loops.

20.6.4 What is not allowed

The background stage must not:

- add or remove labeled objects
- cross object boundaries and change silhouettes
- become the source of new `inst_id`, `sem_id`, or `kpts2d`

That is the key safety rule behind the whole integration: backgrounds may vary, but labels do not drift with them.

20.7 Recommended dataset root

```
<dataset-root>/
  shards/
    train_000.jsonl
    predictions_synthgen.json
  <sample>_image.png
  <sample>_depth.npy
  <sample>_inst.npy
  <sample>_sem.npy
  <sample>_kpts.npy
```

Practical note: if the predictions artifact reuses shard-relative asset paths such as `../samples/...`, keep that predictions JSON under `shards/` so the relative base remains the same as the shard rows. If you prefer to store predictions elsewhere, rewrite those asset paths first.

This makes the path rule explicit:

- safest default: keep `predictions_synthgen.json` under `shards/`
- move it elsewhere only after rewriting relative asset paths and verifying one overlay plus one eval run

20.8 Verified cross-repo probe

This handoff was exercised end to end on April 2, 2026 using a local `YOLOZU-synthgen` checkout and the CPU path. GPU or MPS support is not required for these intake checks.

20.8.1 Generator-side commands

```
PYTHONPATH=src ./.venv/bin/python -m yolozu_synthgen generate-demo-dataset \
  --output-dir /tmp/yolozu_synthgen_demo \
  --num-train 2 \
  --num-val 1

PYTHONPATH=src ./.venv/bin/python -m yolozu_synthgen export-yolozu-synthgen \
  --shards-root /tmp/yolozu_synthgen_demo/shards \
  --output-dir /tmp/yolozu_synthgen_export
```

20.8.2 YOLOZU-side acceptance checks

```
python3 tools/validate_synthgen_contract.py \
  --input /tmp/yolozu_synthgen_export/shards/train_000.jsonl \
  --max-samples 20

python3 tools/render_synthgen_overlay.py \
  --dataset-root /tmp/yolozu_synthgen_export \
  --schema-id animal_v1 \
  --sample-index 0 \
  --output /tmp/yolozu_synthgen_overlay.png

python3 tools/eval_synthgen.py \
  --dataset-root /tmp/yolozu_synthgen_export \
  --predictions /tmp/yolozu_synthgen_export/shards/predictions_synthgen.json \
  --schema-id animal_v1 \
  --output /tmp/yolozu_synthgen_eval.json

python3 tools/smoke_synthgen.py \
  --dataset-root /tmp/yolozu_synthgen_export \
  --predictions /tmp/yolozu_synthgen_export/shards/predictions_synthgen.json \
  --output-dir /tmp/yolozu_synthgen_smoke
```

Recommended operator reading order:

1. `validate_synthgen_contract.py`
2. `render_synthgen_overlay.py`
3. `eval_synthgen.py`
4. `smoke_synthgen.py`

20.9 Acceptance criteria

The handoff is ready when all of the following are true:

- `validate_synthgen_contract.py` returns `OK`
- `render_synthgen_overlay.py` writes a non-empty overlay image
- `eval_synthgen.py` writes a report JSON without schema or shape errors
- `smoke_synthgen.py` writes overlay, eval, and summary artifacts under the chosen output directory

20.10 Versioning rule

- Add optional fields freely.
- Add new `schema_id` values freely.
- Do not change required dtype, shape, or range semantics inside `schema_version="1"`.
- If the generator needs a breaking change, version the interface contract in YOLOZU first, then enable it in adapters and docs.

Appendix A

Appendix: Code Graphs (Entry Points and Imports)

This appendix provides two views of the repository's Python code relationships: (1) a human-oriented entry-point map, and (2) an auto-extracted import dependency graph (internal modules only). The goal is to reduce onboarding cost by making the "where does this run from" and "what depends on what" structure visible in the PDF. The import graphs are generated by `tools/generate_manual_import_graphs.py`. Graphviz is optional: the manual renders graphs via TikZ so the build does not require `dot`.

Who should read this chapter: Read this appendix when you need a structural map of the codebase for onboarding, debugging, or agentic navigation.

A.1 Repository entry points (high level)

This diagram focuses on how users (and agents) typically enter the system, how execution flows into adapters/runtimes, and which contracts/artifacts are emitted.

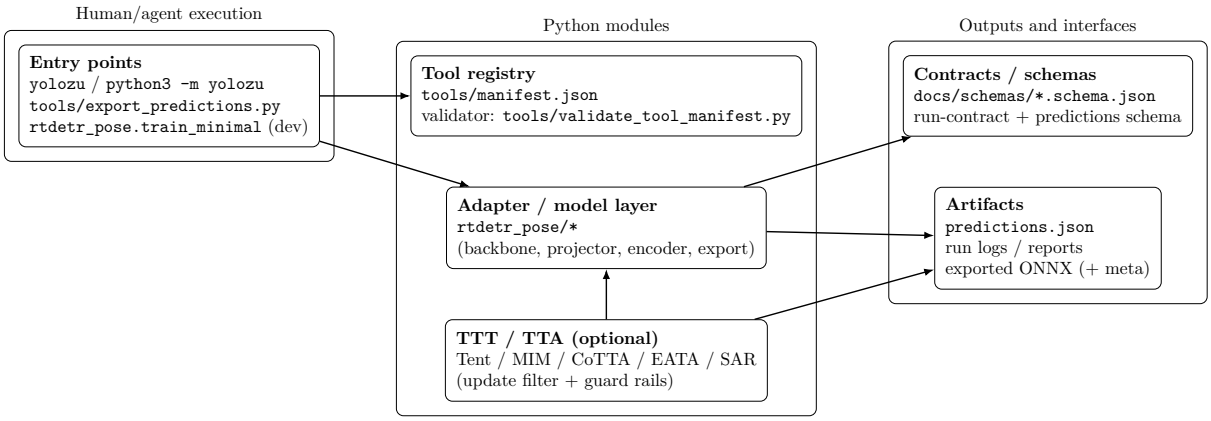


Figure A.1: Entry-point map and major code layers.

A.2 Python import dependency graph (internal)

The following graph is auto-extracted from Python `import` statements. It is intentionally restricted to internal modules and summarized at the package level to keep the PDF readable.

Arrow semantics: an edge $A \rightarrow B$ means A **imports** B (i.e., A depends on B).

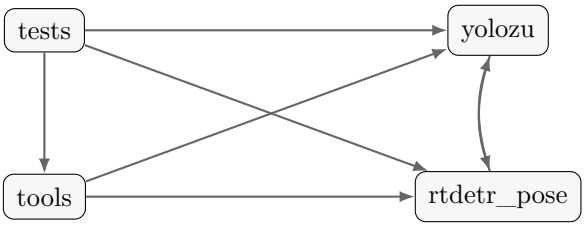


Figure A.2: Internal Python import dependencies (package-level summary).

A.2.1 Graph source (DOT)

The DOT source used to generate the package-level dependency summary is included for auditability:

```
digraph import_deps {
  rankdir=LR;
  bgcolor="transparent";
  node [shape=box, style="rounded,filled", fillcolor="#f7f7f7", color="#333333",
    fontname="Helvetica"];
  edge [color="#555555", penwidth=1.2, arrowsize=0.8];
  "rtdetr_pose";
  "tests";
  "tools";
  "yolozu";
  "rtdetr_pose" -> "yolozu";
  "tests" -> "rtdetr_pose";
  "tests" -> "tools";
  "tests" -> "yolozu";
  "tools" -> "rtdetr_pose";
  "tools" -> "yolozu";
  "yolozu" -> "rtdetr_pose";
}
```

References

[1] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms, 2023.

[2] David Eigen, Christian Puhrsch, and Rob Fergus. Depth map prediction from a single image using a multi-scale deep network. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.

[3] Tommaso Furlanello, Zachary C. Lipton, Michael Tschannen, Laurent Itti, and Anima Anandkumar. Born again neural networks, 2018.

[4] Yossi Gandelsman, Yu Sun, Xinlei Chen, and Alexei A. Efros. Test-time training with masked autoencoders. *CoRR*, abs/2209.07522, 2022.

[5] Yves Grandvalet and Yoshua Bengio. Semi-supervised learning by entropy minimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2005.

[6] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning (ICML)*, 2017.

[7] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[8] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network, 2015.

[9] Tomáš Hodan, Jiří Matas, and ˇ Sárka Obdržálek. On evaluation of 6d object pose estimation. In *European Conference on Computer Vision Workshops (ECCVW)*, 2016.

[10] Tomáš Hodan, Martin Sundermeyer, Bertram Drost, Yann Labbé, Eric Brachmann, Benjamin Kuan, Wadim Kehl, A. Glent Buchanan, Jiří Matas, et al. Bop challenge 2020 on 6d object localization. In *European Conference on Computer Vision Workshops (ECCVW)*, 2020.

[11] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.

[12] Yann Labbé, Justin Carpentier, Mathieu Aubry, Josef Sivic, and Ivan Laptev. Cosypose: Consistent multi-view 6d pose estimation for real-world objects. In *European Conference on Computer Vision (ECCV)*, 2020.

[13] Kenneth Levenberg. A method for the solution of certain non-linear problems in least squares. *Quarterly of Applied Mathematics*, 2(2):164–168, 1944.

[14] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft coco: Common objects in context. In *European Conference on Computer Vision (ECCV)*, 2014.

[15] Youssef Mansour, Xuyang Zhong, Serdar Caglar, and Reinhard Heckel. TTT-MIM: Test-time training with masked image modeling for denoising distribution shifts. In *European Conference on Computer Vision (ECCV)*, 2024.

[16] Donald W. Marquardt. An algorithm for least-squares estimation of nonlinear parameters. *SIAM Journal on Applied Mathematics*, 11(2):431–441, 1963.

[17] Aditya K. Menon, Sadeep Jayasumana, Ankit Singh Rawat, Himanshu Jain, Andreas Veit, and Sanjiv Kumar. Long-tail learning via logit adjustment. In *International Conference on Learning Representations (ICLR)*, 2021.

[18] Microsoft. Onnx runtime. https://onnxruntime.ai/. Accessed: 2026-02-19.

[19] NVIDIA. Nvidia tensorrt. https://developer.nvidia.com/tensorrt. Accessed: 2026-02-19.

[20] Deepak Pathak, Philipp Krähenbühl, Jeff Donahue, Trevor Darrell, and Alexei A. Efros. Context encoders: Feature learning by inpainting. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[21] PyTorch Contributors. Pytorch. https://pytorch.org/. Accessed: 2026-02-19.

[22] René Ranftl, Alexey Bochkovskiy, and Vladlen Koltun. Vision transformers for dense prediction. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

[23] Idan Shenfeld, Mehul Damani, Jonas Hübotter, and Pulkit Agrawal. Self-distillation enables continual learning. https://arxiv.org/abs/2601.19897, 2026.

[24] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *International Conference on Machine Learning (ICML)*, 2020.

[25] Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. Extracting and composing robust features with denoising autoencoders. In *International Conference on Machine Learning (ICML)*, 2008.

[26] Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. Tent: Fully test-time adaptation by entropy minimization. In *International Conference on Learning Representations (ICLR)*, 2021.

[27] Zhenda Xie, Chunyu Lin, Yuxin Bao, Ping Zhang, Jian Sun, and Han Hu. SimMIM: A simple framework for masked image modeling. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[28] Yi Zhou, Connelly Barnes, Jingwan Lu, Jimei Yang, and Hao Li. On the continuity of rotation representations in neural networks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

